



PROJECT INDUSTRY PPG: RECOVERABLE ASSETS (RA) & INVENTORY
RISK MANAGEMENT (IRM)

LEE YIN SHEN (A23CS0236)

POH LOK YEE (A23CS0262)

NUR FIRZANA BINTI BADRUS HISHAM (A23CS0156)

RICHARD YONATHAN JULIO CLAY HENG (X25EC3019)

SYARIFAH DANIA BINTI SYED ABU BAKAR (A23CS0183)

MUHAMMAD AFIQ DANISH BIN MOHD HAZNI (A23CS0118)

FACULTY OF COMPUTING
UNIVERSITI TEKNOLOGI MALAYSIA

JUN 2026

ACKNOWLEDGEMENT

First and foremost, we would like to express our sincere gratitude to Universiti Teknologi Malaysia (UTM) and the Faculty of Computing for providing us with the opportunity to carry out this project as part of our academic learning and practical development experience.

We would also like to extend our appreciation to our lecturer and project supervisor for the guidance, advice, and continuous support provided throughout the completion of this project. The feedback and suggestions received helped us improve our understanding of data engineering concepts, cloud-based pipeline development, system design, dimensional modelling, business intelligence reporting, and project documentation.

Our appreciation is also extended to PPG for providing the industry case study context and business problem related to Recoverable Assets (RA) and Inventory Risk Management (IRM). This case study allowed us to understand how data engineering solutions can be applied to real industry challenges, especially in inventory risk analysis, financial provision monitoring, stockout risk detection, and customer sales order impact tracing.

In addition, we would like to thank all group members for their cooperation, commitment, and contribution throughout the project. Each member contributed to different parts of the system, including Bronze Layer data ingestion and quality checking, Silver Layer business logic transformation, Gold Layer galaxy schema development, Power BI dashboard creation, testing, and report documentation. The teamwork and shared responsibility among group members were important in ensuring the successful completion of the project.

Finally, we would like to thank our classmates, friends, and all individuals who directly or indirectly supported us during the development of this project. Their assistance, encouragement, and feedback were greatly appreciated.

ABSTRACT

This project focuses on the development of an end-to-end data engineering pipeline for PPG Recoverable Assets (RA) and Inventory Risk Management (IRM). The system was designed to address inventory visibility issues, financial write-down risks, stockout risks, and possible customer sales order impacts. Six synthetic operational CSV datasets were used in this project, consisting of materials, inventory snapshots, purchase orders, production orders, suppliers, and sales orders. The pipeline was developed using Microsoft Azure services based on the Medallion Architecture, which consists of Bronze, Silver, and Gold layers.

Azure Data Factory was used to ingest the source datasets into Azure Data Lake Storage Gen2, where the data was organized into raw, validated, and analytics-ready storage layers. Azure Synapse Serverless SQL was used to perform the Silver Layer transformations because the planned Spark-based execution could not be used due to Azure vCore quota limitations. In the Silver Layer, business logic was applied to identify materials below reorder level, Recoverable Assets, Magna Carta Dead Stock, Magna Carta Expired materials, stockout-risk materials, and impacted customer sales orders. The Gold Layer was designed using a galaxy schema, consisting of shared dimension views and two fact views, namely `fact_inventory_status` and `fact_sales_impact`. This structure supports both inventory risk analysis and sales order impact analysis.

The final outputs were connected to Power BI to provide interactive dashboards for inventory health, financial provision, and operational risk analysis. Based on the final results, the system identified 10 Recoverable Assets, 1 MC Dead Stock material, 2 MC Expired materials, USD 30,144 in total MC provision, 5 materials below reorder level, 2 materials at stockout risk, 13 impacted sales order-material records, and 8 affected customers. Overall, the developed Azure-based pipeline successfully transformed raw operational data into reliable business insights for inventory risk monitoring and decision support.

TABLE OF CONTENTS

ACKNOWLEDGEMENT	i
ABSTRACT.....	ii
LIST OF FIGURES	v
LIST OF TABLE	ix
CHAPTER 1	1
1.1 Introduction	1
1.2 Problem Background.....	1
1.3 Project Aim	2
1.4 Project Objectives	2
1.5 Project Scope.....	3
1.6 Project Importance	3
1.7 Report Organization.....	4
CHAPTER 2	5
2.1 Introduction.....	5
2.2 Related Previous Study	5
2.3 Related Theory Used.....	6
2.3.1 Medallion data Architecture.....	6
2.3.2 Supply Chain Theory: Inventory Obsolescence and Risk Management.....	6
2.3.3 Enterprise Resource Planning (ERP) Data Integration.....	7
2.3.4 Predictive Stockout and Fulfillment Theory (Operational Theory)	8
2.3.5 Extract-Transform-Load/Extract-Load-Transform	8
2.4 Related Technology Used	9
2.4.1 Microsoft Azure	9
2.4.2 Microsoft Power BI	10
2.5 Summary	10
CHAPTER 3	13
3.1 Introduction.....	13
3.2 Introduction of Methodology	13
3.3 Phases of Methodology	14
3.3.1 Planning Phase.....	15
3.3.2 Analysis Phase.....	15

3.3.3 Design Phase.....	17
3.3.3.2 Galaxy Schema Data Model Design	18
3.3.3.3 Power BI Analytics Dashboard Design.....	20
3.3.4 Implementation Phase	20
3.4 Project Planning Schedule.....	24
3.5 Summary	26
CHAPTER 4	27
4.1 Introduction	27
4.2 System Analysis	27
4.2.1 Case Study	28
4.2.2 Company Organization Structure	28
4.2.3 Company Requirements Gathering Techniques	29
4.2.4 Use Case Diagram	31
4.2.5 Entity Relationships Diagram.....	32
4.3 System Requirements.....	34
4.3.1 Functional Requirements.....	34
4.3.2 Non-Functional Requirements	36
4.4 System Design.....	36
4.4.1 Overview of the Proposed System Design	37
4.4.2 Data Architecture	37
4.4.3 Explanation of Proposed Data Architecture	40
4.4.4 Scheduling and Error Handling.....	42
4.4.5 Database Design	42
CHAPTER 5	47
5.1 Introduction	47
5.2 System Development.....	47
5.2.1 Bronze Layer	48
5.2.2 Silver Layer — RA/MC Logic	49
5.2.3 Silver Layer — Stockout & Sales Impact	51
5.2.4 Gold Layer – Galaxy Schema.....	53
5.2.5 Power BI Dashboard.....	55
5.3 Coding of system’s main functions.....	55
5.3.1 Bronze Ingestion & Quality Check	55
5.3.2 RA & MC Business Logic.....	63

5.3.3 Stockout & Sales Order Impact Logic.....	67
5.3.4 Galaxy Schema & Fact Table Loading.....	70
5.3.5 Power BI Data Model and Dashboard Measures.....	77
5.3.6 Master Pipeline Orchestration	81
5.4 Essential Interfaces.....	82
5.4.1 Bronze Pipeline Results.....	82
5.4.2 Silver RA/MC Query Results	84
5.4.3 Silver Stockout Query Results.....	86
5.4.4 Gold Layer Tables	88
5.4.5 Power BI Dashboard.....	90
5.4.6 Master Pipeline Execution Results	93
5.5 Testing.....	93
5.5.1 Black Box Testing	94
5.5.2 White Box Testing.....	100
5.5.3 User Testing.....	111
5.6 Summary	114
REFERENCES	116

LIST OF FIGURES

Figure 3.2.1: Waterfall Phases	14
Figure 3.3.3.1: Medallion Architecture Design of PPG RA and IRM System	18
Figure 3.3.3.2: Conceptual Galaxy Schema Design of PPG RA and IRM System	19
Figure 3.4.1: Gantt Chart of the Project	24
Figure 4.2.4.1: Use Case Diagram of PPG RA and IRM System	31
Figure 4.2.5: Entity Relationship Diagram	32
Figure 4.4.2.1: Proposed Azure Data Architecture for PPG Inventory Risk Analytics	38
Figure 4.4.5.1: Galaxy Schema of PPG RA and IRM System	44
Figure 5.2.1.1: Azure Resource Group rg-ppg-pipeline	48
Figure 5.2.1.2: ADLS Gen2 Containers	49
Figure 5.2.1.3: Configured Linked Services in Azure Data Factory	49
Figure 5.2.2.1: SQL view creation for vw_silver_rc_mc_logic	50
Figure 5.2.3.1: SQL view creation for vw_silver_stockout_risk	52
Figure 5.2.3.2: SQL view creation for vw_silver_impacted_sales_orders	53
Figure 5.3.1.1.1: Six Source CSV Files in Raw Directory	57
Figure 5.3.1.2.1: Bronze Ingestion Pipeline	58
Figure 5.3.1.2.2: Six Source CSV Files in Ingested Directory	58
Figure 5.3.1.3.1: Data Flow Canvas for Quality Flag	60
Figure 5.3.1.3.2: Data Flow Canvas for Duplicate Detection	61
Figure 5.3.1.3.3: Validated Ouput Files in Bronze Container	62
Figure 5.3.1.3.4: Quarantined Output Files in Bronze Container	62
Figure 5.3.2.1: Latest inventory selection SQL logic	63
Figure 5.3.3.2 12-month consumption calculation SQL logic	63
Figure 5.3.2.3 Months since consumed calculation SQL logic	64
Figure 5.3.2.4 Recoverable Asset flag SQL logic	65

Figure 5.3.2.5: Q3 MC Dead Stock flag SQL logic.....	65
Figure 5.3.2.6: Q4 MC Expired flag calculation	66
Figure 5.3.2.7: Q5 composite MC flag and provision amount calculation.....	66
Figure 5.3.2.8 Final SELECT JOIN structure and filter	67
Figure 5.3.3.1: Stockout risk calculation SQL logic.....	68
Figure 5.3.3.2: Impacted sales order tracing SQL logic	69
Figure 5.3.4.1: Material Dimension View SQL Scripts.....	71
Figure 5.3.4.2: Supplier Dimension View SQL Scripts.....	72
Figure 5.3.4.3: Customer Dimension View SQL Scripts.....	72
Figure 5.3.4.4: Warehouse Dimension View SQL Script.....	73
Figure 5.3.4.5: Date Dimension View SQL Scripts.....	74
Figure 5.3.4.6: Central Fact View (fact_inventory_status) SQL Scripts.....	75
Figure 5.3.4.7: Central Fact View (fact_sales_impact) SQL Scripts.....	77
Figure 5.3.6.1 Master Pipeline of PPG RA and IRM System.....	82
Figure 5.4.1.1.1: Execution Result of the Pipeline	83
Figure 5.4.1.2.1: Data Flow Execution Details.....	83
Figure 5.4.2.1: Recoverable Assets query result.....	84
Figure 5.4.2.2: MC Dead Stock query result	84
Figure 5.4.2.3: MC Expired query result	85
Figure 5.4.2.4: Total MC provision query result	85
Figure 5.4.3.1: Query result for materials below reorder level.....	86
Figure 5.4.3.2: Query result for materials at stockout risk	87
Figure 5.4.3.3: Query Result for Impacted Sales Orders	87
Figure 5.4.3.4 Query Result for Affected Customer Summary	88
Figure 5.4.5.1: Power BI Dashboard, Inventory Health Page (Q1 to Q5)	92
Figure 5.4.5.2: Power BI Dashboard, Operational Risk Page (Q6 and Q7)	93
Figure 5.4.6.1: Master Pipeline Execution Result	93
Figure 5.5.1.2.1: Final quick check count results for Q2, Q3, Q4 and Q5	96

Figure 5.5.1.4.1: SQL Scripts of Galaxy Schema Validation..... 98

Figure 5.5.1.4.2: Result of Galaxy Schema Validation 98

Figure 5.5.2.1.1: Null Detection Expression Syntax in the QualityFlagMaterials Derived Column..... 104

Figure 5.5.2.2.1: White Box Testing Verification Output 105

Figure 5.5.2.3.1: Final Quick Check Results for Stockout and Sales Impact Views..... 107

LIST OF TABLES

Table 4.2.5: ERD Relationship Justification.....	33
Table 4.4.2.1: Components of the Proposed Data Architecture	38
Table 5.4.2.1: Gold Layer Overview	54
Table 5.3.1.1.1 Source CSV File in Bronze Container	56
Table 5.3.1.3.1 Primary Keys Used for Duplicate Detection	60
Table 5.4.4.1 Gold Layer View Summary	88
Table 5.4.4.2 fact_inventory_status Column References	89
Table 5.4.4.3 fact_sales_impact Column References	89
Table 5.5.1.1: Black Box Test Cases for Bronze Ingestion and Quality Check	94
Table 5.5.1.2.1 Black Box Testing Results for RA and MC Provision Component	96
Table 5.5.1.3.1 Black Box Testing Results for Stockout and Sales Impact	97
Table 5.5.1.4.3: Dimensions and Fact Record Count Tests.....	98
Table 5.5.1.5: Black Box Testing Results for Power BI Dashboard Output	99
Table 5.5.2.1.1: White Box Test Cases for Bronze Ingestion and Quality Check.....	101
Table 5.5.2.2.1: White Box Testing Results for RA and MC Provision Logic	105
Table 5.5.2.3.1: White Box Testing Results for Stockout Risk and Sales Impact Logic	107
Table 5.5.2.4.1: White Test Cases in Gold Layer Star Schema.....	108
Table 5.5.2.5.1: White Box Testing Results for Power BI Data Model and DAX Logic	110

LIST OF ABBREVIATIONS

ADLS	-	Azure Data Lake Storage
ADF	-	Azure Data Factory
CSV	-	Comma-Separated Value
DAX	-	Data Analysis Expressions
ELT	-	Extract-Load-Transform
FR	-	Functional Requirement
IRM	-	Inventory Risk Management
KPI	-	Key Performance Indicator
MC	-	Magna Carta
NFR	-	Non-Functional Requirement
RA	-	Recoverable Asset
SQL	-	Structured Query Language

CHAPTER 1

INTRODUCTION

1.1 Introduction

In the modern manufacturing landscape, efficient inventory management is a critical driver of financial stability and operational success. This project focuses on the development of an end-to-end data engineering pipeline for PPG, specifically designed to address Recoverable Assets (RA) and Inventory Risk Management (IRM). By utilizing a fully synthetic dataset comprising six operational files, including materials, inventory snapshots and sales orders, this initiative seeks to automate the identification of inventory health. By implementing a three-layer Medallion Architecture (Bronze, Silver, and Gold) within the Microsoft Azure ecosystem, the project transforms raw operational data into actionable business intelligence. The ultimate goal is to provide a centralized analytics platform that empowers stakeholders to optimize stock levels, minimize financial waste and ensure customer satisfaction through data-driven insights.

1.2 Problem Background

Global organizations like PPG face significant challenges in maintaining visibility over vast inventories of raw materials. Without an automated pipeline, it is difficult to identify “Recoverable Assets” which are materials held in excess of 12 months of consumption, leading to inefficient capital allocation. Furthermore, the lack of a standardized system to flag Magna Charta (MC) materials specifically “Dead Stock” with no consumption for nine months or materials that have surpassed their expiry dates presents a major financial risk. These materials often require a 100% financial write-down. Yet without precise tracking, the total provision amount remains unknown and unquantified.

Beyond financial reporting, operational risks such as stockouts pose a direct threat to the supply chain. Currently, the inability to accurately identify materials running lower than their lead time makes it nearly impossible to predict which production orders will fail or which open customer sales orders will be delayed. Manually reconciling these disparate

data sources is prone to error and time-consuming. Therefore, there is a critical need for a robust, automated data engineering solution that can ingest raw data, apply complex business logic and visualize inventory risks in real-time to prevent production failures and financial losses.

1.3 Project Aim

The main objective of this research project is to design and implement a scalable and end-to-end data engineering pipeline with the Medallion Architecture deployed in the Microsoft Azure platform (Codiin., 2024). This system is designed to automate the inventory obsolescence risk identification and quantification of financial provision for Recoverable Assets (RA) and Magna Carta (MC) materials. In addition, a centralized analytical framework is created to assess the effect of the material shortage on customer orders via high-fidelity Power BI dashboard.

1.4 Project Objectives

The following technical objectives were identified to achieve the project aim:

- **Ingestion and Data Quality Assurance:** A strong ingestion mechanism is created to pull data from six different source files into the Azure Data Lake (Bronze Layer), and data quality is thoroughly checked by automated methods such as null detection and type validation.
- **Business Logic Implementation:** Complex inventory logic is developed and built into the Silver Layer to calculate 12-month consumption rates and systematically mark up materials as Recoverable Assets, Dead Stock or Expired based on industry defined PPG criteria.
- **Galaxy Schema Optimization:** Standardized data warehouse model is built in the Gold Layer and transformed datasets are loaded into the central layer table to support high-performance analytical query.
- **Predictive Risk Assessment:** Analytical procedures are carried out to determine if raw materials are at risk of being out of stock based on the current inventory and lead time, then the effect on the open customer sales order is also tracked.

- **Interactive Visualization:** An enterprise-level Power BI dashboard is created to give stakeholders a clear view of the total amount of financial write downs (Magna Carta provisions) and to answer critical operational questions about the level of materials to order and the impact on customers.

1.5 Project Scope

This project aims to design and implement an end-to-end data engineering pipeline on Microsoft Azure platform that processes PPG's operational inventory data and produces structured analytical outputs for visualization. The final outputs focused on answering seven defined business questions related to inventory risk, financial provision, and customer order impact.

This project covers the processing of six static CSV datasets representing PPG's core operational data consisting of materials, inventory snapshots, purchase orders, production orders, suppliers, and sales orders. All processing is carried out within a standalone Microsoft Azure environment spanning the Medallion architecture with Bronze, Silver, and Gold pipeline layers, and the Power BI as the final reporting interface.

This project is only apply batch data processing using static provided datasets and does not include real-time data streaming, predictive analytics, or any artificial intelligence techniques. It also does not integrate with external enterprise systems, as the project uses synthetic data in a standalone environment only.

1.6 Project Importance

Manufacturing companies like PPG need to handle large volumes of raw materials across multiple warehouses in different locations daily which makes it difficult to detect inventory risks manually. Without a structured system, the inventory risks will remain hidden within disconnected data storage while expired and idle materials continue to occupy warehouse space and tie up working capital without being written down. Moreover, the risks of stockout will go undetected until production is already disrupted, and the financial impact of excess inventory only discovered during periodic audits rather than in time for corrective action.

This project reduces the time and effort to detect the problematic materials by automating the detection of inventory risks across the pipeline. It provides decision makers with structured, validated outputs rather than manual spreadsheet reviews to improve both the speed and accuracy of inventory risk responses across the organization. Aside from operational efficiency, the pipeline also ensures that financial risk from excess and expired stock are identified consistently and early, which giving the visibility to manage and perform needed action before losses accumulate.

The primary beneficiaries are PPG's inventory and finance teams who require accurate and timely data to perform on stock redistribution, financial write-downs, and procurement decisions. Operations and customer service teams also benefit through improved visibility into whether customer orders may be affected by material shortages and conditions.

1.7 Report Organization

The report was organized into several chapters to guide from the problem context to the final outcomes of the project. Chapter 1 introduces the overall background about of inventory risk at PPG, outlines problem statement, and presents the project aim, objectives, scope, and importance, focus on why an end-to-end data engineering pipeline is needed for Recoverable Assets (RA) and Inventory Risk Management (IRM). Chapter 2 discusses the literature review, covering previous studies, key data engineering concepts such as the Medallion Architecture, inventory obsolescence theories including RA and Magna Carta classifications, and the main technologies used in this work, namely Microsoft Azure and Power BI. Chapter 3 explains the methodology and system design, describing the data sources, Bronze–Silver–Gold pipeline, data quality checks, transformation logic, and how PPG’s business rules are implemented across the layers. Chapter 4 presents the system analysis and design, including the company organization structure, requirements gathering techniques, usecase diagram, functional and non-functional requirements, proposed data architecture, and the database design utilizing a galaxy schema. Chapter 5 discusses the system development and testing, covering the implementation of the Bronze, Silver, and Gold layers, the main coding functions and essential interfaces, the Power BI dashboard deployment, and the validation results from black box, white box, and user testing.

CHAPTER 2

LITERATURE REVIEW

2.1 Introduction

When companies in manufacturing start generating more data than they can manually make sense of, the gap between raw numbers and real decisions grows fast. This chapter pulls together the academic groundwork that supports the PPG Recoverable Assets (RA) and Inventory Risk Management (IRM) system — covering previous research, core theories, and the technologies that make the whole thing work. The goal here is not just to list references, but to show why each theoretical and technical choice actually fits the problem at hand.

This chapter studies on supply chain analytics and data pipeline design, then moves into the five theories that directly shape this system: Medallion Architecture, inventory obsolescence management, ERP data integration, stockout risk theory, and the shift from ETL to ELT processing. From there, it looks at Microsoft Azure and Power BI, the two platforms explains what each one contributes to scalable data processing, business rule execution, and reporting to builds a solid and traceable rationale for every design decision that follows in the next chapters.

2.2 Related Previous Study

In recent industrial case studies in the field of Data Engineering, the shift from inflexible ETL (Extract, Transform, Load) processes to more flexible ELT (Extract, Load, Transform) approaches is emphasized. Manual inventory tracking has been found to be ineffective in prior supply chain analytics pipelines in capturing long-term obsolescence, leading to substantial financial differences (Data Warehousing Architecture | Databricks on AWS, 2025).

A specific focus is on the use of synthetic datasets to simulate more complex operational systems without compromising private company information, in particular, in research related to end-to-end data pipelines. In addition, similar research in the manufacturing

industry has proven that identifying "bottleneck" materials that lead to production delays can be achieved by merging multiple data sources like purchase order, inventory snapshots and sales data (The Data Warehouse Toolkit, 2013). These studies indicate that the precision of financial write-down provisions large increases with the use of computer-based "Dead Stock" flagging systems.

2.3 Related Theory Used

This project is based on three main theoretical fields:

2.3.1 Medallion data Architecture

1. Layer 1: Bronze (Raw Ingestion): The concept of storing the raw data without modifying it. To enable this, raw CSV files need to be ingested and simple data quality control performed on these data, including data deduplication and null detection, while preserving the original data.
2. Layer 2-Silver (Transformation): Focuses on data enhancement and application of complex business rules. Under this layer, raw consumption records are turned into actionable flags, like Recoverable Assets or Magna Carta Dead Stock.
3. Layer 3-Gold (Analytics-Ready): This layer is to store converted data in a data warehouse, which is used for high performance reporting. It is based on the star schema principle. All RA/MC flags are added to the primary fact table (fact_inventory_status) for later use when creating the dashboard (Heizer et al., 2017).

2.3.2 Supply Chain Theory: Inventory Obsolescence and Risk Management

The business logic finds waste in the six supplied synthetic CSV files by applying accepted inventory management principles:

1. Recoverable Assets (RA): The concept of locating capital invested in surplus raw materials, particularly those that have been consumed for more than a year.
2. Magna Carta (MC) Dead Stock: This term describes items that have not been used for nine months or longer and do not have an expiration date (Christopher, 2011).
3. MC Expired: Materials that need a 100% financial write-down or provision because their lot expiration date has passed.

2.3.3 Enterprise Resource Planning (ERP) Data Integration

The Enterprise Resource Planning (ERP) system is a basic software platform relied on by global manufacturing enterprises like PPG, in which accounting, inventory and supply chain data are integrated into a unified coordination system (Bonthu, 2025). Traditional ERP systems are mainly designed for transaction data to be stored, while a special data engineering process is required by the implementation of this project for real-time processing to be achieved, so that analysis can be performed by PPG when the data arrives without reliance on delayed batch updates (Bonthu, 2025). Six synthetic CSV files (representative materials, inventory snapshots and sales orders) are used by the project to simulate the export of data obtained from the actual PPG operational ERP environment, through which this integration is achieved.

A key theoretical component of this integration is Multi-Domain Master Data Management (MDM), by which consistency of information between products, suppliers and customers within the Medallion Architecture is ensured. The synchronization is critical for Recoverable Assets (RA) and Inventory Risk Management (IRM) logic, which will enable the system to recognize materials that are overstocked or running out of inventory "on the fly" (Bonthu, 2025). Furthermore, the combination of real-time processing within the ERP system will enable more efficient operations, with replenishment orders being automated, original transaction variables becoming strategic information regarding the terms of the Magna Carta (MC), and production plans being adjusted. (Bonthu, 2025).

2.3.4 Predictive Stockout and Fulfillment Theory (Operational Theory)

This theory addresses the impact of material shortages on production and sales:

1. Lead Time vs. Inventory Level: The logic used to flag materials at stockout risk when levels fall critically low relative to their replenishment lead time.
2. Sales Order Impact Tracing: The theoretical link between upstream raw material availability and downstream customer fulfillment, identifying which open sales orders are affected by shortages.

2.3.5 Extract-Transform-Load/Extract-Load-Transform

The data integration approach of the project is based on the theoretical pattern of the ETL (Extract, Transform, Load) model to the new cloud-based ELT (Extract, Transform, Load) model. This is an important division in using a Medallion Architecture on Microsoft Azure.

Within a traditional ETL process, information is dependent on a processing unit and then delivered to the destination for storage. Traditional ETL can be inflexible in modern environments because the transformation logic is linked with the ingestion phase (Kancharla, 2025). When the business logic of the Recoverable Assets or Magna Carta classifications of PPG must be changed, the whole extraction process must be repeated many times. It consumes a lot of time and costs only using this old method.

The project is based on the ELT model that utilizes the high-performance capability of the cloud. Raw data in the model will be removed from the source and loaded directly into the layer before any transformation takes place. The ELT approach is a better choice when integrating the cloud because it is more scalable, and the logic of the transformation can be adjusted (Kancharla, 2025). Extract & Load is used for raw synthetic data in the 6 source CSVs and directly loaded into the Bronze Layer of the Azure Data Lake without adjustment. Next, the Silver Layer does some cleaning like removing duplicate rows, fixing nulls, and standardizing the format. Proceed with Transform to perform calculations to identify 12-month consumption and flag Dead Stock are executed in Gold Layer with

distributed computing. Lastly, the Gold Layer is the final year where it can create the dimension table and be ready for the dashboard.

The Medallion Architecture applied is based on the selection of ELT. The project has a 'Source of Truth' because raw synthetic CSV files are loaded into the Azure Data Lake (Gentyala, 2024). Researchers warn that only transferring data is not the way to handle a lot of care to prevent the so-called data debt. The Medallion Architecture is highly efficient and needs proper validation and control of each stage. It is likely to collect anti-patterns and create broken or unreliable data pipelines (Gentyala, 2024).

2.4 Related Technology Used

This project utilizes the cloud-based data to handle the transforming of the data from raw files to the new updated version of business dashboard. The architecture for Recoverable Assets (RA) and Inventory Risk Management (IRM) is created upon these two technologies which is Microsoft Azure and Microsoft Power BI.

2.4.1 Microsoft Azure

Microsoft Azure provides the enterprise backbone for the Medallion Architecture (Bronze, Silver, and Gold Layer) required for this project. There are a few types of Microsoft Azure that can be used in this project.

- **Azure Data Factory (ADF):** ADF functions as an orchestration and integration service. ADF is used to create automated, data-driven workflows that can manage the ingestion of synthetic CSV files into cloud environments. By using the ADF, the process will become a repeatable and automated pipeline where it is necessary for professional data engineers (Global, 2025).
- **Azure Data Lake Storage (ADLS) Gen2:** ADLS Gen2 is specifically designed for big data analytics by integrating the scalability of Blob (Binary Large Object) storage with a hierarchical namespace (On The Use of Hash Functions for Defect Detection in Textures for In-camera Web Inspection Systems, 2002). This allows the project to organize the six operational datasets easily into structured folders. This structure is important for the Medallion Architecture, as it ensures high

throughput and low latency when analytical engines access the data for inventory business.

- **Azure Synapse Analytics:** It can serve as the Gold Layer by providing a unified that joins the big data processing with enterprise data warehousing. The data is organized into a Galaxy Schema, including the center of the inventory status table linked to relevant tables (Das et al., 2019). This part is also the host of the Galaxy Schema where the fact tables are located.
- **Azure Security (Key Vault):** Azure Key Vault is to maintain the integrity of a data engineering pipeline; security and identity management need to be considered. It is used to store cryptographic keys, certificates, and secrets. In this project, Key Vault is used to store the connection strings and keys for the Azure Data Lake and SQL database, preventing the credentials from being hard-coded into the Azure Data Factory and SQL script.

2.4.2 Microsoft Power BI

Power BI serves as the final layer of the stack where it can perform Data Visualization and Business Intelligence. It also can do the report or dashboard for better understanding and observation. This software is used to design data models and interactive reports. It connects directly with the Azure Gold Layer to import the galaxy schema with a complete relationship. The final deliverable, like an interactive dashboard, is used to create a dashboard that contains a lot of useful information like Magna Carta. It also provides simple observations to stakeholders to identify the stockout risks and trace the impact on customer sales orders.

2.5 Summary

In this chapter, the theoretical and technical underpinning used for designing and implementing the PPG Recoverable Assets (RA) and Inventory Risk Management (IRM) data engineering pipeline were introduced. The review was conducted in three key areas: firstly, in relation to previous studies that provide a broader contextual background of the project, secondly, in terms of core theories that underpin the business logic and the architectural approach, and thirdly, in terms of the key technologies that facilitate technical implementation of the project.

This collection of literature revealed that there is a well-documented improvement in data engineering practice that's gaining traction as a new paradigm, moving from traditional Extract-Transform-Load (ETL) pipelines to modern cloud-native Extract-Load-Transform (ELT) pipelines. Previous supply chain analytics projects showed that manual inventory systems fail to identify long term obsolescence consistently and create large monetary mismatch between the inventory and financial statements up till the time of periodic audits. Another key finding in manufacturing analytics was that numerous data domains must be woven together in a single analytical model to identify overall bottlenecks in materials and obsolete stock. A third insight from manufacturing analytics was the need to combine multiple data domains into a single analytical model: inventory snapshots, production orders, purchase orders, sales records, etc. The results directly confirmed the multi-source approach used in this project, which uses 6 synthetic CSV files representing the PPG operational environment and combines them into a single Medallion Architecture pipeline.

To inform the project design choices, five theoretical domains were examined. The Medallion Architecture serves as the structure of data that the pipeline uses, with data stored in three increasingly refined layers: Bronze Layer (*immutable* raw data), Silver Layer (business transformation rules applied to the data), and Gold Layer (transformed data structured into a Galaxy Schema for dashboard consumption). The specific business rules for the three core classifications of Recoverable Assets, Magna Carta Dead Stock and MC Expired materials are encoded in the Silver and Gold layers and are directly answering the project's core business questions through the principles of Supply Chain and Inventory Obsolescence Theory. The Enterprise Resource Planning Data Integration Theory described where the six synthetic CSV files came from and explained the concept of Multi Domain Master Data Management data to be consistent across the Material, Supplier, Customer and Inventory Data Domains throughout the pipeline. Predictive Stockout and Fulfillment Theory was created to describe the logic behind determining the materials to watch for a stockout by comparing stock cover days with supplier lead times, and the downstream effect of material shortages on outstanding customer sales orders. Last but not least, in a theory of comparison, the ETL approach proved to be suboptimal, and the ELT approach was deemed as superior, with the decision to load raw data in Azure Data Lake

Storage being made before any data transformation, where the business logic like the classification rules used in the Magna Carta and the consumption calculations can be changed without bringing the data back from the data lake.

Microsoft Azure and Microsoft Power BI were identified by the technology review as the two pillars of the platform implementation. In the Microsoft Azure environment, four services were studied according to their specific roles in the pipeline. The orchestration layer in Azure Data Factory is used to automate the ingestion of synthetic CSV files into the cloud environment. Azure Data Lake Storage Gen2 offers the hierarchical structure of storage in which the six operational datasets are organized in the Bronze and Silver layers. As per ELT principle, Azure Synapse Analytics Serverless SQL Pool is hosting the Gold Layer Galaxy Schema that allows creation of SQL views over the validated data in the Bronze layer, without any need for physical data movement. All connection strings and credentials that are used throughout the pipeline are secured in Azure Key Vault, which removes the need to store sensitive information in pipeline scripts or configurations. At the top of the pipeline stack is Microsoft Power BI, where the Galaxy Schema is imported, cross-fact relationships are created, and the interactive dashboard that aggregates all seven answers to the business questions is created and presented in a single reporting interface to PPG stakeholders.

CHAPTER 3

METHODOLOGY

3.1 Introduction

This chapter introduces the system development methods adopted by the industry case research institute based on PPG. The project is mainly committed to solving problems related to Recoverable Assets (RA) and Inventory Risk Management (IRM), especially for the detection of inventory outage and out-of-stock risks. A structured method is adopted to ensure that the development process can be carried out systematically and in an orderly manner. The method is divided into several stages, namely planning, analysis, design and implementation, and each stage is described in detail to illustrate the process of system development. In addition, the project planning schedule and system requirements, including hardware and software components, were also discussed. Overall, this chapter outlines the methods and processes involved in developing the proposed data engineering solution.

3.2 Introduction of Methodology

The waterfall development method has been selected as the development method of this project. The method is characterized by a linear and sequential pattern, and the next stage will begin only after each stage is completed. The main stages include planning, analysis, design and implementation. This method emphasizes the methodical advancement of activities, so that the development process can be carried out in a clear and clear way, which is suitable for projects with stable and easy-to-understand requirements.

The reason for choosing the waterfall approach is the nature of the PPG case study, because the scope of the problem, data set and business rules have been clearly defined in the initial stage. The project needs to develop a data engineering pipeline to process multiple data sets, apply the business logic of RA and Magna Carta (MC) classification, and generate insights related to inventory risks. The sequential development method can be implemented because the demand is fixed and it is not expected to be changed to a large extent. In addition, the method supports comprehensive documentation and systematic verification at

each stage, which is crucial to ensure the accuracy and reliability of data processing. Thus, the waterfall approach is appropriate to help in the structured development of the proposed system.

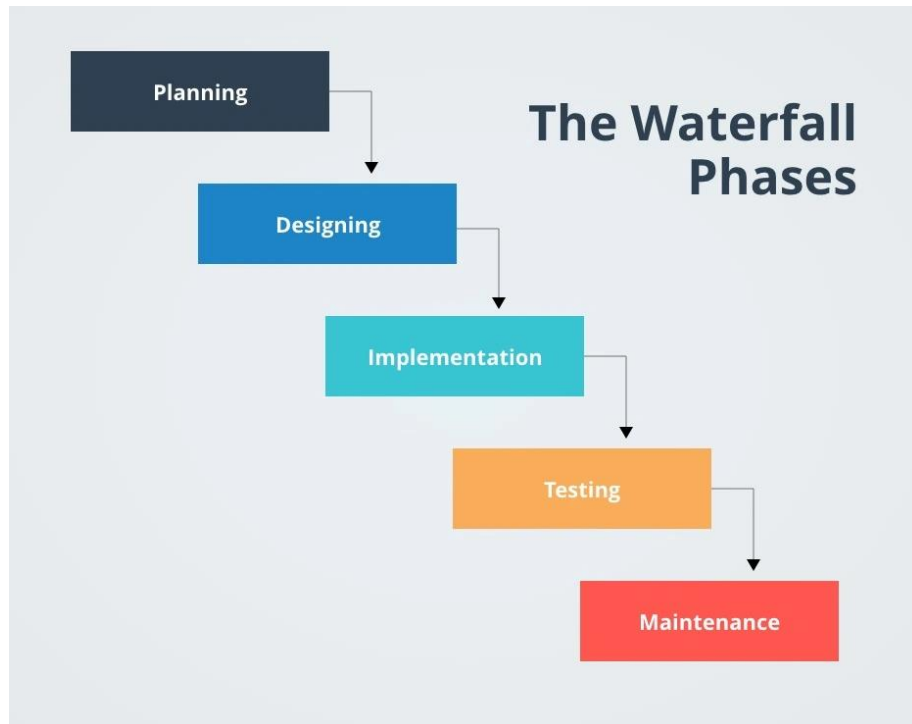


Figure 3.2.1: Waterfall Phases

Figure 3.2.1 shows all five stages waterfall phases, planning, designing, implementation, testing and maintenance.

3.3 Phases of Methodology

This project using Waterfall methodology as its development framework, structured around four sequential phases: Planning, Analysis, Design, and Implementation. Each phase is built outputs that become the direct inputs for whatever comes next. That way, business objectives, technical requirements, and system specifications are all locked in before any actual development work begins.

The team using Waterfall method for this project because PPG's requirements came in well-defined from the start. The inventory risk classifications, the business rules for RA and Magna Carta, and the six operational datasets were already established before the first line of planning documentation was written. That gave the team a stable foundation to work from, with no scenario where requirements would suddenly shift mid-development.

What this approach also does is maintain a clear line of traceability from the original business problem all the way down to the final analytical output. Every pipeline decision made during implementation can be traced back to a design choice, which traces back to a requirement, which traces back to a real operational challenge identified at PPG. That structured accountability is exactly what a data engineering project with financial reporting implications needs to hold up under scrutiny.

3.3.1 Planning Phase

The planning sets direction of the project before any technical work begins. Working from the operational challenges identified at PPG, the team formalized the goal of building an end-to-end data engineering pipeline capable of automatically detecting Recoverable Assets, classifying Magna Carta inventory items, and flagging stockout risks across six structured ERP-derived datasets. Getting these objectives written down and agreed upon early meant that every subsequent decision had a fixed reference point to check against.

The objectives this phase also involved identifying the key stakeholders who had a direct interest in the system's outputs, people from supply chain operations, finance and governance, and IT and data engineering. The project scope, technology stack, timeline, and risk factors were all documented here as well. The result was a clear roadmap that kept the team aligned throughout the project and significantly reduced the room for ambiguity in later phases.

3.3.2 Analysis Phase

The Analysis Phase is an important part of the waterfall methodology as it involves breaking down a high-level business problem into specific technical requirements. For this project, the analysis is concerned with converting the business problems faced by PPG Industries into a data engineering specification.

3.3.2.1 Data Collection

This phase included an industry engagement with PPG Industries, a multinational manufacturer based in Malaysia. Through this industry visit, it is crucial to understand the issue of inventory outdated, where the stock on hand is not aligned with the business needs. The lack of a data-driven system at the plant was identified as the reason why stock excesses and financial write-offs are not being identified and quantified effectively. The six core datasets representing PPG's operating systems were obtained: materials.csv, inventory_snapshot.csv, purchase_orders.csv, production_orders.csv, suppliers.csv and sales_orders.csv. The visit provided the operational insights into PPG's internal risk classifications which are Recoverable Assets (RA) and Magna Carta (MC). The first-hand knowledge of these definitions enables us to identify the data requirements for the next phase of the design process.

3.3.2.2 Data Requirement Analysis

The source files were analyzed in detail to inform the design of the Medallion Architecture. The primary and foreign keys were identified to facilitate the joining of different tables to join the inventory with stock consumption data. The importance of dataset in the pipeline was evaluated. For example, the inventory_snapshot.csv was identified as the key source to detect expiry of materials because of the inclusion of lot_expiry_date field in it. The specification for automated quality checks was defined to be implemented in the Silver layer.

3.3.2.3 Business Logic Formalization

The descriptive business logic provided by PPG was formalized into quantitative logical expressions to be implemented in the Silver, and Gold layers. This guarantees a methodical movement of cleaning information to reasonable business insights.

1. Silver Layer (Data Standardization): This layer was formed where the logic was laid down in a manner to ensure data integrity before business calculations started. This involves elimination of duplicate records, standardization of date formats, and the treatment of null values in the consumption fields to provide a reliable source of truth to the successive layers.

2. Recoverable Assets (RA): Criteria were established to identify raw materials for which the current stock on hand is greater than the expected 12-month consumption.
3. Magna Carta (MC) Dead Stock: Criteria was established to identify materials with no consumption for 9 or more months where there is no expiry.
4. Magna Carta Expired: Logic was defined to identify materials where a lot of expiries has passed, and a 100% financial provision is required.
5. Operational Risk Tracing: Logic was set to trace the effect of material shortages to sales orders to ensure a link from material availability to sales order execution.
6. Gold Layer (Data Modelling): The last phase and it involves formalization of the data into a Galaxy Schema design. Logic was also organizing the transformed data into two central fact tables that are linked with the dimension tables.

3.3.3 Design Phase

This design phase combines the findings of analysis phase into a structured technical blueprint for the PPG inventory risk management system. Based on the business rules formalized in the analysis phase covering RA classification, MC dead stock and expired detection, financial provision calculation, and stockout risk tracing, three design decisions were made on designing the pipeline which included the data pipeline architecture, the analytical data model, and the reporting layer.

3.3.3.1 Medallion Architecture Design

Based on the requirements identified during the analysis phase, the design approach for the pipeline was decided to be Medallion Architecture as shown in Figure 3.3.3.1. This three-layer architecture consisting of Bronze, Silver, and Gold layers was selected as it provides a distinct separation between raw data storage, business logic processing, and analytical output which directly supporting PPG's need for traceable and auditable inventory risk classification.

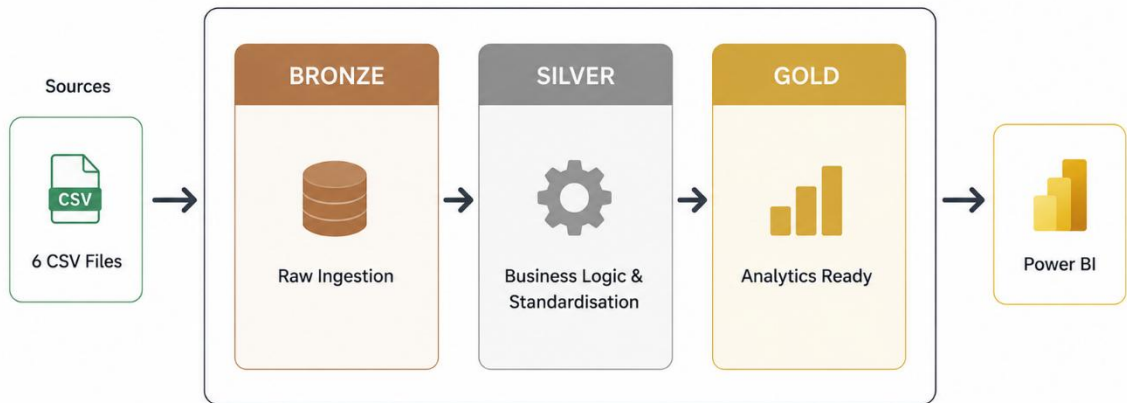


Figure 3.3.3.1: Medallion Architecture Design of PPG RA and IRM System

The Bronze layer was designed to be the raw ingestion zone. There are six source of CSV datasets consisting of materials, inventory snapshot, purchase orders, production orders, suppliers, and sales orders ingested into data lake without any modification on their original structure. This design ensures that the source data always remains fully preserved and traceable. At this stage, basic data quality checks such as null detection, identification of duplicate data, and data type validation are scheduled to identify the records that are valid and those that will need quarantine prior to the next downstream processing step.

The Silver layer was designed to represent the business logic and standardisation zone. Based on the business logic determined during analysis phase, this layer used to clean and combined the data across multiple source files before applying PPG's inventory risk rules. The Silver layer has two parallel workstreams. The first workstream is aimed at the logic for RA classification and provision of MC, and is expected to deliver outputs for Q2, Q3, Q4 and Q5. The second workstream will be responsible for stockout detection and sales order impact tracing to give out stockout Q1, Q6 and Q7 outputs.

The Gold layer was intended to be the analytics ready layer. The transformed outputs from the Silver layer will be aggregated into a structured galaxy schema reporting optimized data model. Power BI will be directly integrated with the Gold layer to provide the final interactive dashboard.

3.3.3.2 Galaxy Schema Data Model Design

The Gold layer data model was designed using a galaxy schema, also referred to a fact constellation schema. This approach was chosen over a star schema model as the system needs to handle two separate analytical areas at the same time which are inventory risk status and sales order impact. Each area carries its own set of measurable indicators that are better kept in separate fact tables rather than forced into one, while still sharing common reference data.

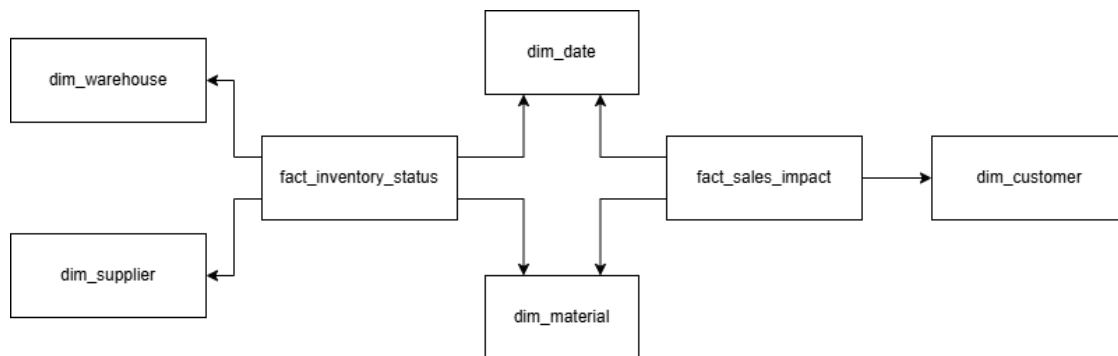


Figure 3.3.3.2: Conceptual Galaxy Schema Design of PPG RA and IRM System

The galaxy schema consists of two fact tables as shown in Figure 3.3.3.2. The `fact_inventory_status` designed to store all computed inventory risk indicators per material including RA flags, MC classifications, stockout risk status, financial provision values, stock cover days, and consumption metrics. The second fact table, `fact_sales_impact` designed to store customer sales orders that may face delayed delivery due to material shortages, and connects each order to the relevant materials, finished products, and customers.

Both fact tables share `dim_date` and `dim_material` as common dimension tables, which enables consistent time-based and material-level analysis across both domains. Beyond the shared dimensions, each fact table is connected to its own dedicated dimension tables based on its analytical scope. The `fact_inventory_status` table is additionally linked to `dim_warehouse` and `dim_supplier`, which provide warehouse location attributes and supplier profile information to support inventory positioning and stockout risk analysis. The `fact_sales_impact` table is additionally linked to `dim_customer` that holds customer identifiers to enable the tracing of which customers are affected by material shortages.

There were three reasons for choosing the galaxy schema design. First, this two fact structure separates inventory risk analysis from sales order impact analysis while avoiding the duplication of shared reference data. Second, sharing dim_date and dim_material across both fact tables ensures the consistent filtering and cross domain analysis within Power BI. Lastly, the structure also supports future expansion since new fact tables can be added without modifying any of the existing shared dimensions.

3.3.3.3 Power BI Analytics Dashboard Design

Based on the seven business questions defined during the analysis phase, the reporting layer was designed using Power BI Desktop connected directly to the Gold layer in Azure Synapse. Power BI was selected because it is compatible with Azure Synapse that supports the galaxy schema data model through its relationship engine, and provides an interactive reporting experience suitable for business stakeholders. The dashboard was designed across 2 reporting pages, each addressing a distinct group of business questions. The first page is planned to cover materials management and inventory stock health by presenting Q1, Q2, Q3, Q4 and Q5 findings related to reorder levels, Recoverable Assets, MC Dead Stock, MC Expired materials, and the total financial writedown amount. The second page is planned to focus on operational risk by presenting Q6, and Q7 results related to stockout risk materials and impacted customer sales orders. The dashboard design includes key performance indicator cards for high-level metrics, detailed material tables and suitable charts for drill-down analysis to allow stakeholders to explore inventory risk across different material groups. All visuals are planned to connect directly to the Gold layer to ensure that every figure displayed on the dashboard is derived from consistent, validated, and fully traceable data that has passed through all three layers of the Medallion Architecture.

3.3.4 Implementation Phase

The implementation phase brings the technical blueprints created during the design phase to life and incorporates a complete data engineering pipeline and reporting system on Microsoft Azure and MS Power BI. This phase includes the physical design of storage architectures, the orchestration of data flows, the programmatic deployment of business

rules, the data warehouse modelling and the creation of dashboards. The implementation is implemented in four main stages corresponding to the Medallion Architecture:

3.3.4.1 Storage and Ingestion Environment Setup (Bronze Layer)

The foundation of the pipeline is established using Azure Data Lake Storage (ADLS) Gen2, organized into containers reflecting the pipeline layers.

1. **ADF Pipeline Orchestration: Azure Data Factory (ADF)** is set up as the main management engine for ADF Pipeline Orchestration. There are six separate Copy Data activities in an ADF pipeline to move the synthetic operational files in the landing zone directly into the Bronze Layer container (materials.csv, inventory_snapshot.csv, purchase_orders.csv, production_orders.csv, suppliers.csv, and sales_orders.csv).
2. **Raw Data Preservation:** These files are copied to the Bronze container, maintaining the integrity of the data by preserving its schema and format, with perfect data lineage.
3. **Basic Ingestion Checks:** Basic quality validation steps performed at the ADF level before triggering downstream processing (completely empty file or missing column).

3.3.4.2 Cleansing and Standardization Implementation (Silver Layer)

Data processing is no longer merely ingested, but transformed by running Azure Synapse Notebooks using an ADF Notebook Activity.

1. **Data Laundering & Deduplication:** Scripts are run in the Synapse Notebooks to use SQL queries to parse the raw Bronze data, find and drop duplicate records and to apply strict type validation.

2. **Schema Standardization:** Date string representations differ in source files and are all converted to the standard date format (YYYY-MM-DD) to support easy relational joining.
3. **Null Handling for Consumption:** If there are fields for tracking material use and production volume, then the null values are programmatically isolated and converted to a zero (0) value using SQL functions such as COALESCE or IFNULL. During the next Gold phase, this essential treatment in Data Engineering eliminates the possibility of calculation errors and nulls while calculating historical consumption.

3.3.4.3 Business Logic & Semantic Transformation (Gold Layer)

The core PPG business rules and risk management logic are embedded within specialized Azure Synapse SQL Notebooks that transform the cleaned Silver datasets into metrics ready for analytics:

1. **Recoverable Assets (RA) Scripting:** These queries will use inventory data and consumption data to calculate a 12-month rolling historical consumption per material. The consumption figures are programmed into conditional flags to define a material as an RA if the amount on hand is greater than those 12-months.
2. **Magna Carta (MC) Dead Stock & Expired Rules:** Multi-conditional logic is devised to filter items:
 - I. *Dead Stock:* Flagged when a material has had no use for the last 9 months without having any active lot expiry date.
 - II. *Expired:* Automatically set to Expired when lot_expiry_date is older than the Reporting snapshot period.
3. **Financial Provisioning & Stockout Risk:** For all flagged MC components, a calculation field is constructed to output a 100% financial write-down provision. Concurrently, inventory replenishment parameters evaluate stock levels against vendor lead times to calculate stockout risks and flag impacted open customer sales orders.

3.3.4.4 Power BI Dashboard Deployment

1. **Power BI Integration: MS Power BI Desktop** is linked directly to the Azure SQL data warehouse containing the Gold star schema. Data Analysis Expressions (DAX) are written to build highly responsive, dynamic measures for cumulative financial provisions, stock-to-consumption ratios, and order counts.
2. **Visualizing Insights:** An interactive multi-tab dashboard is deployed with cross-filtering components, KPI blocks, and alert charts. This visualization layer is engineered specifically to address the 7 core business questions—enabling stakeholders to seamlessly trace inventory risk from high-level corporate financial write-downs down to specific impacted sales orders and customers.

3.4 Project Planning Schedule

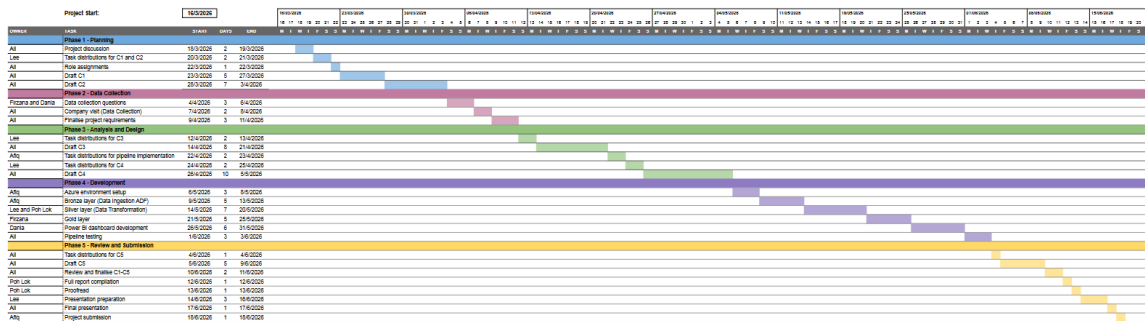


Figure 3.4.1: Gantt Chart of the Project (Source Link: https://docs.google.com/spreadsheets/d/12Vp29ctoFKUH4rvy9-zlv1IJVezrK8v5kXG_VTk3ajs/edit?usp=sharing)

Figure 3.4.1 above illustrates the overall project schedule spanning from 18 March 2026 to 18 June 2026. The project is divided into five phases, each represented by a distinct color to differentiate the phases and their respective timelines. Each horizontal bar represents the duration of a task with the length of the bar corresponding to the number of days allocated for that particular activity. The project is broken down into several phases as follows:

1. **Phase 1 - Planning (18 March – 3 April)** is represented in blue. This phase marks the beginning of the project where the team conducts an initial project discussion to understand the scope and objectives. This is followed by the distribution of tasks for Chapter 1 and Chapter 2 among team members as well as the assignment of roles and responsibilities. Once the planning is complete, team members begin drafting Chapter 1 and Chapter 2 simultaneously as both chapters cover the introduction and literature review which can be done before the data collection phase.
2. **Phase 2 - Data Collection (4 April – 11 April)** is represented in pink. This phase focuses on gathering real-world data from the client organisation, PPG. The team begins by preparing data collection questions to be brought to the company visit. On 7 and 8 April, the team conducted an on-site visit to PPG's premises to collect the required operational data including the six CSV files used in the pipeline. Following the visit, the team finalises the project requirements based on the information and data gathered during the visit.

3. **Phase 3 - Analysis and Design (12 April – 5 May)** is represented in green. This phase involves the planning and design of the system before development begins. Task distributions are made for Chapter 3, after which the team proceeds to draft Chapter 3 which covers the system analysis. Simultaneously, task distributions are made for the pipeline implementation and Chapter 4 to prepare for the development phase. The phase concludes with the drafting of Chapter 4 which documents the system design including the data architecture, database design, and system requirements.
4. **Phase 4 - Development (6 May – 3 June)** is represented in purple. This is the most technical and intensive phase of the project where the actual data pipeline is being built on Microsoft Azure. The phase begins with setting up the Azure environment linked service which includes the Data Lake and Synapse Analytics services. The team then proceeds to implement the three-layer Medallion architecture in sequence starting with the first layer which is Bronze layer for raw data ingestion using Azure Data Factory, followed by the Silver layer for data transformation and business logic implementation using Synapse Analytics and then the Gold layer for loading the analytics-ready galaxy schema into Azure Synapse Analytics. Once the pipeline layers are complete, the Power BI dashboard is developed to answer the seven business questions given in the project brief. The phase concludes with pipeline testing and debugging to ensure data accuracy and system reliability.
5. **Phase 5 - Review and Submission (4 June – 18 June)** is represented in yellow. This final phase focuses on completing the written report and preparing for the final presentation. Task distributions for Chapter 5 are made at the start of this phase, after that the team drafts Chapter 5 covering the implementation and results. All chapters from C1 to C5 are then reviewed and finalised to ensure consistency and quality across the report. The full report is compiled and formatted according to the FYPI report format followed by a proofreading session. Then, the team prepares the presentation slides before delivering the final presentation on 17 June. The completed project report is submitted on 18 June which marks the conclusion of the project.

3.5 Summary

This chapter has presented the methodology and project planning schedule for the development of the PPG Recoverable Assets and Inventory Risk Management project. The Waterfall methodology was selected as the development approach for this project due to its structured and sequential nature which is well-suited for a project with clearly defined requirements and deliverables.

The methodology is carried out across four sequential phases. The first phase, Planning, involves establishing the project scope, distributing tasks among team members, assigning roles and responsibilities and developing the project timeline as illustrated in the Gantt chart. The second phase, Analysis, focuses on gathering requirements through an on-site visit to PPG where the six operational CSV datasets were collected, followed by a thorough analysis of the business rules governing Recoverable Assets and Magna Carta classifications. The third phase, Design, covers the system architecture design including the three-layer Medallion Architecture on Microsoft Azure, the galaxy schema database design for the Gold layer and the Power BI dashboard design. The fourth phase, Implementation, involves the actual development of the end-to-end data pipeline encompassing the Bronze, Silver, and Gold layers, followed by the development of the Power BI dashboard to answer the seven key business questions defined by PPG.

The Gantt chart presented in this chapter provides a visual overview of the project schedule for three months, ensuring that all tasks are completed within the stated deadlines. Overall, the structured approach of the Waterfall methodology ensures that each phase is completed thoroughly before proceeding to the next, thereby maintaining the quality and integrity of the final deliverable.

CHAPTER 4

ANALYSIS AND DESIGN

4.1 Introduction

This chapter explain the problem background, project objectives, theoretical foundations, and methodology and translates into a concrete, implementable technical plan for the PPG RA and IRM Analytics System. The focus here is on demonstrating how inventory risks, financial exposure, and operational disruptions can be systematically identified through an integrated analytics framework built on cloud infrastructure.

The chapter moves progressively from business context through to system architecture, data pipeline structure, analytical data model, and reporting components. A consistent thread throughout is how the Medallion Architecture holds up three core properties at once: data quality enforced at ingestion, traceability preserved by keeping raw data untouched in the Bronze layer, and scalability built into the layer-by-layer structure so the system can grow without needing to be redesigned from scratch. Everything described in this chapter serves as the working blueprint that the implementation chapter then builds on.

4.2 System Analysis

The system analysis examines how PPG's current inventory risk challenges can be translated into a structured data engineering solution that supports Recoverable Assets (RA) and Inventory Risk Management (IRM). At a business level, PPG needs to systematically identify excess materials held beyond 12 months of consumption, classify Magna Carta (MC) Dead Stock and Expired items that require financial write-downs, and detect stockout risks that may delay production or open customer sales orders. At a technical level, these needs are mapped into clear functional requirements for data ingestion, quality checks, business rule implementation, schema modeling, and interactive reporting, using six synthetic ERP-like datasets hosted within a Medallion Architecture on Microsoft Azure.

This analysis links organizational roles, business rules, and data structures into one coherent view. Supply chain and warehousing define the operational scenarios for stock coverage and lead times, finance establishes the rules for RA, MC Dead Stock, and MC Expired provisioning, and the IT and data engineering unit provides the cloud platform, governance, and security required for an automated pipeline. Through industry visits, interviews, and dataset profiling, these perspectives are consolidated into functional and non-functional requirements that guide the design of the Bronze, Silver, and Gold layers, ensuring that the resulting system not only automates existing manual processes but also delivers reliable, analytics-ready outputs for decision makers across the company.

4.2.1 Case Study

PPG Industries is a global manufacturing company that handles large volumes of raw materials to keep its production lines running. The problem the company ran into that tracking inventory health relied heavily on manual processes and spreadsheets an setup that made it genuinely difficult to catch excess stock, identify obsolete materials, or quantify financial exposure in any consistent or timely way. The longer that gap persisted, the more it showed up as miscounted provisions, missed write-downs, and operational blind spots.

This project proposes an automated pipeline that takes six ERP-derived synthetic datasets through a Medallion Architecture on Microsoft Azure. The pipeline applies PPG's own business rules to identify Recoverable Assets, Magna Carta Dead Stock, Magna Carta Expired materials, financial provisioning amounts, and materials at stockout risk that could delay customer orders. The business rules and classification logic used throughout the system were drawn directly from an on-site industry visit, stakeholder discussions, and close analysis of PPG's actual operational processes means the system is grounded in how PPG really works, not just how a textbook would model it. Synthetic datasets were used in place of live data to protect confidentiality while still reflecting the structure of PPG's real operational environment.

4.2.2 Company Organization Structure

The company organization needs to be completely organized the structure by aligning the inventory methods to automated analytical architecture. This system deployment has an organizational structure consisting of the following functional divisions:

1. **Supply Chain and Warehousing Operations:** This division is responsible for ensuring the physical inventory, always keeping track of replenishment of lead times and as the business owner for material availability. The platform allows the operational employees to monitor the stockout risk, track safety of stock levels, and position the raw materials in relation to production schedules. Having this operational-to-analytical alignment is critical to help the people working on the job clearly understand at risk resources much faster than legacy manual operations (Krystofik et al., 2020).
2. **Finance and Governance Department:** This used the outputs from the automated pipeline to audit the inventory valuations as a responsibility that it also falls within the scope of finance and governance role. It heavily depends on the systematic extraction of Magna Carta classifications to estimate the correct monthly write-down provisions and ensure that the corporate accounting standards are met.
3. **Technical Unit:** The Unit that handles the infrastructure of the enterprise is called Information Technology (IT) and Data Engineering Unit. As part of this architecture, the responsibility for supplying the cloud ecosystem is held, maintaining the connection security through key vaults, setting up the pipeline triggers and ensuring the smooth flow of transactional data from data source environments towards the analytical layer.

4.2.3 Company Requirements Gathering Techniques

A variety of approaches to requirements was used to define a complete and accurate set of functional and non-functional requirements. This framework was based on qualitative and quantitative methods to connect operational reality and technical system specifications.

The primary data collection involved by the industry visit to PPG manufacturing based in Malaysia and did some interviews with the workers. Qualitative business rules and guidelines were gathered through talk and semi-structured interviews with domain experts, including the rules for what constitutes Dead Stock after 9 months and dependency rules between the raw materials and the open customer sales orders.

An in-depth review of Enterprise spreadsheets, the historical reporting templates, and material definitions were completed. This technical review also detailed the exact formulas to be used to convert business rules that were not explicit in the text, into mathematical expressions. This is to understand the inventory variables before formalizing the logic.

The requirement discovery method involved auditing the structure attributes of the six core operational datasets from enterprise resource planning (ERP) environment to gain the quantitative requirements. Profiling these files (materials.csv, inventory_snapshot.csv, purchase_orders.csv, production_orders.csv, suppliers.csv, and sales_orders.csv) enabled primary and foreign key constraints to be validated, data quality issues to be expected and structure specifications for the Data Lakehouse refinement layers to be formalized.

4.2.4 Use Case Diagram

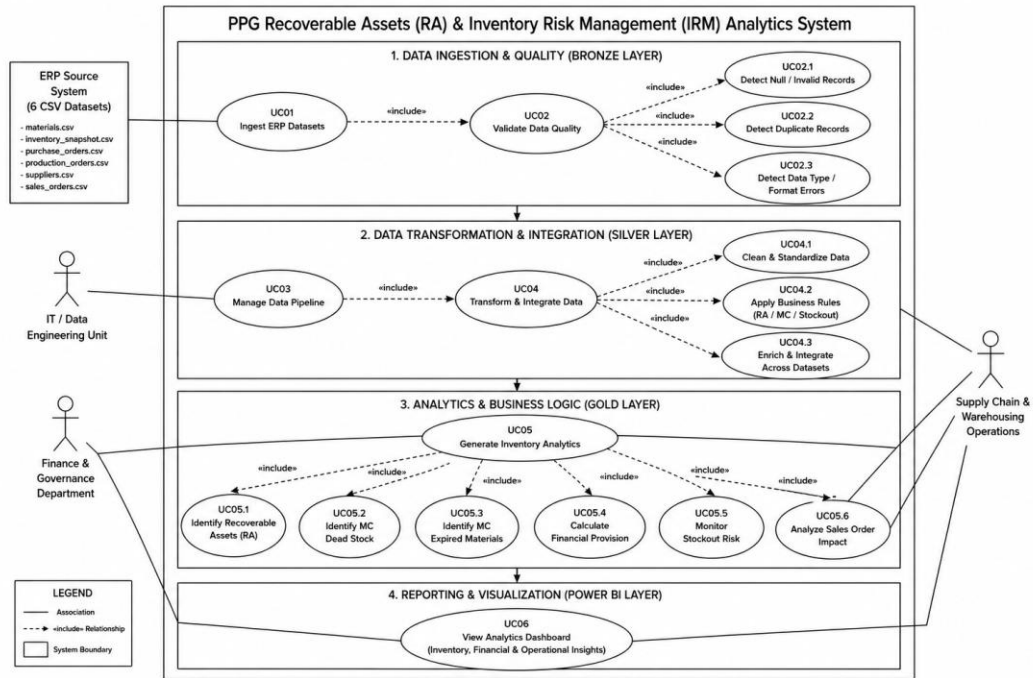


Figure 4.2.4.1: Conceptual Star Schema Design of PPG RA and IRM System

Figure 4.2.4.1: Use Case Diagram of PPG RA and IRM System

The Use Case Diagram shows the complete end-to-end workflow of the PPG RA and IRM Analytics System, from the moment raw data enters the pipeline to the point where business stakeholders are reading insights on a dashboard. The system pulls from six operational datasets: materials, inventory snapshots, purchase orders, production orders, suppliers, and sales orders, all hosted within a cloud-based Medallion Architecture on Microsoft Azure. Four main actors interact with this system: the ERP Source System that provides the raw data, the IT and Data Engineering Unit that manages and runs the pipeline, the Finance and Governance Department that uses financial risk outputs for reporting and write-down calculations, and the Supply Chain and Warehousing Operations team that monitors stockout risks and material coverage levels.

The first stage of the workflow runs across the Bronze and Silver layers. Raw datasets are loaded into the system, then immediately run through a series of quality checks, missing value detection, duplicate record identification, and data type verification to make sure only clean, reliable data gets passed forward. Once validated, the IT and Data Engineering team takes over: cleaning and standardizing data from multiple source domains, then applying

PPG's business rules covering Recoverable Asset classification, Magna Carta logic, and stockout risk analysis. By the end of this stage, the data is consistent, verified, and ready for the analytics layer.

The Gold Layer is where the system generates the actual inventory risk intelligence that answers PPG's business questions. This includes identifying Recoverable Assets, detecting Magna Carta Dead Stock and Expired materials, calculating financial write-down provisions, monitoring stockout conditions, and tracing which open customer sales orders are affected by material shortages. All of this gets delivered through a Power BI dashboard that gives stakeholders a single, unified view of inventory health, financial exposure, and operational risk. The combination of automated data processing, embedded business logic, and interactive reporting means PPG can move faster, govern its inventory better, and make decisions grounded in data rather than estimates.

4.2.5 Entity Relationships Diagram

The relationship between the raw data files has been analysed and a relational entity-relationship diagram (ERD) has been created as illustrated in Figure 4.2.5 below.

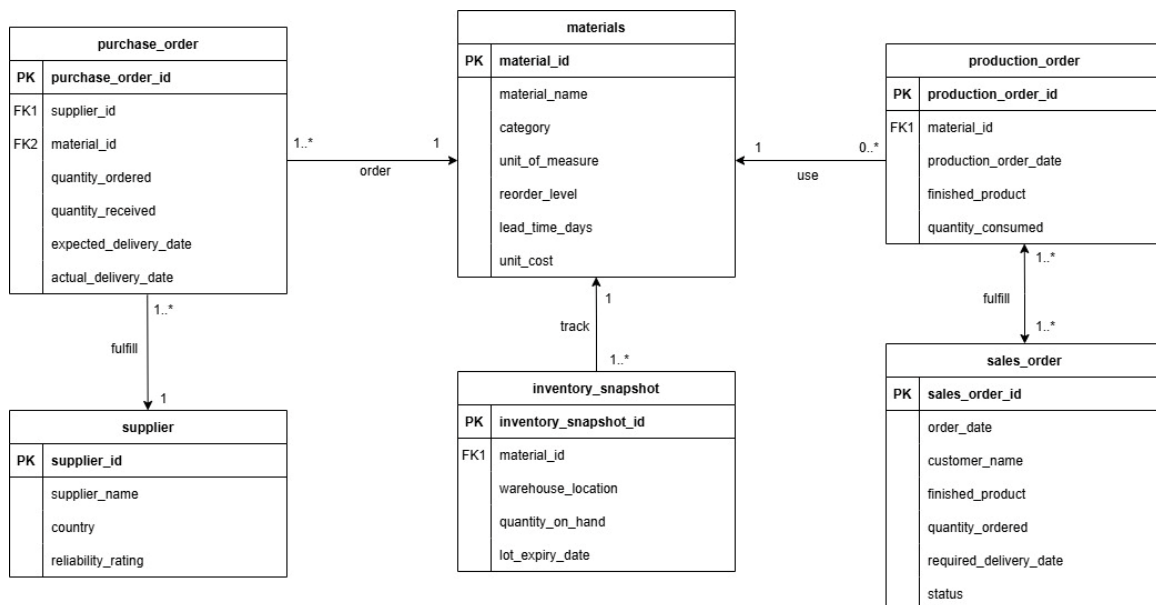


Figure 4.2.5: Entity Relationship Diagram

This architecture is constructed from an arrow direction notation where arrowhead is towards source of truth (unique primary key) and the tail of the arrow is from the child

entity (foreign key). This design was represented graphically and ensured transactional integrity of the raw data by providing an operational dependency. The ERD diagram above is broken structurally as shown in Table 4.2.5 below.

Table 4.2.5: ERD Relationship Justification

Pathway	Cardinality	Relationship	Justification
purchase_order → supplier	1..* to 1	Many-to-One	Supplier information for each individual purchase order will be linked to 1 valid supplier.
purchase_order → materials	1..* to 1	Many-to-One	Only one material per purchase order.
inventory_snapshot → materials	1..* to 1	Many-to-One	A single material that is repeated over time and which has many historic log records for different snapshot dates.
production_order → materials	0..* to 1	Many-to-One	A material may be used in several different production runs, or may not have been used in any production runs if it has not been consumed in production.
production_order ↔ sales_order	1..* to 1..*	Many-to-Many	The two tables are a non-key (in this case, the text string finished_product) based join.

A very important insight in this model is that between production_order and sales_order, there is a bidirectional relationship. Since the product names are repeated in hundreds of row logs and the typical relational constraints require a foreign key to refer to a completely unique primary key, these two operational logs cannot be bound together with strict schema keys. Instead, they link dynamically via text values matching in the finished_product column which creates a logic-based join. This connection is highly intentional for data engineering pipeline as it lets the business easily trace raw material shortages from warehouse records directly up to the specific customer sales orders they will impact.

4.3 System Requirements

The system requirements for the PPG Recoverable Assets and Inventory Risk Management system are defined in this section with functional and non-functional requirements. The requirements were identified in the business rules collected as part of the industry visit to PPG and the dataset profiling as part of the analysis phase.

4.3.1 Functional Requirements

Functional requirements refer to the specific functions that the system should perform to meet the project's objectives. The functional requirements for this project were described below.

FR1: The system shall ingest all six raw CSV datasets into the Bronze layer of data lake without modifying the original file structure and format.

FR2: The system shall perform data quality checks on all ingested datasets automatically, including detecting missing values in critical fields, identifying incorrect data types, and removing or flagging of duplicate records.

FR 3: The system shall calculate the total quantity consumed for each raw material over the past 12 months using historical production order records.

FR 4: The system shall flag a material as a Recoverable Asset if its current quantity on hand, based on most recent inventory snapshot exceeds its calculated 12-month consumption. The excess quantity shall be computed and stored for each flagged material.

FR 5: The system shall flag a material as MC Dead Stock if it has not been consumed for nine or more consecutive months and does not have a lot expiry date recorded in the inventory snapshot.

FR 6: The system shall flag a material as MC Expired if its lot expiry date field is populated and the date has passed relative to the snapshot reporting date.

FR 7: The system shall calculate a 100% financial write-down provision amount equal to the material's unit cost multiplied by its quantity on hand for all materials classified as either MC Dead Stock or MC Expired, The system shall also aggregate the total provision amount across all MC materials.

FR 8: The system shall flag materials whose current quantity on hand falls below their defined reorder level, as specified in the materials dataset, to support replenishment planning.

FR9: The system shall calculate the average daily consumption rate per material and the number of stock cover days remaining. A material shall be flagged as having stockout risk if its stock cover days are less than its supplier lead time in days.

FR10: The system shall identify which open customer sales orders are at risk of delayed fulfilment due to stockout risk in their required raw materials. This is achieved by linking at-risk materials through production orders to finished products, and finished products to open sales order records.

FR11: The system shall organise all transformed and classified outputs into a galaxy schema data warehouse in Azure Synapse Analytics SQL.

FR12: The system shall provide an interactive Power BI dashboard that visualises all seven business questions, including materials below reorder level, Recoverable Asset listings, MC Dead Stock and Expired material tables, total financial provision amounts, stockout risk materials, and impacted customer sales orders.

FR13: The system shall organize all pipeline stages includes Bronze ingestion, Silver transformation, and Gold loading through a master Azure Data Factory pipeline that

enforces sequential execution with dependency control. It should ensure each stage completes successfully before downstream stages begin.

4.3.2 Non-Functional Requirements

Non-functional requirements refer to the quality attributes and constraints that the system should meet. The non-functional requirements for this project were described below.

NFR1: The system shall include error handling at each pipeline stage so that failures are detected and reported without affecting the downstream data.

NFR2: The system shall ensure all the raw data sources in the Bronze layer never modified or overwritten. All business logic transformations shall be applied only in the Silver and Gold layers to maintain a clear and auditable lineage from source to final output.

NFR3: The pipeline architecture shall be designed to handle increasing volumes of data in the future without requiring fundamental structural changes.

NFR4: The system shall be built in a modular approach where each pipeline layer operates independently and able to be modified without affecting others.

NFR5: The Power BI dashboard shall be designed to load and respond to user interactions in a reasonable time. Data shall be pre-processed and arranged in the Gold layer so that the dashboard does not have to conduct complex computations during query time.

NFR6: The system shall specifically designed for batch processing of static synthetic datasets.

4.4 System Design

This section outlines the architecture for the PPG Recoverable Assets and Inventory Risk Management system. It details the data pipeline flow, database model, and operational

mechanisms such as scheduling and error handling developed to address the project’s seven core business questions.

4.4.1 Overview of the Proposed System Design

The proposed system is designed as an end-to-end data engineering pipeline for the PPG Recoverable Assets (RA) and Inventory Risk Management (IRM) case study. The system is intended to process six operational datasets, apply inventory-related business rules, and produce analytics-ready data for reporting through Power BI. The overall design combines Microsoft Azure services with the Medallion Architecture approach, where data is processed progressively through Bronze, Silver, and Gold stages. This structure allows raw data to be preserved, cleaned, transformed, modelled, and visualized in a systematic manner.

4.4.2 Data Architecture

The proposed data architecture will be based on the Microsoft Azure services and Medallion Architecture approach. It is designed to meet the requirements of the PPG Recoverable Assets (RA) and Inventory Risk Management (IRM) case study, by providing the data engineering workflow from raw data ingestion to dashboard reporting. The architecture consists of six main components: data sources, data ingestion, data storage, data processing, data governance and security, and data consumption. (Krantz, T., 2024).

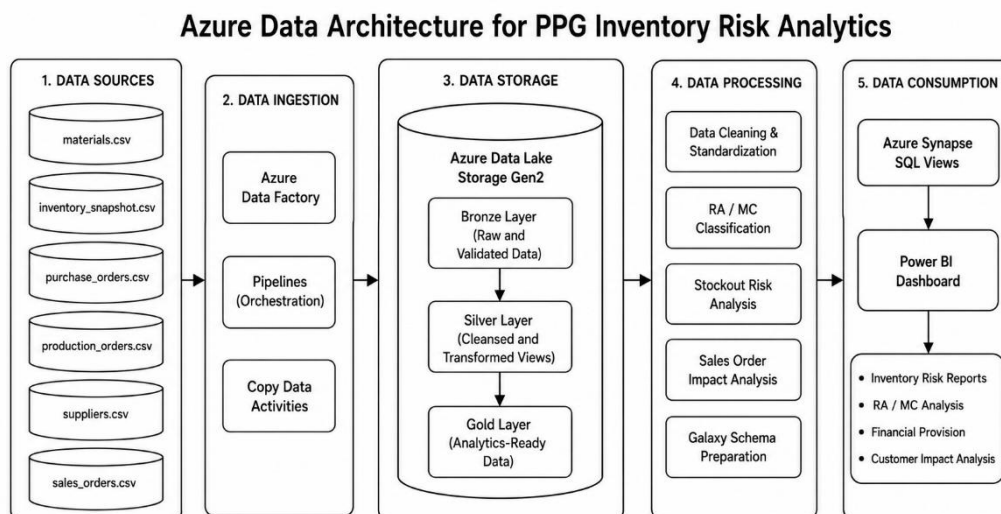


Figure 4.4.2.1: Proposed Azure Data Architecture for PPG Inventory Risk Analytics

Figure 4.4.2.1 shows the proposed Azure data architecture for the PPG inventory risk analytics system. The architecture was designed based on a simplified data warehouse flow, consisting of data sources, data ingestion, data storage, data processing, and data consumption. It illustrates how the six CSV datasets are ingested using Azure Data Factory, stored in Azure Data Lake Storage Gen2, processed using Azure Synapse Serverless SQL, and finally consumed through Azure Synapse SQL views and Power BI dashboard.

Table 4.4.2.1: Components of the Proposed Data Architecture

Component	Tool / Layer Used	Description
Data Sources	Six CSV datasets	Operational datasets are provided for material, inventory, supplier, production, purchase, and sales order analysis.
Data Ingestion	Azure Data Factory	CSV files are ingested into the Azure environment through Copy Data activities and pipeline orchestration.
Data Storage	Azure Data Lake Storage Gen2	A centralized storage platform is used to organize the data into Bronze, Silver, and Gold layers.
Data Processing	Azure Synapse Serverless SQL	Data cleaning, standardization, RA/MC classification, stockout risk analysis, sales order impact analysis, and star schema preparation are performed.
Data Consumption	Azure Synapse SQL views and Power BI	Business-ready outputs and dashboard visualizations are provided for inventory risk analysis, RA/MC analysis, financial provision, and customer impact analysis.

Table 4.4.2.1 summarizes the main components of the proposed data architecture. It shows

the tools and layers used in each component, including data sources, data ingestion, data storage, data processing, and data consumption.

The data source component consists of six synthetic CSV files, namely materials.csv, inventory_snapshot.csv, purchase_orders.csv, production_orders.csv, suppliers.csv, and sales_orders.csv. These datasets are used to support the identification of Recoverable Assets, Magna Carta Dead Stock, Magna Carta Expired materials, stockout risks, and impacted customer sales orders.

Azure Data Factory is used to handle the data ingestion component. Pipeline orchestration is used to transfer the datasets to Azure Data Lake Storage Gen2, along with Copy Data activities. Azure Data Lake Storage Gen2 is the main storage container and the data is stored in the following stages: Bronze, Silver, Gold. The Bronze layer is used to store raw and validated data, the Silver layer is used to store cleansed and transformed views, and the Gold layer is used to store analytics-ready data for reporting.

Finally Azure Synapse Serverless SQL is used to perform data processing. It is used to clean and normalize data, to apply RA and MC classification logic, to compute the stockout risk, to track the impact of sales orders, and to create the analytical data model. The updated implementation will use two fact views to create the analytical model, which is a Galaxy Schema. The fact_inventory_status view supports Q1 to Q6 and fact_sales_impact view supports Q7. These fact views have the same dimensions (dim_material, dim_date, dim_warehouse), and dim_customer is used to help analyze customer impact.

Last but not least, Azure Synapse SQL views and Power BI are employed as the data consumption layer. The processed outputs are consumed in dashboard visualizations and reports, such as inventory risk reporting, RA/MC analysis, financial provision analysis, stockout risk analysis and customer impact analysis.

Data quality checks, validation rules, access control, and traceability are applied across the entire pipeline, although these are not depicted as separate entities in the logical architecture diagram.

4.4.3 Explanation of Proposed Data Architecture

The suggested architecture will facilitate a structured data engineering process, starting from raw data ingestion to business intelligence reporting. The process starts with six CSV datasets that are the operational data needed for the PPG RA and IRM case study. Material master, inventory snapshots, purchase order, production order, supplier, and sales order are included in these datasets.

Ingestion and orchestration tool used is Azure Data Factory. Copy Data activities are used to copy each of the CSV files to Azure Data Lake Storage Gen2. This enables the ingestion process to be repeated and managed in a systematic manner. The Azure Data Factory also enables pipeline control, allowing you to monitor and validate the ingestion activities before downstream processing takes place.

Data storage component is implemented with Azure Data Lake Storage Gen2. The storage design is based on a three-layered Medallion Architecture: Bronze, Silver and Gold. The Bronze layer contains raw and validated CSV datasets. Unprocessed data is retained for traceability and validated data is generated following simple data quality validation. These checks include missing value detection, duplicate checking, data type validation, and column structure verification. Records that are invalid or are problematic may be quarantined in a folder so they will not affect later steps in processing.

The data goes through the Bronze layer and then processed into the Silver layer. The Silver layer is responsible for cleaning, standardization and transformation. In this project, Azure Synapse Serverless SQL is used to implement the Silver layer processing. This is done because the Synapse Spark pool could not be executed due to the workspace vCore quota. Thus, SQL views are employed to execute the transformation logic, and the business rules and the desired output remain the same.

The RA and MC classification logic are used for the Silver layer. Production order data is used to calculate material consumption and inventory snapshot data is used to determine quantity on hand. Recoverable Assets are those assets determined by comparing the current stock of assets with Expected Consumption. In order to identify Magna Carta Dead Stock,

the materials are checked for a specified amount of time; in order to identify Magna Carta Expired, the `lot_expiry_date` field is used. The classifications are necessary because of the need for full financial provision of MC materials.

In the Silver layer, a stockout risk analysis is also carried out. The quantity on hand is checked against the reorder level to determine what materials are below the level. Average daily consumption and stock cover days are computed and compared then with material lead time. If there is not enough stock to meet lead time, it is marked as stockout risk. At-risk materials are also associated with finished products and open sales orders, so that products and sales orders that are affected by the at-risk materials can be identified.

The Gold layer is utilized to structure the processed results into a model that is suitable for analytics. Azure Synapse SQL is used to prepare the Gold layer tables and views. The `fact_inventory_status` table is related to the dimension tables, including `dim_material`, `dim_supplier`, `dim_customer` and `dim_date`, and the design is used as a star schema design. The fact table contains calculated measures and flags such as inventory quantity, consumption values, RA and MC indicators, stock out risk status and financial provision amount. This is ideal for filtering and reporting in Power BI.

Data governance and security are included throughout the architecture. Access control is ensured using role-based access control and data quality checks and validation rules are applied to ensure data reliability. The quarantine folder is used to store invalid records, and data lineage is preserved through the use of distinct layers in the workflow. These controls help to ensure that inventory risk calculations and financial provision outputs are based on reliable data.

Lastly, it includes Azure Synapse SQL views and Power BI. Power BI accesses the ready-made analytical results to generate dashboards and reports. The dashboard provides inventory risk reports, RA/MC analysis, financial provision values, and customer impact analysis. The architecture allows for transforming raw operational data into business-ready insights in a structured, traceable, and scalable way.

4.4.4 Scheduling and Error Handling

To make sure the system runs reliably without needing human intervention, it includes automated schedules and error management:

1. **Automated Trigger:** The pipeline is attached to an ADF **Schedule Trigger**. This trigger is set to run automatically at the end of every month or operational cycle, right when the factory generates new inventory records and order sheets.
2. **Error Tracking:** If something goes wrong inside a Synapse Notebook, for example, if a file has a misspelled column name or a mathematical calculation crash, the notebook will stop running immediately. It sends a failure message back to Azure Data Factory.
3. **Data Rollback:** The system uses safe transactional database commands when writing data to the Gold layer. This means if a notebook crashes halfway through updating the final database, it will undo its partial changes and roll back to the last safe state. This stops incomplete or corrupted data from showing up on the Power BI dashboards.

4.4.5 Database Design

The Gold layer adopts a galaxy schema, also known as a fact constellation as its analytical data model, a design pattern used in dimensional modelling for data warehouses that require multiple fact tables to represent distinct business processes. In a galaxy schema, two or more fact tables share one or more common dimension tables, allowing related business processes to be analysed both independently and in combination through their shared dimensions. This structure was adopted because the project requires two distinct analytical perspectives which are inventory status and sales order impact that share common reference data but capture different grain and measures.

Figure 4.4.5.1 below shows the schema implemented in this project, consisting of two fact tables and five dimension tables. The first fact table, `fact_inventory_status`, captures the monthly inventory position of each material, including Recoverable Asset and Magna Carta flags, provision amounts, reorder status, and stockout risk metrics. The second fact table,

fact_sales_impact, captures open sales orders affected by stockout-risk materials, including delivery risk and stock cover details at the point of impact.

The five dimension tables are dim_material, dim_date, dim_supplier, dim_warehouse, and dim_customer. Of these, dim_material and dim_customer serve as conformed dimensions, each connected to both fact tables, enabling cross-fact analysis such as linking a customer's affected sales order back to the material driving the stockout. dim_date connects to fact_inventory_status via snapshot_date, while dim_supplier and dim_warehouse connect exclusively to fact_inventory_status via supplier_id and warehouse_location respectively, as these attributes are relevant only to the inventory dimension of the business process.

To support standalone querying of fact_sales_impact without requiring a join back to dim_material for every analytical query, selected material attributes (material_name, category, quantity_on_hand, reorder_level, avg_daily_consumption, stock_cover_days, lead_time_days, is_stockout_risk) are denormalised directly into the fact table at the time of order impact assessment. This reflects the snapshot nature of fact_sales_impact as it records the inventory condition of the material at the moment the sales order was identified as impacted, rather than its current state, which may differ from the corresponding row in fact_inventory_status at query time.

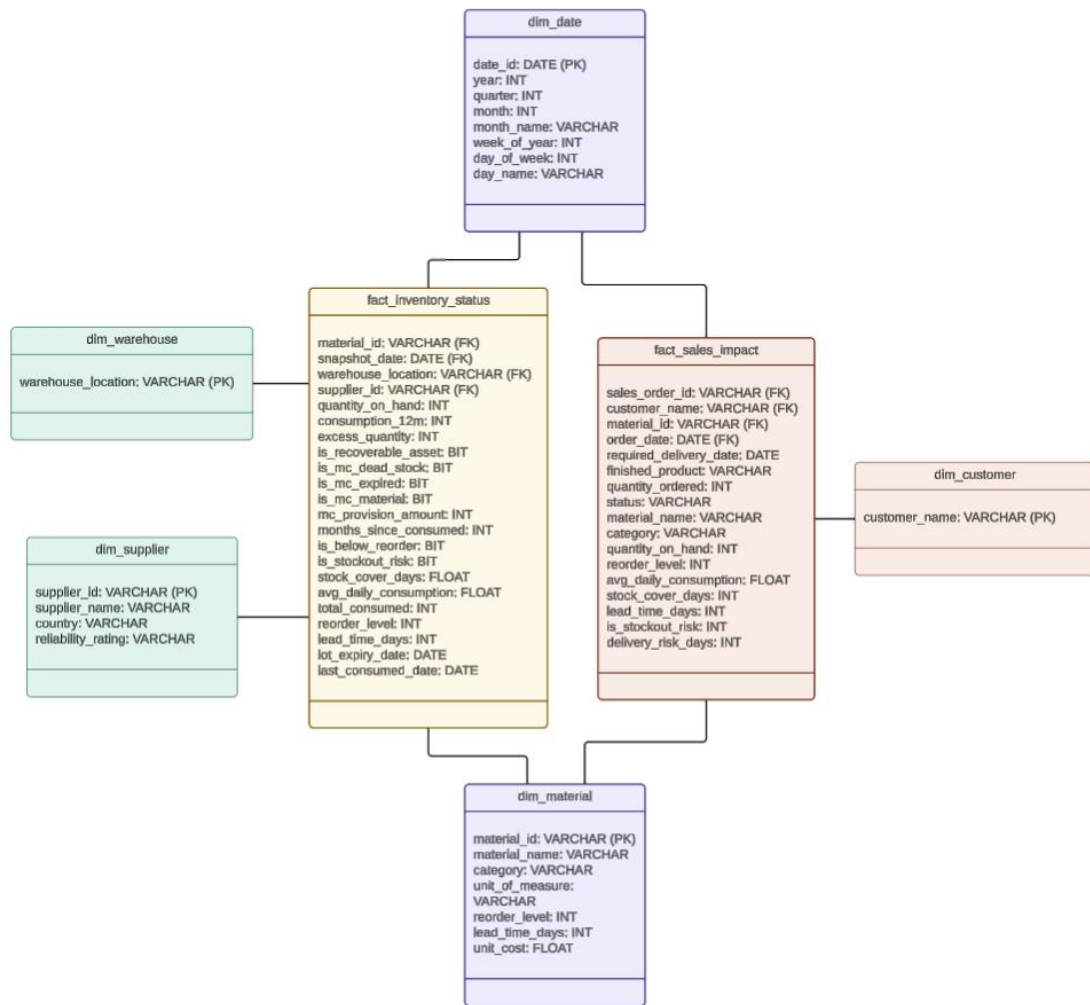


Figure 4.4.5.1: Galaxy Schema of PPG RA and IRM System

4.4.5.1 Fact Table

The `fact_inventory_status` table serves as the central fact table of the star schema and is the primary analytical table of the entire pipeline. Each row in this table represents the inventory status of a single material at a specific point in time, capturing the outcome of all business logic applied during the Silver layer transformations. The table is identified by a surrogate primary key, `inventory_status_id`, which uniquely distinguishes each record. It also carries four foreign keys which are `material_id`, `supplier_id`, `customer_id`, and `date_id` to establish the relationships to each of the surrounding dimension tables.

Beyond the key columns, the fact table holds a comprehensive set of measures and derived indicators. The `snapshot_date` and `warehouse_location` fields provide operational context for each record. Quantitative measures include `quantity_on_hand`, `consumption_12m`,

excess_quantity, and months_since_consumed which together capture the stock position and consumption behaviour of each material. The fields lot_expiry_date and last_consumed_date are retained as supporting date attributes to preserve traceability back to the Silver layer.

4.4.5.2 Dimension Tables

The dimension tables are created as follows:

1. **dim_material** - A dimension table that holds the reference attributes for all raw materials tracked within the pipeline. It is keyed on material_id as its primary key and contains descriptive attributes including material_name, category, unit_of_measure, reorder_level, lead_time_days, and unit_cost. These attributes provide the contextual information needed to interpret the measures in the fact table — for example, reorder_level is used in conjunction with quantity_on_hand to determine whether a material has fallen below its replenishment threshold, and unit_cost is the basis for calculating the Magna Carta provision amount. The category field, which classifies materials into groups such as Pigment, Resin, Solvent, and Packaging, also serves as a key filtering dimension in the Power BI dashboard, allowing analysts to slice inventory risk results by material type.
2. **dim_date** – A dimension table that provides the time dimension for the star schema, enabling date-based filtering and aggregation within the Power BI dashboard. It is keyed on date_id as its primary key and contains the derived calendar attributes date, month, quarter, and year. Each row in this table corresponds to a distinct snapshot date present in the inventory data, with the current dataset spanning twelve monthly entries from January to December 2025. By separating date intelligence into its own dimension, the schema enables analysts to filter dashboard visuals by time period without embedding date logic directly into the fact table or the reporting layer. This also allows future expansion of the date range without structural changes to the fact table.
3. **dim_supplier** – A dimension table that holds reference information about the raw material suppliers engaged by PPG. It is keyed on supplier_id as its primary key and contains supplier_name, country, and reliability_rating as descriptive

attributes. The `supplier_id` foreign key in the fact table establishes a direct relationship between each inventory record and its associated supplier, enabling supply chain analysis within the dashboard. The `reliability_rating` field, which categorises suppliers by their delivery performance, is particularly relevant in the context of stockout risk — materials sourced from lower-reliability suppliers may carry a higher risk of supply disruption, which can be surfaced through Power BI filters applied against this dimension.

4. **dim_customer** – A dimension table that captures the distinct set of customers whose open sales orders are potentially affected by material shortages. It is keyed on `customer_id` as its primary key, with `customer_name` as the identifying attribute. This table is populated from the sales orders data and is linked to the fact table through the `customer_id` foreign key, establishing the connection between inventory risk outcomes and downstream customer impact. In the context of the dashboard, `dim_customer` supports the Q7 analysis together with the analytical view `vw_q7_impacted_orders`, it enables the identification of which specific customers are exposed to fulfilment risk as a result of stockout conditions at the material level.

CHAPTER 5

IMPLEMENTATION AND TESTING

5.1 Introduction

This chapter presents the practical implementation of the PPG Recoverable Assets (RA) and Inventory Risk Management (IRM) data engineering pipeline based on the Medallion Architecture. It explains how the Bronze, Silver, and Gold layers were developed on Microsoft Azure to ingest six synthetic CSV datasets, enforce data quality checks, apply RA and Magna Carta business rules, detect stockout risks, and load the transformed data into a star schema centered on the `factinventorystatus` table. The chapter also describes the integration of this Gold layer with a Power BI dashboard that delivers interactive visualizations for financial provisions, inventory classifications, and customer order impacts.

This chapter explained details the testing activities carried out to ensure the correctness and reliability of the system. Black Box Testing validates the outputs produced by each layer and the Power BI dashboard against the expected business results, while White Box Testing verifies the internal logic of the ingestion flows, transformation scripts, and star schema loading processes. User Testing is then conducted to confirm that stakeholders can interpret the dashboards effectively and use the insights to support inventory risk decisions.

5.2 System Development

This section details the deployment of the PPG RA and Inventory Risk Management system on Microsoft Azure. It covers the implementation of a three-tier Medallion Architecture and its integration with Power BI. This design focuses on transforming raw operational data into a structured galaxy schema optimized for business intelligence and visualization idea.

5.2.1 Bronze Layer

The Bronze Layer constitutes the first stage of the three-tier Medallion Architecture implemented in this project. Its primary responsibility is the automated ingestion of raw source data from six Comma-Separated Values (CSV) files into Azure Data Lake Storage Gen2 (ADLS Gen2), followed by systematic data quality and duplicate detection checks to ensure only validated data progresses to the Silver Layer. The Bronze Layer was developed entirely within the Azure Data Factory (ADF).

Prior to pipeline construction, the foundational Azure infrastructure was provisioned under the resource group `rg-ppg-pipeline` in the Southeast Asia region. The storage account `ppgdatalake` was created with the hierarchical namespace enabled, configuring it as an ADLS Gen2 instance. Three containers which are bronze, silver, and gold were created within the storage account, corresponding to each layer of the Medallion Architecture. The ADF instance `adf-ppg-pipeline` and the Synapse Analytics workspace `synapse-ppg` were provisioned within the same resource group to centralise all data engineering resources, as illustrated in Figure 5.2.1.1.

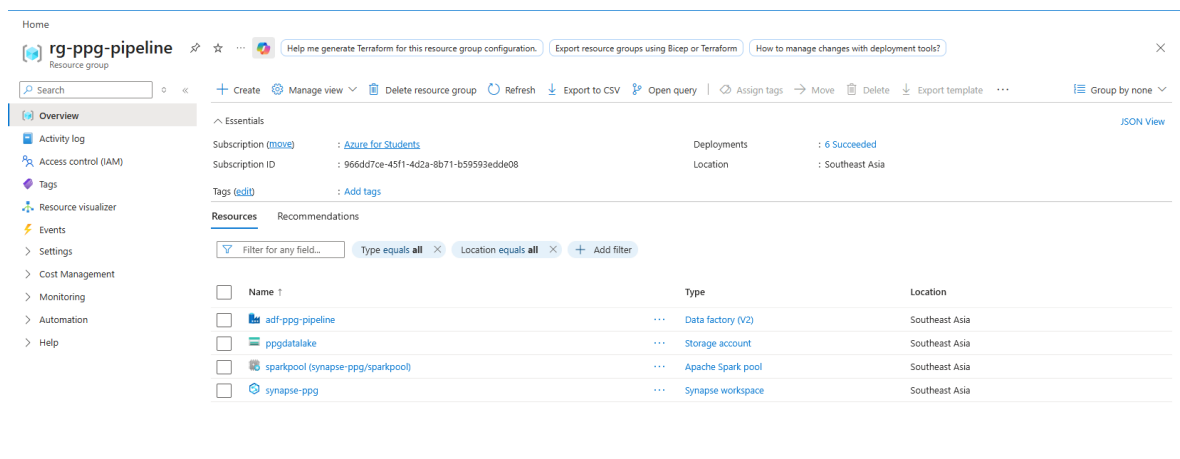


Figure 5.2.1.1: Azure Resource Group `rg-ppg-pipeline`

The storage account was configured with three dedicated containers corresponding to each layer of the Medallion Architecture, as shown in Figure 5.2.1.2.

☐	 \$logs
☐	 bronze
☐	 gold
☐	 silver

Figure 5.2.1.2: ADLS Gen2 Containers

Two Linked Services were configured in ADF Studio to establish authenticated connections to the project's data stores. The first, `ls_adls_ppg`, connects to `ppgdatalake` using System-Assigned Managed Identity authentication and serves as the connection reference for all Bronze Layer datasets. The second, `ls_synapse_ppg`, connects to the Synapse Analytics Serverless SQL Pool via the endpoint `synapse-ppg-ondemand.sql.azuresynapse.net`, targeting the `ppg_gold` database using SQL authentication. Both linked services were validated using the Test Connection function prior to publication, as shown in Figure 5.2.1.3.

Name ↑↓	Type ↑↓
 <code>ls_adls_ppg</code>	Azure Data Lake Storage Gen2
 <code>ls_synapse_ppg</code>	Azure SQL Database

Figure 5.2.1.3: Configured Linked Services in Azure Data Factory

5.2.2 Silver Layer — RA/MC Logic

The Silver layer RA and MC component was implemented using Azure Synapse Serverless SQL by replacing the originally planned PySpark notebook due to Spark pool vCore quota constraints in the project environment. It was designed to identify materials where stock levels exceeded actual operational demand, detect materials that had been idle for an extended period or had passed their lot expiry date, and calculate the total financial write-down provision required for all affected materials. This implementation is contained in a single SQL script named `M2_silver_ra_mc_logic` which creates a reusable SQL view called `vw_silver_ra_mc_logic` in the `ppg_gold` database as shown in Figure 5.2.2.1.

```

CREATE OR ALTER VIEW vw_silver_ra_mc_logic AS
WITH materials AS (
    SELECT DISTINCT *
    FROM OPENROWSET(
        BULK 'https://ppgdatalake.dfs.core.windows.net/bronze/validated/materials/*.csv',
        FORMAT = 'CSV',
        PARSER_VERSION = '2.0',
        FIRSTROW = 2
    )
    WITH (
        material_id VARCHAR(20) COLLATE Latin1_General_100_BIN2_UTF8,
        material_name VARCHAR(100) COLLATE Latin1_General_100_BIN2_UTF8,
        category VARCHAR(50) COLLATE Latin1_General_100_BIN2_UTF8,
        unit_of_measure VARCHAR(20) COLLATE Latin1_General_100_BIN2_UTF8,
        reorder_level INT,
        lead_time_days INT,
        unit_cost FLOAT
    ) AS m
),
inventory AS (
    SELECT
        TRY_CONVERT(DATE, snapshot_date_raw, 103) AS snapshot_date,
        material_id,
        warehouse_location,
        quantity_on_hand,
        TRY_CONVERT(DATE, lot_expiry_date_raw, 103) AS lot_expiry_date
    FROM OPENROWSET(
        BULK 'https://ppgdatalake.dfs.core.windows.net/bronze/validated/inventory_snapshot/*.csv',
        FORMAT = 'CSV',
        PARSER_VERSION = '2.0',
        FIRSTROW = 2
    )
    WITH (
        snapshot_date_raw VARCHAR(20) COLLATE Latin1_General_100_BIN2_UTF8,
        material_id VARCHAR(20) COLLATE Latin1_General_100_BIN2_UTF8,
        warehouse_location VARCHAR(50) COLLATE Latin1_General_100_BIN2_UTF8,
        quantity_on_hand INT,
        lot_expiry_date_raw VARCHAR(30) COLLATE Latin1_General_100_BIN2_UTF8
    ) AS i
),
production AS (
    SELECT DISTINCT *
    FROM OPENROWSET(
        BULK 'https://ppgdatalake.dfs.core.windows.net/bronze/validated/production_orders/*.csv',
        FORMAT = 'CSV',
        PARSER_VERSION = '2.0',
        FIRSTROW = 2
    )
    WITH (
        production_order_id VARCHAR(20) COLLATE Latin1_General_100_BIN2_UTF8,
        production_date DATE,
        finished_product VARCHAR(100) COLLATE Latin1_General_100_BIN2_UTF8,
        material_id VARCHAR(20) COLLATE Latin1_General_100_BIN2_UTF8,
        quantity_consumed INT
    ) AS p

```

Figure 5.2.2.1: SQL view creation for vw_silver_rc_mc_logic

The view reads directly from the three Bronze validated CSV files includes **materials.csv**, **inventory_snapshot.csv**, and **production_orders.csv** that stored in Azure Data Lake Storage Gen2. The materials dataset was used to obtain unit cost, reorder level, lead time days, and material category. The inventory snapshot dataset was used to obtain the latest quantity on hand and lot expiry date for each material. The production orders dataset was used to calculate 12-month material consumption and identify the last date each material was consumed.

Although the Bronze layer had already applied data quality checks, additional safeguards were retained in the Silver layer as a precaution. Date fields in the inventory snapshot were

stored in DD/MM/YYYY string format and required conversion to DATE using TRY_CONVERT with style code 103, where empty or invalid values safely return NULL. UTF-8 collation and SELECT DISTINCT applied to prevent encoding errors and remove any duplicates records.

5.2.3 Silver Layer — Stockout & Sales Impact

The stock out and sales impact process was created during the implementation of the Silver Layer. This process was created to identify raw material which is below reorder level, identify raw material at risk of stockout and trace the potential effect of raw material stockout on open customer sales order. This element was crucial, linking upstream stock levels with downstream customer order delivery.

Implementing this was done with Azure Synapse Serverless SQL. Synapse PySpark Notebook was used to plan the initial approach for implementing. The Spark notebook failed to run, however, due to a vCore quota limit of zero in the Azure workspace. So the solution was altered to a SQL-based one. Although the tool was changed, the original business logic was maintained.

After the mock data was increased, the updated validated Bronze Layer datasets were used as the source for this process. The inventory snapshot dataset increased from 240 rows to 300 rows after January, February, and March 2026 inventory records were added for all 20 materials. The production order dataset increased from 336 rows to 384 rows, while the sales order dataset increased from 48 rows to 67 rows. The purchase order dataset also increased from 40 rows to 50 rows, although it was not directly used in this Silver Layer stockout and sales impact logic.

The main datasets involved in this component were materials.csv, inventory_snapshot.csv, production_orders.csv, and sales_orders.csv. The material dataset was used to obtain reorder level, lead time days, unit cost, and material category. The inventory snapshot dataset was used to obtain the latest quantity on hand for each raw material. The production order data was utilized to measure the consumption of materials, and to associate raw materials with finished products. The sales order data was analysed to identify open sales orders which may be impacted by shortage risks. The SQL logic was using a fixed reference date (31 March 2026) because the updated data set was tied to March 2026. This way, the

current inventory snapshot and the 12-month production consumption period were calculated the same way. If the actual current date was used, the calculation period would change and the stockout outcomes may not be as consistent with the revised mock data set. Two SQL views were created in Azure Synapse. The first view, vw_silver_stockout_risk, was created to calculate reorder status, average daily consumption, stock cover days, and stockout risk. The second view, vw_silver_impacted_sales_orders, was created to trace affected open sales orders based on stockout-risk materials.

```
CREATE OR ALTER VIEW vw_silver_stockout_risk AS
WITH params AS (
    SELECT CAST('2026-03-31' AS DATE) AS reference_date
),
materials_raw AS (
    SELECT DISTINCT *
    FROM OPENROWSET(
        BULK 'https://ppgdatalake.dfs.core.windows.net/bronze/validated/materials/*.csv',
        FORMAT = 'CSV',
        PARSER_VERSION = '2.0',
        FIRSTROW = 2
    )
    WITH (
        material_id VARCHAR(20) COLLATE Latin1_General_100_BIN2_UTF8,
        material_name VARCHAR(100) COLLATE Latin1_General_100_BIN2_UTF8,
        category VARCHAR(50) COLLATE Latin1_General_100_BIN2_UTF8,
        unit_of_measure VARCHAR(20) COLLATE Latin1_General_100_BIN2_UTF8,
        reorder_level VARCHAR(50) COLLATE Latin1_General_100_BIN2_UTF8,
        lead_time_days VARCHAR(50) COLLATE Latin1_General_100_BIN2_UTF8,
        unit_cost VARCHAR(50) COLLATE Latin1_General_100_BIN2_UTF8,
        lot_expiry_date VARCHAR(50) COLLATE Latin1_General_100_BIN2_UTF8
    ) AS m
),
materials AS (
    SELECT
        LTRIM(RTRIM(material_id)) AS material_id,
        LTRIM(RTRIM(material_name)) AS material_name,
        LTRIM(RTRIM(category)) AS category,
        LTRIM(RTRIM(unit_of_measure)) AS unit_of_measure,
        TRY_CAST(reorder_level AS INT) AS reorder_level,
        TRY_CAST(lead_time_days AS INT) AS lead_time_days,
        TRY_CAST(unit_cost AS FLOAT) AS unit_cost,
        COALESCE(
            TRY_CONVERT(DATE, lot_expiry_date, 103),
            TRY_CONVERT(DATE, lot_expiry_date, 23),
            TRY_CONVERT(DATE, lot_expiry_date, 120)
        ) AS lot_expiry_date
    FROM materials_raw
)
```

Figure 5.2.3.1: SQL view creation for vw_silver_stockout_risk

Figure 5.2.3.1 shows the SQL view creation for vw_silver_stockout_risk. The view reads the validated Bronze Layer datasets from Azure Data Lake Storage and prepares the required fields for stockout risk analysis.

```

CREATE OR ALTER VIEW vw_silver_impacted_sales_orders AS
WITH params AS (
    SELECT CAST('2026-03-31' AS DATE) AS reference_date
),
production_raw AS (
    SELECT DISTINCT *
    FROM OPENROWSET(
        BULK 'https://ppgdatalake.dfs.core.windows.net/bronze/validated/production_orders/*.csv',
        FORMAT = 'CSV',
        PARSE_VERSION = '2.0',
        FIRSTROW = 2
    )
    WITH (
        production_order_id VARCHAR(20) COLLATE Latin1_General_100_BIN2_UTF8,
        production_date VARCHAR(50) COLLATE Latin1_General_100_BIN2_UTF8,
        finished_product VARCHAR(100) COLLATE Latin1_General_100_BIN2_UTF8,
        material_id VARCHAR(20) COLLATE Latin1_General_100_BIN2_UTF8,
        quantity_consumed VARCHAR(50) COLLATE Latin1_General_100_BIN2_UTF8
    ) AS p
),
production AS (
    SELECT
        LTRIM(RTRIM(production_order_id)) AS production_order_id,
        COALESCE(
            TRY_CONVERT(DATE, production_date, 103),
            TRY_CONVERT(DATE, production_date, 23),
            TRY_CONVERT(DATE, production_date, 120)
        ) AS production_date,
        LTRIM(RTRIM(finished_product)) AS finished_product,
        LTRIM(RTRIM(material_id)) AS material_id,
        TRY_CAST(quantity_consumed AS INT) AS quantity_consumed
    FROM production_raw
    WHERE production_order_id <> 'production_order_id'
    AND production_date <> 'production_date'
    AND material_id <> 'material_id'
    AND finished_product IS NOT NULL
    AND material_id IS NOT NULL

```

Figure 5.2.3.2: SQL view creation for vw_silver_impacted_sales_orders

Figure 5.2.3.2 shows the SQL view creation for vw_silver_impacted_sales_orders. This view was created to connect stockout-risk materials with finished products and open customer sales orders.

5.2.4 Gold Layer – Galaxy Schema

The Gold Layer is the last and most advanced layer of the data pipeline in the PPG Recoverable Assets (RA) and Inventory Risk Management (IRM) project. It is the job of this layer to convert the cleaned, enriched data in the Silver Layer into a structured, query-friendly data format that can be used directly for business reporting and analytical decision making.

Azure Synapse SQL was selected to build the Gold Layer, since SQL-based views can be created and queried on top of Silver Layer views without needing to move any physical

data or store in the cloud. This method is efficient, cost-effective, and appropriate for data in the project.

The overall design of the Gold Layer is based on the Galaxy Schema design, well-known as Fact Constellation Schema. A Galaxy Schema is a way to better support the multi-subject nature of the project's business questions. This allows different business subjects like inventory risk analysis and sales order impact to be modelled independently while reusing the same descriptive dimensions, where it can reduce redundancy and enable cross-subject analysis in Power BI.

The Gold Layer includes seven objects, which are five dimension views and two fact views, where all the objects created using ppg_gold database in Azure Synapse SQL Analytics as shown in Table 5.2.4.1.

Table 5.4.2.1: Gold Layer Overview

Object Name	Type	Business Subject	Purpose
dim_material	View	Shared	Material attributes
dim_supplier	View	Shared	Supplier details
dim_customer	View	Sales Orders	Unique customer names from sales orders
dim_warehouse	View	Shared	Warehouse locations
dim_date	View	Shared	Date breakdown by year, quarter, month and week
fact_inventory_status	View	Inventory Risk	Central fact views that answer all Q1 - Q6
fact_sales_impact	View	Sales Impact	Central fact views that answers Q7

Both fact views shared two-dimension which are the dim_material and dim_date to form the galaxy structure. The fact_sales_impact view also shares the dim_customer for filtering the customer's name from sales order in the Power BI dashboard. The dim_warehouse view was also shared in both facts view to enable the warehouse analysis.

5.2.5 Power BI Dashboard

The final presentation layer of the PPG inventory analytics pipeline is the Power BI dashboard. It is directly linked to the Gold Layer views hosted on Azure Synapse Analytics and demonstrates the results of all 7 business questions in an interactive visual format. Power BI Desktop was used to build the dashboard which is organised into 2 pages, each addressing a unique category of inventory risk.

Before building the dashboard, a data model was established in Power BI by importing the Gold Layer views from the `ppg_gold` database. The imported tables are `fact_inventory_status`, `fact_sales_impact`, `dim_material`, `dim_supplier`, `dim_customer`, `dim_date`, `dim_warehouse`, `vw_silver_ra_mc_logic`, `vw_silver_stockout_risk`, and `vw_silver_impacted_sales`. Some issues were encountered due to the multiple analytical views with overlapping material identifiers that standard Power BI relationships were not enough to generate filter context correctly across all visuals. Hence, TREATAS-based DAX measures were deployed to resolve the issue. These measures explicitly map filter context from `dim_material` into each view table without relying on model relationships.

The dashboard is designed and structured across two pages which are Materials Management and Inventory Health Tracker and Operational Risk. Page 1 covers Q1 to Q5 by presenting materials below reorder level, recoverable assets, MC dead stock, MC expired materials, and the total financial provision amount while Page 2 covers Q6 and Q7 by presenting materials at stockout risk and the customer orders impacted by those shortages.

5.3 Coding of system's main functions

This section documents the technical implementation of the automated pipeline. Key components include Azure Data Factory orchestration, data cleansing logic, multi-conditional SQL views for business rule enforcement, and the custom DAX measures used for dashboard reporting.

5.3.1 Bronze Ingestion & Quality Check

This section describes the implementation of the two core components of the Bronze Layer: the ingestion pipeline `pl_bronze_ingestion` and the Data Flow `df_bronze_quality`. Together, these components automate the transfer of raw source data from the landing zone into a validated, quality-assured state ready for Silver Layer processing.

5.3.1.1 Source Data Upload

Prior to pipeline execution, the six synthetic CSV source files were manually uploaded to the `bronze` container of `ppgdatalake` under the structured directory path `bronze/raw/<entity_name>/`. A dedicated subdirectory was created per file to isolate each data source and support individual dataset referencing within ADF. Table 5.3.1.1 summarises the uploaded source files and their respective characteristics.

Table 5.3.1.1.1 Source CSV File in Bronze Container

File	Directory Path	Primary Key
<code>materials.csv</code>	<code>bronze/raw/materials/</code>	<code>material_id</code>
<code>inventory_snapshot.csv</code>	<code>bronze/raw/inventory_snapshot/</code>	<code>snapshot_date</code> + <code>material_id</code>
<code>purchase_orders.csv</code>	<code>bronze/raw/purchase_orders/</code>	<code>po_id</code>
<code>production_orders.csv</code>	<code>bronze/raw/production_orders/</code>	<code>production_order_id</code>
<code>suppliers.csv</code>	<code>bronze/raw/suppliers/</code>	<code>supplier_id</code>
<code>sales_orders.csv</code>	<code>bronze/raw/sales_orders/</code>	<code>sales_order_id</code>

It is worth noting that `inventory_snapshot` requires a composite primary key comprising both `snapshot_date` and `material_id` as neither field alone uniquely identifies a record which the same material appears across multiple monthly snapshots. All six files were verified to be present in the portal before pipeline construction commenced, as shown in Figure 5.3.1.1.1.

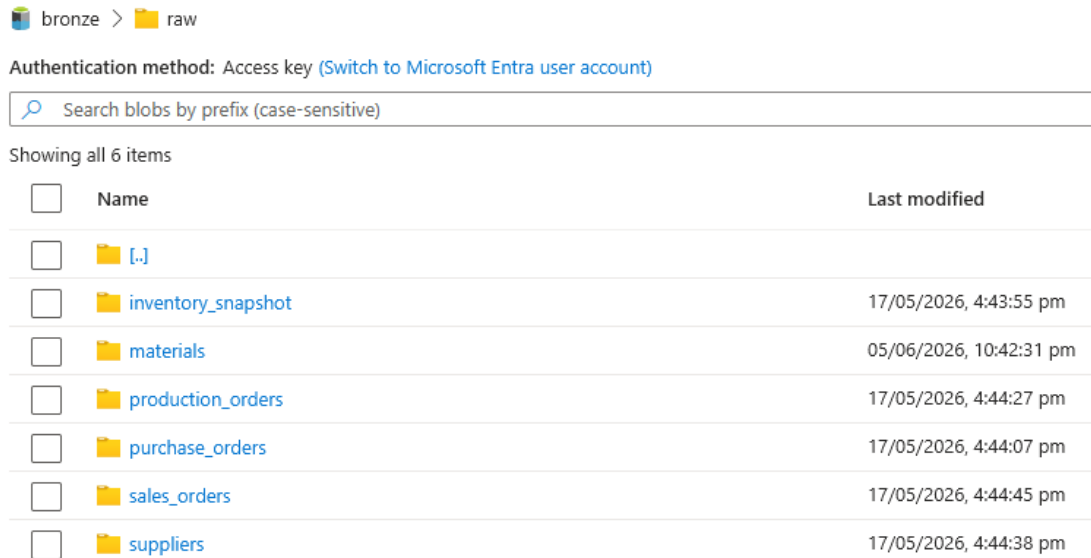


Figure 5.3.1.1.1: Six Source CSV Files in Raw Directory

5.3.1.2 Bronze Ingestion Pipeline

The pipeline `pl_bronze_ingestion` was constructed in ADF Studio to automate the transfer of all six raw CSV files from `bronze/raw/` to `bronze/ingested/`. The pipeline contains six Copy Data activities, one per source file, all configured to execute in parallel with no sequential dependency between them. Parallel execution was deliberately chosen to minimise total ingestion time and to reflect the independence of each source entity. Each Copy Data activity references a dedicated source dataset pointing to the raw subdirectory, and a corresponding sink dataset pointing to the ingested subdirectory. All datasets were configured with the DelimitedText format, First row as header enabled, and schema imported from the connection store. The pipeline canvas is shown in Figure 5.3.1.2.1.

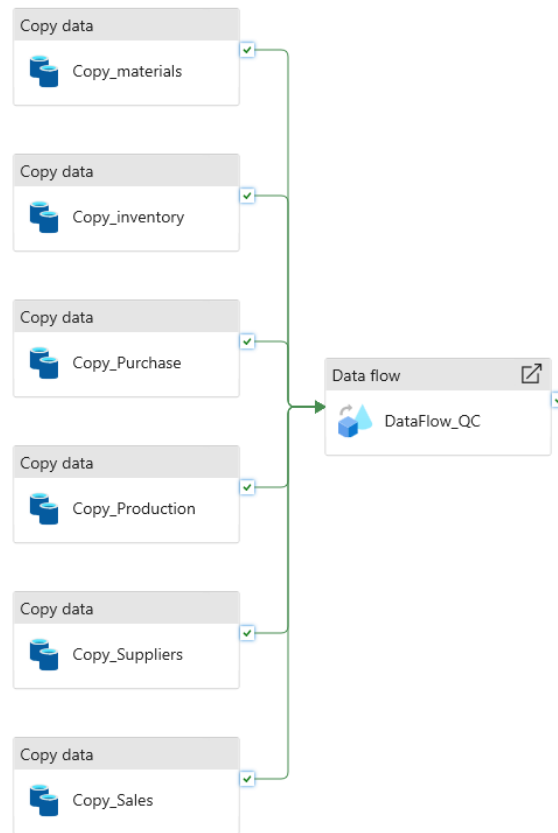


Figure 5.3.1.2.1: Bronze Ingestion Pipeline

Upon triggering the pipeline, all six CSV files were successfully copied from bronze/raw/ to bronze/ingested/, as confirmed in Figure 5.3.1.2.2.

bronze > ingested

Authentication method: Access key ([Switch to Microsoft Entra user account](#))

Showing all 6 items

☐	Name	Last modified
☐	[-.]	
☐	inventory_snapshot	19/05/2026, 9:26:05 pm
☐	materials	19/05/2026, 9:26:06 pm
☐	production_orders	19/05/2026, 9:26:05 pm
☐	purchase_orders	19/05/2026, 9:26:06 pm
☐	sales_orders	19/05/2026, 9:26:05 pm
☐	suppliers	19/05/2026, 9:26:10 pm

Figure 5.3.1.2.2: Six Source CSV Files in Ingested Directory

5.3.1.3 Data Quality Checks

Following ingestion, data quality validation was implemented using an ADF Data Flow named `df_bronze_quality`. A single Data Flow canvas was constructed containing twelve independent transformation chains, six for null and quality flag checking and six for duplicate detection, designed one pair per source entity.

Quality Flag Chains

Each quality chain begins with a dedicated Source transformation referencing the corresponding ingested dataset. A Derived Column transformation named `QualityFlags` then evaluates each critical field for null values using the ADF expression `iif(isNull(column), 1, 0)`. For example, the expressions applied for materials dataset are as follows:

```
is_null_material_id = iif(isNull(material_id), 1, 0)
is_null_unit_cost   = iif(isNull(unit_cost), 1, 0)
is_null_reorder     = iif(isNull(reorder_level), 1, 0)
is_null_category    = iif(isNull(category), 1, 0)
is_null_lead        = iif(isNull(lead_time_days), 1, 0)
```

A Conditional Split transformation then routes rows into two output streams based on these flags. Rows where any mandatory field is null are directed to the `BadRows` stream and sinked to `bronze/quarantine/<entity>/`. Rows where all mandatory fields are present are directed to the `GoodRows` stream and sinked to `bronze/validated/<entity>/`. The six quality flag chains on the Data Flow canvas are shown in Figure 5.3.1.3.1.



Figure 5.3.1.3.1: Data Flow Canvas for Quality Flag

Duplicate Detection

Each duplicate detection chain branches independently from a separate Source transformation pointing to the same ingested dataset. This architectural separation is necessary because chaining the Aggregate after the QualityFlags Derived Column would cause it to operate on an annotated schema and produce incorrect grouping results. An Aggregate transformation groups rows by the primary key of each entity, with `row_count = count(1)` as the computed column. A Filter transformation then retains only rows where `row_count > 1`, representing genuine duplicates. Table 5.3.1.3.1 lists the grouping keys applied per entity.

Table 5.3.1.3.1 Primary Keys Used for Duplicate Detection

Source File	Grouping Key
-------------	--------------

materials.csv	material_id
inventory_snapshot.csv	snapshot_date + material_id
purchase_orders.csv	po_id
production_orders.csv	production_order_id
suppliers.csv	supplier_id
sales_orders.csv	sales_order_id

Detected duplicates are sinked to bronze/quarantine/duplicates/<entity>/. The six duplicate detection chains are shown in Figure 5.3.1.3.2.

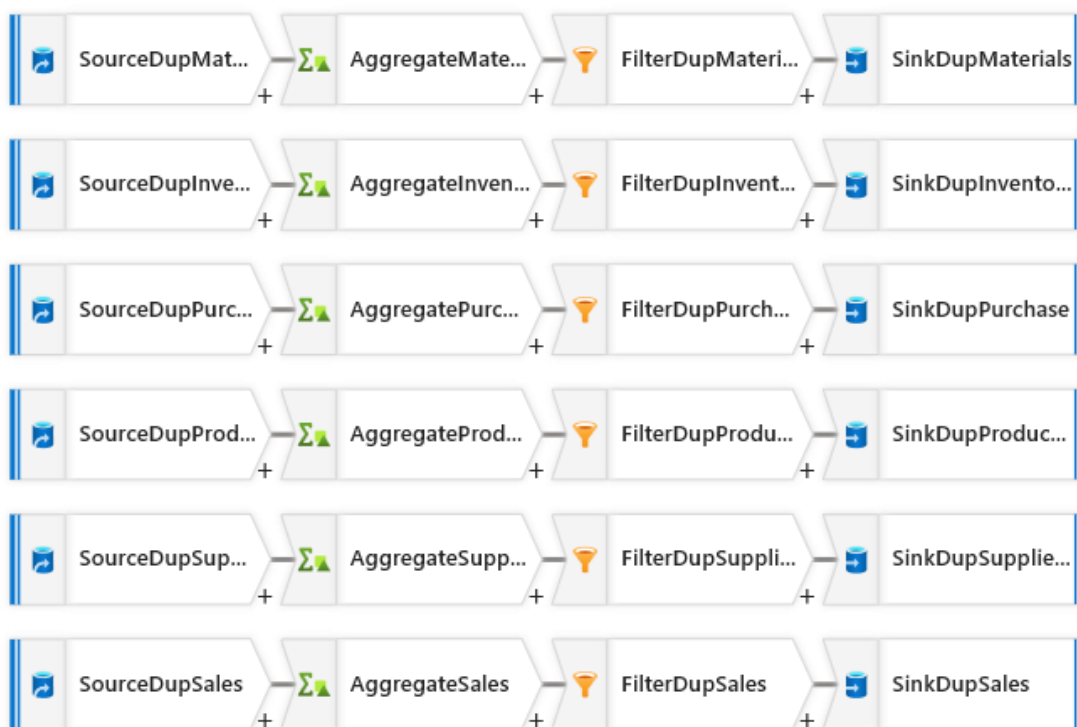


Figure 5.3.1.3.2: Data Flow Canvas for Duplicate Detection

The Data Flow activity df_bronze_quality was linked to pl_bronze_ingestion via a success dependency arrow, ensuring that quality checks execute only after all six Copy Data activities have completed successfully. Upon execution, the bronze/validated/ and bronze/quarantine/ output directories were populated as shown in Figures 5.3.1.3.3 and 5.3.1.3.4.

bronze > validated

Authentication method: Access key ([Switch to Microsoft Entra user account](#))

Search blobs by prefix (case-sensitive)

Showing all 6 items

☐	Name	Last modified
☐	[.]	
☐	inventory_snapshot	20/05/2026, 11:35:37 pm
☐	materials	20/05/2026, 11:35:44 pm
☐	production_orders	20/05/2026, 11:35:54 pm
☐	purchase_orders	20/05/2026, 11:35:23 pm
☐	sales_orders	21/05/2026, 4:58:13 pm
☐	suppliers	20/05/2026, 11:35:13 pm

Figure 5.3.1.3.3: Validated Output Files in Bronze Container

bronze > quarantine

Authentication method: Access key ([Switch to Microsoft Entra user account](#))

Search blobs by prefix (case-sensitive)

Showing all 7 items

☐	Name	Last modified
☐	[.]	
☐	duplicates	21/05/2026, 6:32:19 pm
☐	inventory_snapshot	20/05/2026, 11:35:35 pm
☐	materials	20/05/2026, 11:35:47 pm
☐	production_orders	20/05/2026, 11:35:57 pm
☐	purchase_orders	20/05/2026, 11:35:28 pm
☐	sales_orders	20/05/2026, 11:35:01 pm
☐	suppliers	20/05/2026, 11:35:17 pm

Figure 5.3.1.3.4: Quarantined Output Files in Bronze Container

5.3.2 RA & MC Business Logic

The main function of this component was divided into four parts that including 12-month consumption calculation, Recoverable Asset detection, Magna Carta Dead Stock detection, Magna Carta Expired detection, and financial provision calculation. These functions were implemented using SQL logic within the vw_silver_ra_mc_logic view in Azure Synapse Serverless SQL. A fixed reference date of 2026-03-31 was applied consistently across all calculations that include date instead of the current system date, as the dataset is anchored to March 2026. This ensures the 12-month consumption window, idle period calculation, and expiry date comparisons produce consistent results.

```
latest_inventory AS (  
  SELECT  
    i.*,  
    ROW_NUMBER() OVER (  
      PARTITION BY i.material_id  
      ORDER BY i.snapshot_date DESC  
    ) AS rn  
  FROM inventory i  
)
```

Figure 5.3.2.1: Latest inventory selection SQL logic

The inventory snapshot dataset was found to have 12 monthly snapshots for each material, so it was important to find the latest snapshot for each material prior to any calculation work. The latest_inventory used the ROW_NUMBER() window function follow with partitioned by material_id and ordered by the snapshot_date in descending order as shown in Figure 5.3.2.1. This was an assigned rank with the highest number showing the most recently snapshot row for each material. Only rows with rn = 1 were passed to the final calculation, so that all flags and quantities were calculated with the current stock position, not based on historic data.

```
consumption_12m AS (  
  SELECT  
    material_id,  
    SUM(quantity_consumed) AS total_consumed_12m,  
    MAX(production_date) AS last_consumed_date  
  FROM production  
  WHERE production_date >= DATEADD(MONTH, -12, CAST('2026-03-31' AS DATE))  
  GROUP BY material_id  
)
```

Figure 5.3.3.2 12-month consumption calculation SQL logic

The consumption calculation step used production order records to determine the total quantity consumed over the last 12 months for each material as illustrated in Figure 5.3.2.2.

12-Month Consumption = Total Quantity Consumed

WHERE production_date >= Reference Date - 12 months

Only the production orders with a production_date of 2025-03-31 or later were selected by follow the formula production_date >= DATEADD(MONTH, -12, CAST('2026-03-31' AS DATE)). SUM(quantity_consumed) accumulated the total units consumed per material, and MAX(production_date) captured the most recent date each material was used in production, which is required for dead stock detection in Q3.

```
idle_stats AS (  
  SELECT  
    m.material_id,  
    c12m.last_consumed_date,  
    CASE  
      WHEN c12m.last_consumed_date IS NULL THEN 999.0  
      ELSE CAST(DATEDIFF(MONTH, c12m.last_consumed_date, CAST('2026-03-31' AS DATE))) AS FLOAT  
    END AS months_since_consumed  
  FROM materials m  
  LEFT JOIN consumption_12m c12m ON m.material_id = c12m.material_id  
)
```

Figure 5.3.2.3 Months since consumed calculation SQL logic

As shown in Figure 5.3.2.3, the idle calculation step calculated how many months had passed since each material was last consumed. The calculation logic

Month since Consumed = DATEDIFF between Last Consumed Date and Reference Date

If never consumed → Months Since Consumed = 999.0

A LEFT JOIN was applied between the materials and consumption calculation results to ensure all 20 materials were included even if they had no production records. The number of months was calculated using DATEDIFF(MONTH, last_consumed_date, CAST('2026-03-31' AS DATE)). The same fixed reference date of 2026-03-31 was applied here to maintain consistency with the consumption calculation window. Materials where last_consumed_date was NULL, meaning they had never appeared in any production order. They were assigned a placeholder value of 999.0 to ensure they would always exceed the 9-month dead stock threshold.

```

-- Q2: Recoverable Asset Calculations
ISNULL(c12m.total_consumed_12m, 0) AS consumption_12m,
CASE
    WHEN li.quantity_on_hand > ISNULL(c12m.total_consumed_12m, 0)
    THEN li.quantity_on_hand - ISNULL(c12m.total_consumed_12m, 0)
    ELSE 0
END AS excess_quantity,
CASE
    WHEN li.quantity_on_hand > ISNULL(c12m.total_consumed_12m, 0) THEN 1
    ELSE 0
END AS is_recoverable_asset,

```

Figure 5.3.2.4 Recoverable Asset flag SQL logic

As shown in Figure 5.3.2.4, the Q2 logic compared each material's current quantity_on_hand from the latest inventory snapshot against its total 12-month consumption.

Excess Quantity = Quantity on Hand – 12-Month Consumption

Recoverable Asset = 1 if Quantity on Hand > 12-Month Consumption, else 0

ISNULL(c12m.total_consumed_12m, 0) ensured that materials with no production records were treated as zero consumed rather than NULL, preventing calculation errors. A material was flagged as is_recoverable_asset = 1 when quantity on hand exceeded 12-month consumption, indicating PPG held more stock than it could realistically use within a year. The difference was stored as excess_quantity to show the exact surplus amount.

```

-- Q3: Magna Carta Dead Stock Calculations
ids.last_consumed_date,
ids.months_since_consumed,
CASE
    WHEN ids.months_since_consumed >= 9 AND li.lot_expiry_date IS NULL THEN 1
    ELSE 0
END AS is_mc_dead_stock,

```

Figure 5.3.2.5: Q3 MC Dead Stock flag SQL logic

As shown in Figure 5.3.2.5, the Q3 logic flagged a material as is_mc_dead_stock = 1 when two conditions were simultaneously true: months_since_consumed >= 9 and lot_expiry_date IS NULL.

MC Dead Stock = 1

if Months since Consumed >= 9 AND lot_expiry_date is NULL, else 0

Both conditions had to be met together where a material idle for 9 or more months but having an expiry date would be handled separately under Q4, not Q3. The last consumed

date and months since consumed values were also included in the output to provide supporting detail for each flagged material.

```
-- Q4: Magna Carta Expired Calculations
CASE
  WHEN li.lot_expiry_date IS NOT NULL AND li.lot_expiry_date < CAST('2026-03-31' AS DATE) THEN 1
  ELSE 0
END AS is_mc_expired,
```

Figure 5.3.2.6: Q4 MC Expired flag calculation

As shown in Figure 5.3.2.6, the Q4 logic flagged a material as `is_mc_expired = 1` when its `lot_expiry_date` was not NULL and had already passed relative to the fixed reference date.

MC Expired = 1 if `lot_expiry_date` is not NULL AND

`lot_expiry_date < Reference Date`, else 0

`CAST('2026-03-31' AS DATE)` was used as the reference point for comparison instead of the current system date. A material with a lot expiry date before 2026-03-31 was classified as expired, while materials with future expiry dates or no expiry date were not flagged under Q4.

```
-- Q5: Composite Flags and Financial Provision Logic
CASE
  WHEN (ids.months_since_consumed >= 9 AND li.lot_expiry_date IS NULL)
  OR (li.lot_expiry_date IS NOT NULL AND li.lot_expiry_date < CAST('2026-03-31' AS DATE)) THEN 1
  ELSE 0
END AS is_mc_material,
CASE
  WHEN (ids.months_since_consumed >= 9 AND li.lot_expiry_date IS NULL)
  OR (li.lot_expiry_date IS NOT NULL AND li.lot_expiry_date < CAST('2026-03-31' AS DATE))
  THEN CAST(li.quantity_on_hand AS FLOAT) * m.unit_cost
  ELSE 0.0
END AS mc_provision_amount
```

Figure 5.3.2.7: Q5 composite MC flag and provision amount calculation

As shown in Figure 5.3.2.7, the Q5 logic applied a composite flag `is_mc_material` that equals 1 when a material qualifies as either MC Dead Stock or MC Expired.

MC Provision Amount = Quantity on Hand × Unit Cost

Total MC Provision = SUM of all `mc_provision_amount` where `is_mc_material = 1`

Either condition alone was sufficient to trigger the flag. The financial provision amount was then calculated as a 100% write-down by multiplying `quantity_on_hand` by `unit_cost`.

CAST(li.quantity_on_hand AS FLOAT) was applied to ensure decimal precision in the multiplication. Materials that did not meet either MC condition received a provision amount of 0.0.

```
SELECT ***  
FROM latest_inventory li  
JOIN materials m ON li.material_id = m.material_id  
LEFT JOIN consumption_12m c12m ON li.material_id = c12m.material_id  
LEFT JOIN idle_stats ids ON li.material_id = ids.material_id  
WHERE li.rn = 1;  
GO
```

Figure 5.3.2.8 Final SELECT JOIN structure and filter

As shown in Figure 5.3.2.8, the final SELECT joined all calculation steps together to produce one output row per material. An inner JOIN was applied between the latest inventory and materials to ensure only materials existing in the master list were included. LEFT JOIN was applied for both the consumption and idle calculation results to preserve all materials even if they had no production history. The WHERE li.rn = 1 filter was applied to retain only the most recent inventory snapshot per material, producing a clean single-row output per material containing all computed flags and financial values ready for the Gold layer.

5.3.3 Stockout & Sales Order Impact Logic

The main function of this component was divided into three parts: below reorder level detection, stockout risk calculation, and impacted sales order tracing. These functions were implemented using SQL logic in Azure Synapse Serverless SQL.

First, below reorder level detection was performed by comparing the latest quantity on hand with the reorder level in the material dataset. The latest inventory snapshot for each material was selected using the ROW_NUMBER() function. After the dataset update, the latest inventory snapshot used by the system was 31 March 2026 for all 20 materials. This verified that the new inventory record for January, February and March 2026 were added to the Silver Layer logic. A material was identified as below reorder level if the on hand quantity was below the reorder level.

Second, the risk of stockout was computed based on the average daily consumption and stock cover days. The total amount consumed for each material was determined by production order records for 12 months around a fixed reference date of 31 March 2026.

The average daily consumption was then derived by dividing the total consumed quantity by the active production date range. After that, stock cover days were calculated using the available stock quantity and average daily consumption.

$$\text{Stock Cover Days} = \text{Quantity on Hand} / \text{Average Daily Consumption}$$

A material was classified as having stockout risk when its stock cover days were lower than its lead time days. This means that the current available stock may not be sufficient to cover material usage before replenishment can arrive.

$$\text{Stockout Risk} = \text{Stock Cover Days} < \text{Lead Time Days}$$

```
ISNULL(cs.total_consumed, 0) AS total_consumed,
ISNULL(cs.avg_daily_consumption, 0.001) AS avg_daily_consumption,

CASE
  WHEN ISNULL(cs.avg_daily_consumption, 0.001) > 0
  THEN CAST(li.quantity_on_hand AS FLOAT) / ISNULL(cs.avg_daily_consumption, 0.001)
  ELSE 9999
END AS stock_cover_days,

CASE
  WHEN
    CASE
      WHEN ISNULL(cs.avg_daily_consumption, 0.001) > 0
      THEN CAST(li.quantity_on_hand AS FLOAT) / ISNULL(cs.avg_daily_consumption, 0.001)
      ELSE 9999
    END < m.lead_time_days
  THEN 1
  ELSE 0
END AS is_stockout_risk
```

Figure 5.3.3.1: Stockout risk calculation SQL logic

Figure 5.3.3.1 shows the SQL logic used to calculate average daily consumption, stock cover days, and stockout risk. The calculation compares stock cover days with lead time days to determine whether a material is at risk of stockout.

Third, impacted sales order tracing was performed by linking stockout-risk materials to finished products through the production order dataset. After the finished products were identified, they were linked to open sales orders from the sales order dataset. This allowed the system to identify which customer orders could be affected by materials with stockout risk.

Stockout-risk material:

- Finished product
- Open sales order
- Affected customer

```
✓ SELECT
    os.sales_order_id,
    os.order_date,
    os.customer_name,
    os.finished_product,
    os.quantity_ordered,
    os.required_delivery_date,
    os.status,
    pml.material_id,
    arm.material_name,
    arm.category,
    arm.quantity_on_hand,
    arm.reorder_level,
    arm.avg_daily_consumption,
    arm.stock_cover_days,
    arm.lead_time_days,
    arm.is_stockout_risk
FROM open_sales os
✓ JOIN product_material_link pml
    ON os.finished_product = pml.finished_product
✓ JOIN at_risk_materials arm
    ON pml.material_id = arm.material_id;
```

Figure 5.3.3.2: Impacted sales order tracing SQL logic

Figure 5.3.3.2 shows the SQL join logic used to trace impacted sales orders. Stockout-risk materials were linked to finished products, and the finished products were then linked to open customer sales orders.

Several data preparation steps were included in the SQL script to improve reliability. Duplicate records were reduced using SELECT DISTINCT. Date fields were read as text and converted using TRY_CONVERT because different date formats were found in the source datasets. The production order date used the yyyy-mm-dd format, while other date fields used different formats. In the updated implementation, the SQL logic was also adjusted to use a fixed reference date of 31 March 2026. This was required to ensure that

the 12-month consumption window and the latest inventory snapshot were aligned with the updated mock dataset. After the date conversion and reference date logic were updated, the consumption, stock cover, and stockout risk calculations were generated correctly.

5.3.4 Galaxy Schema & Fact Table Loading

This section documents the key SQL logic implemented to build the Gold Layer galaxy schema. All scripts were developed and executed using Azure Synapse Analytics SQL , connected to the ppg_gold database.

All views use CREATE OR ALTER VIEW, which is the correct approach for Synapse SQL. Plain CREATE TABLE is not supported on SQL Pool as it does not allow persistent physical tables. All Gold Layer objects are non-persistent views that compute results on-demand from the Bronze and Silver Layers.

The reference date is fixed at 2026-03-31 across all views to align with the dataset anchor date used to ensure consistent results regardless of when the scripts are executed.

Dimension View: dim_material

The dim_material view reads directly from the Bronze validated materials CSV files using OPENROWSET as shown in Figure 5.3.4.1. It provides descriptive attributes for each material including its category, unit of measure of reorder level, lead time and unit cost. These attributes are used as reference data by the fact view and by downstream Power BI visuals.

```

-- 1. Material Dimension (shared by fact_inventory_status + fact_sales_impact)
CREATE OR ALTER VIEW dim_material AS
SELECT
    material_id,
    material_name,
    category,
    unit_of_measure,
    reorder_level,
    lead_time_days,
    unit_cost
FROM OPENROWSET(
    BULK 'https://ppgdatalake.dfs.core.windows.net/bronze/validated/materials/*.csv',
    FORMAT = 'CSV',
    PARSE_VERSION = '2.0',
    FIRSTROW = 2
) WITH (
    material_id          VARCHAR(20) COLLATE Latin1_General_100_BIN2_UTF8,
    material_name        VARCHAR(100) COLLATE Latin1_General_100_BIN2_UTF8,
    category             VARCHAR(50) COLLATE Latin1_General_100_BIN2_UTF8,
    unit_of_measure      VARCHAR(20) COLLATE Latin1_General_100_BIN2_UTF8,
    reorder_level        INT,
    lead_time_days       INT,
    unit_cost            FLOAT
) AS r;
GO

```

Figure 5.3.4.1: Material Dimension View SQL Scripts

Dimension View: dim_supplier

The dim_supplier view reads from the Bronze validated suppliers for CSV files. It provides supplier-level context including supplier name, country of origin, and reliability rating. This view supports potential future analysis of supplier performance and its relationship to stockout risk. The creation of supplier dimension view is shown in Figure 5.3.4.2.

```

-- 2. Supplier Dimension (standalone – supports future supplier risk analysis)
CREATE OR ALTER VIEW dim_supplier AS
SELECT
    supplier_id,
    supplier_name,
    country,
    reliability_rating
FROM OPENROWSET(
    BULK 'https://ppgdatalake.dfs.core.windows.net/bronze/validated/suppliers/*.csv',
    FORMAT = 'CSV',
    PARSER_VERSION = '2.0',
    FIRSTROW = 2
) WITH (
    supplier_id          VARCHAR(20)  COLLATE Latin1_General_100_BIN2_UTF8,
    supplier_name        VARCHAR(100) COLLATE Latin1_General_100_BIN2_UTF8,
    country              VARCHAR(50)  COLLATE Latin1_General_100_BIN2_UTF8,
    reliability_rating    VARCHAR(10)  COLLATE Latin1_General_100_BIN2_UTF8
) AS r;
GO

```

Figure 5.3.4.2: Supplier Dimension View SQL Scripts

Dimension View: dim_customer

The dim_customer view extracts distinct customer names from the sales orders Bronze CSV. A DISTINCT as in Figure 5.3.4.3 is used to ensure that each customer appears only once in the dimension. All seven columns in the sales orders of CSV are declared in the WITH clause to satisfy the Synapse Serverless column mapping requirements, even though only customer_name is selected in the output.

```

-- 3. Customer Dimension (shared by fact_sales_impact for Q7 analysis)
CREATE OR ALTER VIEW dim_customer AS
SELECT DISTINCT
    customer_name
FROM OPENROWSET(
    BULK 'https://ppgdatalake.dfs.core.windows.net/bronze/validated/sales_orders/*.csv',
    FORMAT = 'CSV',
    PARSER_VERSION = '2.0',
    FIRSTROW = 2
) WITH (
    sales_order_id      VARCHAR(20)  COLLATE Latin1_General_100_BIN2_UTF8,
    order_date          VARCHAR(50)  COLLATE Latin1_General_100_BIN2_UTF8,
    customer_name        VARCHAR(100) COLLATE Latin1_General_100_BIN2_UTF8,
    finished_product     VARCHAR(100) COLLATE Latin1_General_100_BIN2_UTF8,
    quantity_ordered     VARCHAR(50)  COLLATE Latin1_General_100_BIN2_UTF8,
    required_delivery_date VARCHAR(50) COLLATE Latin1_General_100_BIN2_UTF8,
    status               VARCHAR(50)  COLLATE Latin1_General_100_BIN2_UTF8
) AS r;
GO

```

Figure 5.3.4.3: Customer Dimension View SQL Scripts

Dimension View: dim_warehouse

The dim_warehouse view is a new dimension introduced as part of the Galaxy Schema upgrade. It extracts the distinct warehouse location values from the inventory snapshot Bronze Layer CSV. This dimension is shared across both fact views, enabling the Power BI to filter and slice inventory risk metrics and sales order impact by warehouse location. In Figure 5.3.4.4, a WHERE clause filters out the column header row and NULL values to ensure only valid warehouse codes are included.

```
-- 4. Warehouse Dimension (NEW – extracted from inventory snapshot)
-- Shared by both fact views to support warehouse-level filtering in Power BI
CREATE OR ALTER VIEW dim_warehouse AS
SELECT DISTINCT
    warehouse_location
FROM OPENROWSET(
    BULK 'https://ppgdatalake.dfs.core.windows.net/bronze/validated/inventory_snapshot/*.csv'
    FORMAT = 'CSV',
    PARSER_VERSION = '2.0',
    FIRSTROW = 2
) WITH (
    snapshot_date_raw    VARCHAR(20) COLLATE Latin1_General_100_BIN2_UTF8,
    material_id          VARCHAR(20) COLLATE Latin1_General_100_BIN2_UTF8,
    warehouse_location   VARCHAR(50) COLLATE Latin1_General_100_BIN2_UTF8,
    quantity_on_hand     INT,
    lot_expiry_date_raw  VARCHAR(30) COLLATE Latin1_General_100_BIN2_UTF8
) AS r
WHERE warehouse_location IS NOT NULL
    AND warehouse_location <> 'warehouse_location';
GO
```

Figure 5.3.4.4: Warehouse Dimension View SQL Script

Dimension View: dim_date

The dim_date view was expanded as part of the Galaxy Schema upgrade to collect unique dates from four sources. The addition of sales order dates and required delivery dates from Q7 silver view enable Power BI to perform time-based filtering on both fact views using the same shared date dimension. A UNION of all four sources as in Figure 5.3.4.5 ensures all relevant dates are captured without duplicates.

```

-- 5. Dynamic Date Dimension (shared by both fact views)
-- Collects unique dates from both Silver views + sales orders
-- Reference date fixed at 2026-03-31 to match M2 and M3 scripts
CREATE OR ALTER VIEW dim_date AS
WITH unique_dates AS (
    -- Inventory snapshot dates from M2 silver view
    SELECT DISTINCT CAST(snapshot_date AS DATE) AS date_id
    FROM vw_silver_ra_mc_logic

    UNION

    -- Inventory snapshot dates from M3 silver view
    SELECT DISTINCT CAST(snapshot_date AS DATE) AS date_id
    FROM vw_silver_stockout_risk

    UNION

    -- Sales order dates from M3 Q7 silver view (for fact_sales_impact)
    SELECT DISTINCT CAST(order_date AS DATE) AS date_id
    FROM vw_silver_impacted_sales_orders
    WHERE order_date IS NOT NULL

    UNION

    -- Required delivery dates from M3 Q7 silver view
    SELECT DISTINCT CAST(required_delivery_date AS DATE) AS date_id
    FROM vw_silver_impacted_sales_orders
    WHERE required_delivery_date IS NOT NULL
)
SELECT
    date_id,
    DATEPART(YEAR, date_id) AS year,
    DATEPART(QUARTER, date_id) AS quarter,
    DATEPART(MONTH, date_id) AS month,
    DATENAME(MONTH, date_id) AS month_name,
    DATEPART(WEEK, date_id) AS week_of_year,
    DATEPART(WEEKDAY, date_id) AS day_of_week,
    DATENAME(WEEKDAY, date_id) AS day_name
FROM unique_dates;
GO

```

Figure 5.3.4.5: Date Dimension View SQL Scripts

Central Fact View: fact_inventory_status

The fact_inventory_status view is the central object of the Gold Layer. It merges outputs from two Silver Layer views: vw_silver_ra_mc_logic and vw_silver_stockout_risk. A FULL OUTER JOIN is used to ensure all 20 materials are represented regardless of whether they appear in one or both Silver views as shown in Figure 5.3.4.6.

```
CREATE OR ALTER VIEW fact_inventory_status AS
SELECT
    -- Shared keys (COALESCE handles materials only in one silver view)
    COALESCE(mc.material_id, st.material_id)
    | COLLATE Latin1_General_100_BIN2_UTF8 AS material_id,
    COALESCE(mc.snapshot_date, st.snapshot_date) AS snapshot_date,
    COALESCE(mc.warehouse_location, st.warehouse_location)
    | COLLATE Latin1_General_100_BIN2_UTF8 AS warehouse_location,
    COALESCE(mc.quantity_on_hand, st.quantity_on_hand) AS quantity_on_hand,

    -- Foreign keys for dimension joins
    -- JOIN dim_material ON fact_inventory_status.material_id = dim_material.material_id
    -- JOIN dim_date ON fact_inventory_status.snapshot_date = dim_date.date_id
    -- JOIN dim_warehouse ON fact_inventory_status.warehouse_location = dim_warehouse.warehouse_location

    -- M2 metrics: Recoverable Assets & Magna Carta Provision (Q2, Q3, Q4, Q5)
    CAST(ISNULL(mc.consumption_12m, 0) AS FLOAT) AS consumption_12m,
    CAST(ISNULL(mc.excess_quantity, 0) AS FLOAT) AS excess_quantity,
    CAST(ISNULL(mc.is_recoverable_asset, 0) AS BIT) AS is_recoverable_asset,
    CAST(ISNULL(mc.is_mc_dead_stock, 0) AS BIT) AS is_mc_dead_stock,
    CAST(ISNULL(mc.is_mc_expired, 0) AS BIT) AS is_mc_expired,
    CAST(ISNULL(mc.is_mc_material, 0) AS BIT) AS is_mc_material,
    CAST(ISNULL(mc.mc_provision_amount, 0) AS FLOAT) AS mc_provision_amount,
    CAST(ISNULL(mc.months_since_consumed, 0) AS FLOAT) AS months_since_consumed,
    mc.lot_expiry_date,
    mc.last_consumed_date,

    -- M3 metrics: Stockout Risk & Reorder Flags (Q1, Q6)
    CAST(ISNULL(st.is_below_reorder, 0) AS BIT) AS is_below_reorder,
    CAST(ISNULL(st.is_stockout_risk, 0) AS BIT) AS is_stockout_risk,
    CAST(ISNULL(st.stock_cover_days, 0) AS FLOAT) AS stock_cover_days,
    CAST(ISNULL(st.avg_daily_consumption, 0) AS FLOAT) AS avg_daily_consumption,
    CAST(ISNULL(st.total_consumed, 0) AS FLOAT) AS total_consumed,
    st.reorder_level,
    st.lead_time_days

FROM vw_silver_ra_mc_logic mc
FULL OUTER JOIN vw_silver_stockout_risk st
    ON mc.material_id COLLATE Latin1_General_100_BIN2_UTF8
    = st.material_id COLLATE Latin1_General_100_BIN2_UTF8
    AND CAST(mc.snapshot_date AS DATE) = CAST(st.snapshot_date AS DATE);
GO
```

Figure 5.3.4.6: Central Fact View (fact_inventory_status) SQL Scripts

Several important technical decisions were made in this view:

- FULL OUTER JOIN is used to ensure materials that appear in only one silver view are not excluded from the fact view. It was necessary for materials with no consumption history, where it appears in Member 2's view but might not match with Member 3's view.

- COALESCE was used on the material_id, snapshot_date, warehouse_location, and quantity_on_hand to handle the cases where a material only exists in one Silver view.
- COLLATE Latin1_General_100_BIN2_UTF8 was applied in VARCHAR and joined the keys to resolve the collation that potential to mismatch errors between two Silver views.
- ISNULL with default values was used for all flags and columns to prevent any NULL distribution into the dashboard.
- CAST to BIT is applied on all Boolean flag columns (is_recoverable_asset, is_mc_dead_stock, is_mc_expired, is_mc_material, is_below_reorder, is_stockout_risk) for type safety.
- lot_exiry_date and last_consumed_date are preserved in the fact view to support Q3 and Q4 analysis.

Central Fact View: fact_sales_impact

The fact_sales_impact view is the second fact view introduced as part of the Galaxy Schema update. It elevates the Q7 Silver view (vw_silver_impacted_sales_orders) into a proper Gold Layer fact view that participates in the share dimension structure.

Figure 5.3.4.7 shows the SQL Scripts that identify the delivery days either positive or negative. A positive delivery_risk_days value indicates the order is at risk of missing its delivery date because stock will run out before the delivery is due. A negative value indicates there is enough stock to cover the delivery cost.

```

CREATE OR ALTER VIEW fact_sales_impact AS
SELECT
    -- Order identifiers
    iso.sales_order_id,
    iso.customer_name,
    iso.material_id COLLATE Latin1_General_100_BIN2_UTF8 AS material_id,
    CAST(iso.order_date AS DATE) AS order_date,
    CAST(iso.required_delivery_date AS DATE) AS required_delivery_date,

    -- Order details
    iso.finished_product,
    iso.quantity_ordered,
    iso.status,

    -- Material risk context (denormalised from M3 for reporting convenience)
    iso.material_name,
    iso.category,
    iso.quantity_on_hand,
    iso.reorder_level,
    iso.avg_daily_consumption,
    iso.stock_cover_days,
    iso.lead_time_days,
    iso.is_stockout_risk,

    -- Derived urgency metric: days until delivery minus stock cover days
    -- Positive = order at risk of missing delivery
    -- Negative = stock sufficient to cover delivery
    CAST(
        DATEDIFF(DAY, CAST('2026-03-31' AS DATE), CAST(iso.required_delivery_date AS DATE))
        - iso.stock_cover_days
    AS FLOAT) AS delivery_risk_days

FROM vw_silver_impacted_sales_orders iso;
GO

```

Figure 5.3.4.7: Central Fact View (fact_sales_impact) SQL Scripts

5.3.5 Power BI Data Model and Dashboard Measures

This part documents the main data model settings, DAX measures and calculated columns implemented within Power BI Desktop to support PPG inventory analysis dashboard.

5.3.5.1 Data Model Relationships and Filter Context Issue

The Gold Layer views were imported in order to establish the data model, during development, some problems were encountered and solved. Columns like quantity_on_hand, consumption_12m, lead_time_days and stock_cover_days were returning back inflated repeated values across all rows in table visuals. Further analysis confirmed that standard model relationships between dim_material and the analytical views were not able to use propagating filter context correctly which has caused every row to show the grand total instead of unique material values.

The solution was derived after a testing a standalone table visual only columns from fact_inventory_status which showed correct amount of material values. The problem was

isolated to cross-table visuals that mixed dimension columns from dim_material with measure columns from the analytical views. Ambiguous or inactive relationship paths created multiple views sharing the same material_id key field was found as the root cause.

The solution also was implemented by substituting all raw column references with TREATAS-based DAX measures that explicitly tells Power BI to treat the filtered values of dim_material[material_id] as if they were filter values on the target view's material_id column, bypassing the broken relationship path entirely.

The following measures were created by applying the same pattern:

- Consumption 12M Fixed - SUM of vw_silver_ra_mc_logic[consumption_12m]
- Excess Quantity Fixed - SUM of vw_silver_ra_mc_logic[excess_quantity]
- Lead Time Fixed - SUM of vw_silver_stockout_risk[lead_time_days]
- Stock Cover Fixed - SUM of vw_silver_stockout_risk[stock_cover_days]

5.3.5.2 DAX Measures: TREATAS Pattern

All numeric measures that cross from dim_material into an analytical view follow this pattern:

```
Quantity On Hand Fixed =  
CALCULATE(  
    SUM(fact_inventory_status[quantity_on_hand]),  
    TREATAS(  
        VALUES(dim_material[material_id]),  
        fact_inventory_status[material_id]  
    )  
)
```

The same pattern was applied to produce the following measures used across the dashboard:

- Consumption 12M Fixed - SUM of vw_silver_ra_mc_logic[consumption_12m]
- Excess Quantity Fixed - SUM of vw_silver_ra_mc_logic[excess_quantity]
- Lead Time Fixed - SUM of vw_silver_stockout_risk[lead_time_days]
- Stock Cover Fixed - SUM of vw_silver_stockout_risk[stock_cover_days]

5.3.5.3 DAX Measures: Dead Stock and Expired Materials

For dead stock and expired materials in answering Q3 and Q4, measures that combine across filtered subsets of vw_silver_ra_mc_logic use the FILTER iterator pattern instead of TREATAS because these measures are demonstrated in standalone KPI cards without row context from dim_material:

Dead Stock Count =

```
COUNTROWS(  
    FILTER(  
        vw_silver_ra_mc_logic,  
        vw_silver_ra_mc_logic[is_mc_dead_stock] = 1  
    )  
)
```

Dead Stock Provision =

```
SUMX(  
    FILTER(  
        vw_silver_ra_mc_logic,  
        vw_silver_ra_mc_logic[is_mc_dead_stock] = 1  
    ),  
    vw_silver_ra_mc_logic[mc_provision_amount]  
)
```

Similar measures were created for expired materials but by setting is_mc_expired = 1 to be the filtering condition.

5.3.5.4 DAX Measures: Operational Risk

Q6 and Q7 are supported by the following measures:

Stockout Risk Count =

```
COUNTROWS(  
  FILTER(  
    vw_silver_stockout_risk,  
    vw_silver_stockout_risk[is_stockout_risk] = 1  
  )  
)
```

Customers Affected =

```
COUNTROWS(  
  DISTINCT(  
    SELECTCOLUMNS(  
      FILTER(  
        vw_silver_impacted_sales,  
        vw_silver_impacted_sales[is_stockout_risk] = 1  
      ),  
      "customer", vw_silver_impacted_sales[customer_name]  
    )  
  )  
)
```

5.3.5.5 Calculated Column: Risk Status

Risk Status is a calculated column created on the vw_silver_stockout_risk table to categorise each risky material for viewing in Q6 table visual:

Risk Status = IF(vw_silver_stockout_risk[stock_cover_days] = 0, “Critical”, “At Risk”)

This column operates row by row on the table, sorting each material based on its stock cover days value, hence why is it created as a calculated column instead of a measure.

5.3.6 Master Pipeline Orchestration

To orchestrate the complete end-to-end execution of the data pipeline across all three Medallion layers, a master pipeline named `pl_master_ppg` was constructed in ADF Studio. This pipeline coordinates the sequential execution of Bronze ingestion, Silver transformation, and Gold loading through four activities connected via **success** dependency arrows, ensuring that each stage begins execution only after its required predecessor(s) have completed without error.

The pipeline begins with an Execute Pipeline activity named `Run_Bronze`, which invokes `pl_bronze_ingestion`, triggering the six parallel Copy Data activities and the `df_bronze_quality` Data Flow described in Section 5.3.1. Upon successful completion of `Run_Bronze`, two Script activities which are `Script_M2_Views` and `Script_M3_Views` were executed in parallel. Both activities run direct SQL statements against the Synapse Serverless SQL Pool via the `ls_synapse_ppg` linked service, applying the Silver-layer transformation logic developed by Member 2 (RA/MC business rules) and Member 3 (stockout and sales order impact logic) respectively. Parallel execution was used here because the two Silver transformations operate on independent logic paths and have no dependency on each other, allowing both to run concurrently once Bronze validated data is available.

The final activity, a Script activity named `Script_M4_Views`, executes only after both `Script_M2_Views` and `Script_M3_Views` have been completed successfully. This activity contains seven separate query blocks, each responsible for creating and loading one Gold layer dimension or fact table including `dim_material`, `dim_supplier`, `dim_customer`, `dim_date`, `dim_warehouse`, `fact_inventory_status`, and `fact_sales_impact`. The complete master pipeline canvas, showing all four activities with Succeeded status, is shown in Figure 5.3.6.1.

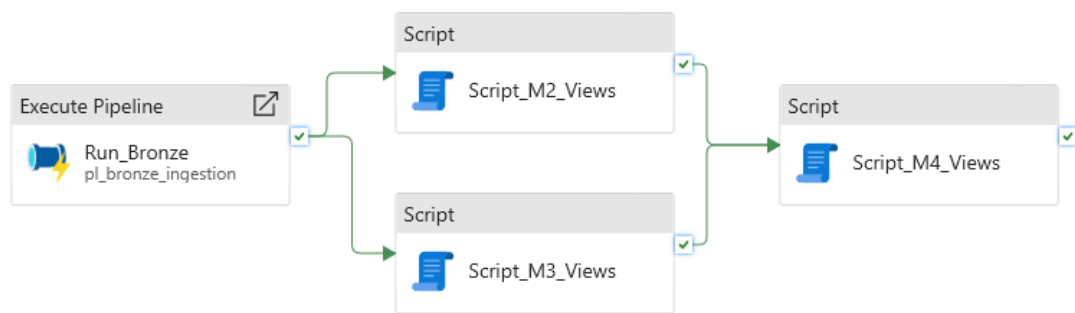


Figure 5.3.6.1 Master Pipeline of PPG RA and IRM System

This design centralises end-to-end pipeline execution behind a single trigger, fulfilling the automation requirement for the Pipeline Design & Architecture criterion of the project rubric. Running Script_M2_Views and Script_M3_Views in parallel reduces overall pipeline runtime compared to sequential execution, while the success dependency into Script_M4_Views ensures the Gold layer is only populated once both sets of Silver-layer SQL objects exist.

5.4 Essential Interfaces

This section presents the visual interfaces and output logs used to verify data flow. It demonstrates the progression from raw ingestion to structured data warehouse views and final reporting. It includes execution logs from ADF, intermediate database query tables, and the interactive dashboard for monitoring inventory risk and financial write-downs.

5.4.1 Bronze Pipeline Results

This section presents the essential interfaces and execution results produced by the Bronze Layer pipeline. The outputs demonstrated here confirm that the ingestion, data quality, and duplicate detection processes were executed successfully within Azure Data Factory.

5.4.1.1 Pipeline Execution Monitor

Upon triggering pl_bronze_ingestion via the Trigger Now function in ADF Studio, all pipeline activities were monitored through the ADF Monitor tab. The monitor view provides a real-time status update for each activity, displaying the execution state, duration, and result for both the six parallel Copy Data activities and the subsequent

df_bronze_quality Data Flow activity. A successful run is indicated when all activities display the Succeeded status, as shown in Figure 5.4.1.1.1.

Activity runs

Pipeline run ID a7a800da-5e6b-4e30-a4d0-98dbc276e809

All status ▾

Showing 1 - 7 items

Activity name ↑↓	Activity st... ↑↓	Activit... ↑↓	Run start ↑↓	Duration ↑↓	Integration runtime ↑↓
DataFlow_QC	✔ Succeeded	Data flow	6/6/2026, 5:29:16 PM	4m 41s	AutoResolveIntegrationRuntime (Southeast Asia)
Copy_Production	✔ Succeeded	Copy data	6/6/2026, 5:28:55 PM	20s	AutoResolveIntegrationRuntime (Southeast Asia)
Copy_Sales	✔ Succeeded	Copy data	6/6/2026, 5:28:55 PM	18s	AutoResolveIntegrationRuntime (Southeast Asia)
Copy_materials	✔ Succeeded	Copy data	6/6/2026, 5:28:55 PM	18s	AutoResolveIntegrationRuntime (Southeast Asia)
Copy_Purchase	✔ Succeeded	Copy data	6/6/2026, 5:28:55 PM	20s	AutoResolveIntegrationRuntime (Southeast Asia)
Copy_Suppliers	✔ Succeeded	Copy data	6/6/2026, 5:28:55 PM	17s	AutoResolveIntegrationRuntime (Southeast Asia)
Copy_inventory	✔ Succeeded	Copy data	6/6/2026, 5:28:55 PM	18s	AutoResolveIntegrationRuntime (Southeast Asia)

Figure 5.4.1.1.1: Execution Result of the Pipeline

5.4.1.2 Data Flow Execution

Beyond the pipeline-level monitor, the ADF Monitor tab also provides activity-level execution details for the Data Flow. Clicking into the df_bronze_quality activity reveals a breakdown of each transformation stage, including the number of rows read from each source, the number of rows passed to GoodRows, and the number of rows routed to BadRows for each of the six entities. This row-level traceability provides direct evidence that the quality logic executed as intended, as shown in Figure 5.4.1.2.1.

Sink	Status	Processing time ↑↓	Highest processing time ↑↓	Rows written ↑↓
SinkDupPurchase	✔ Succeeded	5s	1s 387ms	0
SinkBadProduction	✔ Succeeded	3s 383ms	1s 151ms	0
SinkGoodProduction	✔ Succeeded	4s 63ms	1s 151ms	336
SinkDupInventory	✔ Succeeded	3s 165ms	638ms	0
SinkBadPurchase	✔ Succeeded	5s	941ms	0
SinkGoodPurchase	✔ Succeeded	3s 56ms	941ms	40
SinkBadSuppliers	✔ Succeeded	3s 803ms	717ms	0
SinkGoodSuppliers	✔ Succeeded	3s 101ms	717ms	10
SinkDupSuppliers	✔ Succeeded	3s 918ms	835ms	0
SinkBadMaterials	✔ Succeeded	6s	3s 995ms	0
SinkGoodMaterials	✔ Succeeded	2s 758ms	3s 995ms	20
SinkBadSales	✔ Succeeded	4s 30ms	685ms	0
SinkGoodSales	✔ Succeeded	2s 746ms	685ms	48
SinkDupProduction	✔ Succeeded	3s 942ms	652ms	84
SinkBadInventory	✔ Succeeded	3s 692ms	631ms	0
SinkGoodInventory	✔ Succeeded	2s 807ms	631ms	240
SinkDupMaterials	✔ Succeeded	2s 824ms	563ms	0
SinkDupSales	✔ Succeeded	2s 801ms	530ms	0

Figure 5.4.1.2.1: Data Flow Execution Details

5.4.2 Silver RA/MC Query Results

Four test queries were then sent to the vw_silver_ra_mc_logic view in Azure Synapse Studio to verify the results of the silver layer RA and MC logic.

```
191 -- Q2: Recoverable Assets
192 SELECT material_id, material_name, category, quantity_on_hand, consumption_12m, excess_quantity, is_recoverable_asset
193 FROM vw_silver_ra_mc_logic WHERE is_recoverable_asset = 1;
194
```

material_id	material_name	category	quantity_on_h...	consumption_1...	excess_quantity	is_recoverable...
MAT016	Metal Can 1L	Packaging	19410	9140	10270	1
MAT019	Cardboard Box	Packaging	11300	0	11300	1
MAT020	Plastic Pail 5L	Packaging	4760	3100	1660	1
MAT014	Acetone	Solvent	5200	0	5200	1
MAT005	Chrome Yellow	Pigment	120	0	120	1
MAT009	Polyurethane Resin	Resin	210	0	210	1
MAT012	Toluene	Solvent	6450	1030	5420	1
MAT010	Vinyl Resin	Resin	2220	0	2220	1
MAT015	Butanol	Solvent	1700	900	800	1
MAT017	Metal Can 4L	Packaging	10500	4780	5720	1

Figure 5.4.2.1: Recoverable Assets query result

The query returned 10 materials classified as Recoverable Assets as shown in Figure 5.4.2.1. The quantities on hand for these materials were greater than their 12-month usage. The most prominent excess quantities were found in MAT016 Metal Can 1L with 10270 and MAT019 Cardboard Box with 11300 and MAT017 Metal Can 4L with 5720. Materials that were on hand for the 12 months but had no consumption in any production activity during the previous 12 months include MAT019, MAT014, MAT005, MAT009 and MAT010.

```
183 -- Q3: Magna Carta Dead Stock
184 SELECT material_id, material_name, category, quantity_on_hand, last_consumed_date, months_since_consumed, is_mc_dead_stock
185 FROM vw_silver_ra_mc_logic WHERE is_mc_dead_stock = 1;
```

material_id	material_name	category	quantity_on_hand	last_consumed_date	months_since_consumed	is_mc_dead_stock
MAT010	Vinyl Resin	Resin	2800	(NULL)	999	1

Figure 5.4.2.2: MC Dead Stock query result

As shown in Figure 5.4.2.2, the query returned 1 material, which is classified as MC Dead Stock. The material MAT010 belongs to the Resin category, and it had 2800 units on hand

with a last_consumed_date of NULL and months_since_consumed of 999 that represent it was never consumed in any production order during the entire dataset. It also did not have a lot expiry date recorded, so it met the requirement for being considered as part of the MC dead stock category.

```

183  -- Q4: Magna Carta Expired Stock
184  SELECT material_id, material_name, category, quantity_on_hand, lot_expiry_date, is_mc_expired
185  FROM vw_silver_ra_mc_logic WHERE is_mc_expired = 1;
186

```

material_id	material_name	category	quantity_on_h...	lot_expiry_date	is_mc_expired
MAT014	Acetone	Solvent	5200	2025-03-15	1
MAT009	Polyurethane Resin	Resin	210	2025-05-15	1

Figure 5.4.2.3: MC Expired query result

The query resulted in a return of two items with the classification MC Expired as shown in Figure 5.4.2.3. The current system date was before these materials had a lot expiry date. The materials that have expired are MAT014 Acetone with expiry date 2025-03-15 and MAT009 Polyurethane Resin with expiry date 2025-05-15. All two materials had is_mc_expired = 1 confirming the flag was correctly applied.

```

179  -- Q5: Total Financial Provision
180  SELECT SUM(mc_provision_amount) AS grand_total_mc_provision_USD
181  FROM vw_silver_ra_mc_logic WHERE is_mc_material = 1;
182

```

grand_total_mc_provision_USD
30144

Figure 5.4.2.4: Total MC provision query result

As shown in Figure 5.4.2.4, the query returned a total amount of provisioning for MC, which is USD 30144. A 100% financial write-down value for all 6 materials categorized in MC Dead Stock or MC Expired is the total of the quantity on hand of each material multiplied by the unit cost of each material. This important financial will be presented on the Power BI dashboard for management reporting.

5.4.3 Silver Stockout Query Results

After uploading the updated mock datasets into the pipeline, the query results were generated in Azure Synapse Studio. The new data added to the inventory snapshot records, production order records, sales order records, and purchase order records. The main differences for the stockout and sales impact component were the inclusion of inventory snapshots in January, February and March 2026, the inclusion of additional production orders and the inclusion of additional sales orders.

The first output was created from the `vw_silver_stockout_risk` view. This perspective was adopted for the answers to Q1 and Q6. The latest inventory snapshot was validated prior to verifying the final result. The query result was that for all 20 materials the most recent snapshot date was 31 March 2026. This proved that the newly added inventory records have been captured by the Silver Layer logic.

The final quick check identified 5 materials as being below reorder level, and 2 materials as being stockout risk. The outcome was lower than the previous production due to the newly added March 2026 inventory records, which showed lower stocks. Materials that were previously at stockout or reorder level and were considered risky were no longer considered risky when the most recent stock data was added.

View

Table

Chart

Export results

Search

material_id	material_name	category	quantity_on_hand	reorder_level	is_below_reorder
MAT018	Plastic Drum 20L	Packaging	160	500	1
MAT004	Phthalocyanine Blue	Pigment	0	150	1
MAT006	Alkyd Resin	Resin	740	800	1
MAT008	Epoxy Resin	Resin	0	400	1
MAT009	Polyurethane Resin	Resin	210	300	1

Figure 5.4.3.1: Query result for materials below reorder level

The query result for materials below reorder level is displayed in figure 5.4.3.1. The output reveals that there are five materials with less quantity than reorder level. These materials require attention due to a stock level below minimum.

Results

Messages

View

Table

Chart

Export results

Search

material_id	material_name	category	quantity_on_hand	avg_daily_consumption	stock_cover_days	lead_time_days	is_stockout_risk
MAT004	Phthalocyanine Blue	Pigment	0	7.071428571428571	0	21	1
MAT008	Epoxy Resin	Resin	0	24.651162790697676	0	21	1

Figure 5.4.3.2: Query result for materials at stockout risk

The query result for materials at stockout risk is displayed in figure 5.4.3.2. The result indicates that two materials Phthalocyanine Blue and Epoxy Resin were at risk of being out of stock. There was no quantity available for both materials, which resulted in stock cover days less than lead time days.

The second output was created from the vw_silver_impacted_sales_orders view. The views expressed were used in answering Q7. This revealed 13 affected sales order-material documents were found. These records included 8 affected customers. This validated that the system could be able to establish the association between material shortage risk and customer order fulfilment with the updated mock dataset.

Results

Messages

View

Table

Chart

↗

Export results

🔍

Search

sales_order_id	customer_name	finished_product	material_id	material_name	required_deliv...	quantity_order...	stock_cover_d...	lead_ti
SO-065	BuildRight Hardware	Anti-Rust Coating	MAT008	Epoxy Resin	2026-04-10	333	0	21
SO-066	Gamuda Trading	Marine Blue Coating	MAT008	Epoxy Resin	2026-04-15	410	0	21
SO-066	Gamuda Trading	Marine Blue Coating	MAT004	Phthalocyanine Blue	2026-04-15	410	0	21
SO-064	IJM Corporation	Epoxy Floor Coating	MAT008	Epoxy Resin	2026-04-07	290	0	21
SO-045	Maybank Facilities Mgmt	Marine Blue Coating	MAT008	Epoxy Resin	2026-01-24	275	0	21
SO-045	Maybank Facilities Mgmt	Marine Blue Coating	MAT004	Phthalocyanine Blue	2026-01-24	275	0	21
SO-067	Maybank Facilities Mgmt	Epoxy Floor Coating	MAT008	Epoxy Resin	2026-04-18	275	0	21
SO-061	Nippon Construction Sdn Bhd	Marine Blue Coating	MAT008	Epoxy Resin	2026-03-28	480	0	21
SO-061	Nippon Construction Sdn Bhd	Marine Blue Coating	MAT004	Phthalocyanine Blue	2026-03-28	480	0	21
SO-063	Sime Darby Property	Marine Blue Coating	MAT008	Epoxy Resin	2026-04-03	355	0	21
SO-063	Sime Darby Property	Marine Blue Coating	MAT004	Phthalocyanine Blue	2026-04-03	355	0	21
SO-062	Top Glove Facilities	Epoxy Floor Coating	MAT008	Epoxy Resin	2026-04-01	520	0	21
SO-047	YTL Hardware Centre	Epoxy Floor Coating	MAT008	Epoxy Resin	2025-12-31	385	0	21

Figure 5.4.3.3: Query Result for Impacted Sales Orders

Figure 5.4.3.3 shows the impacted sales order result. The output lists open customer sales orders that may be affected by stockout-risk materials. The result shows 13 impacted sales order-material records.

Results Messages		
View	Table Chart	Export results
Search		
customer_name	impacted_order_count	at_risk_material_count
Maybank Facilities Mgmt	2	2
Nippon Construction Sdn Bhd	1	2
Gamuda Trading	1	2
BuildRight Hardware	1	1
YTL Hardware Centre	1	1
Top Glove Facilities	1	1
Sime Darby Property	1	2
IJM Corporation	1	1

Figure 5.4.3.4 Query Result for Affected Customer Summary

Figure 5.4.3.4 shows the affected customer summary result. The output summarizes the number of impacted orders and at-risk materials for each customer. The result confirms that 8 customers were affected by stockout-risk materials.

5.4.4 Gold Layer Tables

The Gold Layer exposes its data through a set of SQL views that serve as the primary interface between the data pipeline and the Power BI dashboard. All views reside in the ppg_gold database on the Azure Synapse Analytics SQL endpoint. Table 5.4.4.1 show the summary of the Gold Layer data and Table 5.4.4.2 and Table 5.4.4.3 shows the fact table column references.

Table 5.4.4.1 Gold Layer View Summary

View	Columns	Row Count
dim_material	material_id, material_name, category, unit_of_measure, reorder_level, lead_time_days, unit_cost	20
dim_supplier	supplier_id, supplier_name, country, realibility_rating	10
dim_customer	customer_name	15
dim_warehouse	warehouse_location	5

dim_date	date_id, year, quarter, month, month_name, week_of_year, day_of_week, day_name	19
fact_inventory_status	All RA, MC, and stockout flags	20
fact_sales_impact	All RA, MC and sales impact	13

Table 5.4.4.2 fact_inventory_status Column References

Column Name	Data Type	Description
material_id	VARCHAR	Primary key as unique material identifier
snapshot_date	DATE	Date of inventory snapshot
warehouse_location	VARCHAR	Storage loaction of material
quantity_on_hand	INT	Current stock quantity
consumption_12m	FLOAT	Total quantity consumed in past 12 months
excess_quantity	FLOAT	Quantity above 12-month consumption
is_recoverable_asset	BIT	1 = excess stock qualifies as recoverable asset
is_mc_dead_stock	BIT	1 = no consumption for 9+ months, no expiry
is_mc_expired	BIT	1 = lot expiry date has passed
is_mc_material	BIT	1 = material qualifies for MC financial provision
mc_provision_amount	FLOAT	Financial write-down value (quantity x unit cost)
months_since_consumed	FLOAT	Months elapsed since last consumption (999 = never)
lot_expiry_date	DATE	Lot expiry date (NULL if no expiry)
last_consumed_date	DATE	Most recent production consumption date
is_below_reorder	BIT	1 = quantity on hand is below reorder level
is_stockout_risk	BIT	1 = stock cover days less than lead time days
stock_cover_days	FLOAT	Estimated days of stock remaining based on consumption rate
avg_daily_consumption	FLOAT	Average units consumed per day
total_consumed	FLOAT	Total units consumed across all production history
reorder_level	INT	Minimum stock threshold before reorder
lead_time_days	INT	Supplier lead time in days

Table 5.4.4.3 fact_sales_impact Column References

Column Name	Data Type	Description
sales_order_id	VARCHAR	Sales order identifier
customer_name	VARCHAR	Customer name (FK to dim_customer)
material_id	VARCHAR	At-risk material (FK to dim_material)
order_date	DATE	Sales order date (FK to dim_date)
required_delivery_date	DATE	Required delivery date (FK to dim_date)
finished_product	VARCHAR	Finished product affected by the at-risk material
quantity_ordered	INT	Ordered quantity
status	VARCHAR	Order status (OPEN/PENDING/BACKORDER)
material_name	VARCHAR	At-risk material name
category	VARCHAR	Material category
quantity_on_hand	INT	Current stock of at-risk material
reorder_level	INT	Reorder threshold for the material
avg_daily_consumption	FLOAT	Average daily consumption rate
stock_cover_days	FLOAT	Days of stock remaining
lead_time_days	INT	Supplier lead time in days
is_stockout_risk	BIT	1 = material confirmed at stockout risk
delivery_risk_days	FLOAT	Days until delivery minus stock cover days; positive = at risk

5.4.5 Power BI Dashboard

The dashboard was built in Power BI Desktop that has been connected to ppg_gold database on Azure Synapse Analytics via Import connection mode, organised into two pages in order to address different category of inventory risk.

5.4.5.1 Page 1: Inventory Health (Q1 to Q5)

The Inventory Health page answers Q1 – Q5 with four quadrants.

A clustered column chart is used for Q1. On the axis of this chart, material_name is used, and Quantity On Hand Fixed and Reorder Level Fixed are the two value series. This chart is also filtered by rows where is_below_reorder equals 1. In addition, a dashed reference line is placed at zero to provide an anchoring point for visual comparison. There were five materials identified as being below their respective reorder levels. These include Alkyd

Resin, Polyurethane Resin, Plastic Drum 20L, Epoxy Resin and Phthalocyanine Blue. These five materials are all visible in the chart as being "At risk."

A table visual is used for Q2. The table includes the following variables: material_name, Quantity On Hand Fixed, Consumption 12M Fixed, and Excess Quantity Fixed. The table is filtered by rows where is_recoverable_asset equals 1, and the table is sorted in descending order based upon excess quantity. This table identifies 11 materials that are classified as Recoverable Assets, and these 11 materials represent a combined total excess quantity of 42,920 units.

Two KPI cards are used for Q3 to display Dead Stock Count and Dead Stock Provision. The Dead Stock Count KPI card displays 1 material, while the Dead Stock Provision KPI card displays \$16.87K. A supporting table directly below the KPI cards provides a detailed row for Vinyl Resin. The row contains information about the category of Vinyl Resin, its quantity on hand, and its provision amount.

A donut chart is used for Q4. Material_name is used as the legend, and mc_provision_amount is used as the value. The donut chart is filtered by rows where is_mc_expired equals 1. The donut chart illustrates the proportion of provision amount contributed by each of the two expired materials. Acetone contributes the greatest percentage (82.28%) of the provision amount (\$10.92K) of the two expired materials. Polyurethane Resin contributes the second greatest percentage (17.72%) of the provision amount (\$2.35K) of the two expired materials. Two additional KPI cards are located next to the donut chart and illustrate the expired material count and total expired provision amount.

A single KPI card is used for Q5. This KPI card illustrates the total MC provision amount (\$30.14K) which represents the combined dead stock and expired material provisions.

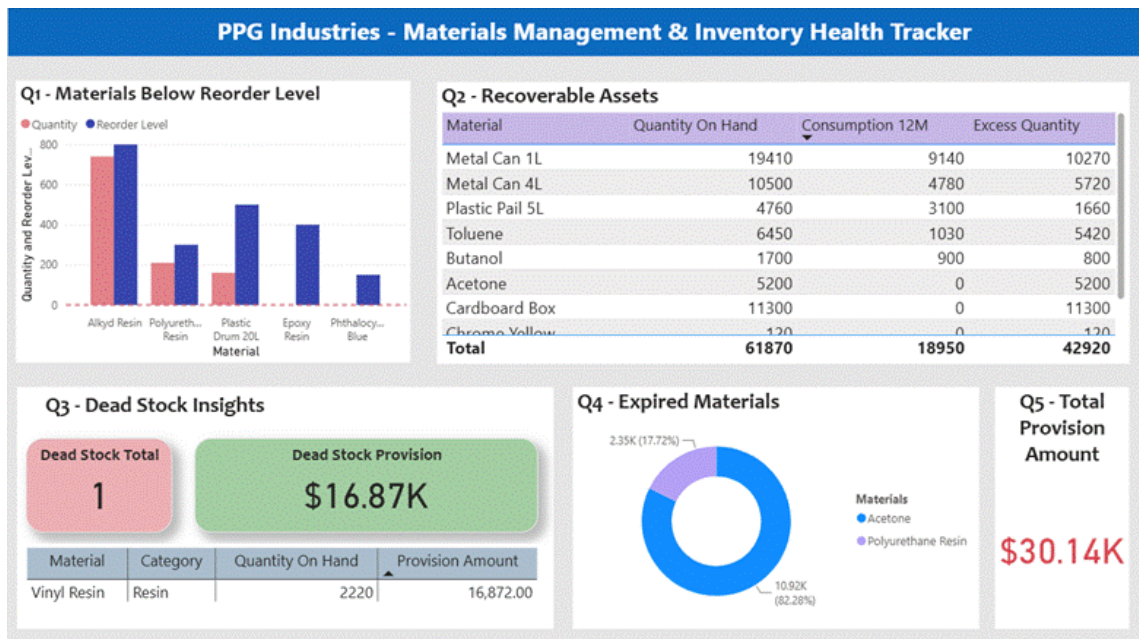


Figure 5.4.5.1: Power BI Dashboard, Inventory Health Page (Q1 to Q5)

5.4.5.2 Page 2: Operational Risk (Q6 and Q7)

Q6 and Q7 are addressed in the Operational Risk Page. Two materials in Stockout Risk, Eight Affected Customers and 13 Orders at Risk are displayed using three KPI cards at the top of the page.

To create a table visual for Q6, material_name, Lead Time Fixed, Stock Cover Fixed, Quantity On Hand Fixed measures and the calculated column Risk Status were used, and the rows were filtered for is_stockout_risk = 1. The Quantity On Hand Fixed column was conditionally formatted to show red background cells for stock levels that are very low. The Risk Status column states that both materials are Critical because both Epoxy Resin and Phthalocyanine Blue are 0 stock cover days with a supplier lead time of 21 days.

For Q7, a matrix visual was created with customer_name as rows, material_name as columns and order_count as values. The cells in the matrix were then conditional formatted with a gradient colour scale from white to red to produce a "heatmap" effect, making it immediately apparent which customers are exposed to the most orders. Maybank Facilities Mgmt is the most affected customer with 3 orders for both at-risk materials. The matrix lists 13 orders, which are affected by 8 customers.

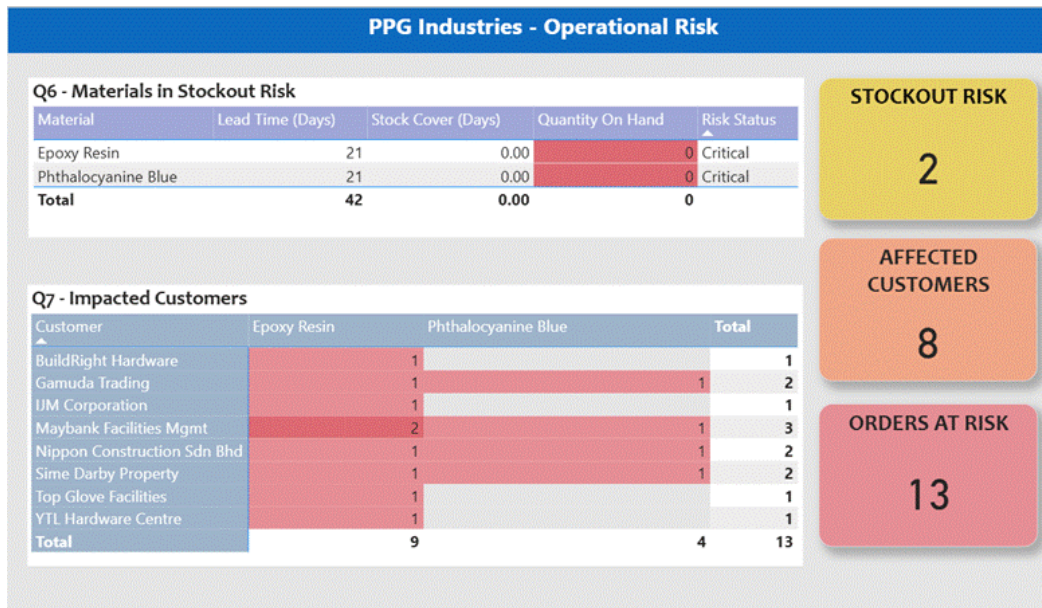


Figure 5.4.5.2: Power BI Dashboard, Operational Risk Page (Q6 and Q7)

5.4.6 Master Pipeline Execution Results

Upon triggering pl_master_ppg via Trigger Now, the ADF Monitor tab displayed the execution status of all four activities: Run_Bronze, Script_M2_VIEWS, Script_M3_VIEWS, and Script_M4_VIEWS. A successful end-to-end run is confirmed when all four activities display the Succeeded status, with Script_M2_VIEWS and Script_M3_VIEWS beginning only after Run_Bronze completes, executing concurrently, and Script_M4_VIEWS beginning only after both have finished. This sequence is shown in Figure 5.4.6.1.

Activity runs

Pipeline run ID 62ace2ae-4446-4428-947c-c3215ee59f5b

All status ▾

Showing 1 - 4 items

Activity name ↑↓	Activity st... ↑↓	Activit... ↑↓	Run start ↑↓	Duration ↑↓	Integration runtime ↑↓
Script_M4_VIEWS	✓ Succeeded	Script	6/14/2026, 11:45:29 AM	9s	AutoResolveIntegrationRuntime (Southeast Asia)
Script_M2_VIEWS	✓ Succeeded	Script	6/14/2026, 11:45:17 AM	10s	AutoResolveIntegrationRuntime (Southeast Asia)
Script_M3_VIEWS	✓ Succeeded	Script	6/14/2026, 11:45:17 AM	11s	AutoResolveIntegrationRuntime (Southeast Asia)
Run_Bronze	✓ Succeeded	Execute Pipelin	6/14/2026, 11:40:01 AM	5m 16s	

Figure 5.4.6.1: Master Pipeline Execution Result

5.5 Testing

To ensure the pipeline is both accurate and reliable, the entire system is subjected to evaluation process. This section documents the system testing strategy, which examines the

platform from multiple technical perspectives to verify overall stability. The evaluation framework is divided into three main stages, black box testing to validate that each layer outputs the correct operational results, white box testing to verify the internal logic of the underlying code and schemas, and user acceptance testing to confirm that stakeholders can easily interpret the final dashboard views. Together, these procedures provide objective proof that the data pipeline performs exactly as required.

5.5.1 Black Box Testing

This section evaluates the functional behavior of the system by focusing purely on inputs and outputs without examining the internal code structure. The primary objective of black box testing is to verify that the end-to-end data pipeline correctly handles file ingestion, applies data quality checks, and executes business logic accurately.

5.5.1.1 Bronze Ingestion and Quality Check Output Testing

Black box testing conducted in Bronze layer was to verify that given a set of known input CSV files, the pipeline produces the correct and expected output directories within ppgdatalake storage. The tester interacts only with the pipeline trigger and the resulting storage outputs, treating the internal ADF pipeline and Data Flow logic as opaque.

Table 5.5.1.1 presents the black box test cases designed and executed for the Bronze ingestion and quality check components.

Table 5.5.1.1: Black Box Test Cases for Bronze Ingestion and Quality Check

Test Case	Expected Output	Actual Output	Status
All six CSV files are successfully ingested from raw to ingested directory	Six corresponding files appear in bronze/ingested/	Six files found in bronze/ingested/	Pass
Clean rows are routed to the validated directory	Files appear in bronze/validated/<entity>/ for all six entities	All six validated subfolders populated with output files	Pass

Null rows are routed to the quarantine directory	Affected rows appear in bronze/quarantine/<entity>/	Quarantine folder exists and check executed, no nulls found in synthetic data	Pass
Duplicate rows are detected and routed to quarantine	Duplicate rows appear in bronze/quarantine/duplicates/<entity>/	Duplicates folder exists and check executed, no duplicates found in synthetic data	Pass

The results of the black box tests confirm that the Bronze Layer pipeline behaves correctly from an end-user perspective. All six source files were successfully ingested into bronze/ingested/, and the quality check routing logic correctly directed clean rows to bronze/validated/ and flagged rows to bronze/quarantine/. The empty quarantine and duplicates directories for the synthetic dataset are an expected and valid outcome, as the source data was deliberately designed to be free of nulls and duplicates. Their existence nonetheless confirms that the detection logic was triggered and evaluated every row.

5.5.1.2 RA and MC Logic Output Testing

To confirm that the RA and MC provision component generated the anticipated results based on the specified business needs, black box testing was carried out. Without looking at the core SQL statements, the testing concentrated on the end outcomes produced by the SQL view. Q2, Q3, Q4, and Q5 were validated using the result of vw_silver_ra_mc_logic.

For Q2, the expected output was number of materials where their current quantity on hand exceeded their total 12-month consumption. The test result showed that 10 materials were flagged as Recoverable Assets. For Q3, the expected output was number of materials that had not been consumed for 9 or more months and had no lot expiry date recorded. The test result showed only 1 material was identified as MC Dead Stock. For Q4, the expected output was a list of materials whose expiry date had passed relative to the reference date. The test result showed that 2 materials were identified as MC Expired. For Q5, the expected output was the total financial provision amount calculated as a 100% write-down across all MC materials. The test result returned a total of 3 MC materials with a combined provision of USD 30144.

185	SELECT				
186	SUM(CASE WHEN is_recoverable_asset = 1 THEN 1 ELSE 0 END)	AS	Q2_recoverable_asset_count,		
187	SUM(CASE WHEN is_mc_dead_stock = 1 THEN 1 ELSE 0 END)	AS	Q3_mc_dead_stock_count,		
188	SUM(CASE WHEN is_mc_expired = 1 THEN 1 ELSE 0 END)	AS	Q4_mc_expired_count,		
189	SUM(CASE WHEN is_mc_material = 1 THEN 1 ELSE 0 END)	AS	Q5_mc_material_count,		
190	SUM(mc_provision_amount)	AS	grand_total_mc_provision_USD		
191	FROM vw_silver_ra_mc_logic;				
192					
193					

Results	Messages
---------	----------

Select	Query 5	View	Table	Chart	Export results
--------	---------	------	-------	-------	----------------

Search

Q2_recoverable_asset_count	Q3_mc_dead_stock_count	Q4_mc_expired_count	Q5_mc_material_count	grand_total_mc_provision_USD
10	1	2	3	30144

Figure 5.5.1.2.1: Final quick check count results for Q2, Q3, Q4 and Q5

Figure 5.5.1.2.1 shows the quick count check results for the RA and MC provision components. The results confirm that the expected output was successfully produced for all four business questions.

Table 5.5.1.2.1 shows the quick count check results for the RA and MC provision component. The results confirm that the expected output was successfully produced for all four business questions.

Table 5.5.1.2.1 Black Box Testing Results for RA and MC Provision Component

Test Case	Expected Output	Actual Output	Status
Q2 Recoverable Asset	Materials where stock exceeds 12-month consumption are identified	10 materials identified	Passed
Q3 MC Dead Stock	Materials with 9+ months no consumption and no expiry date are identified	1 material identified	Passed
Q4 MC Expired	Materials with passed lot expiry date are identified	2 materials identified	Passed
Q5 Total Provision	Total financial write-down amount is calculated	3 MC materials, USD 30144 returned	Passed

5.5.1.3 Stockout and Sales Impact Output Testing

Black box testing was carried out to verify whether the SQL views produced the expected outputs based on the business questions. The testing focused on the final query results without examining the internal SQL code.

For Q1, the expected output was a list of materials below reorder level. The updated query result showed that 5 materials were below reorder level. For Q6, the expected output was a list of materials at stockout risk. The updated query result showed that 2 materials were at stockout risk. For Q7, the expected output was a list of open sales orders and customers affected by stockout-risk materials. The updated query result showed 13 impacted sales order-material records involving 8 customers.

Table 5.5.1.3.1 Black Box Testing Results for Stockout and Sales Impact

Test Case	Expected Output	Actual Output	Status
Q1 Below Reorder Level	Materials below reorder level are identified	5 materials identified	Passed
Q6 Stockout Risk	Materials at stockout risk are identified	2 materials identified	Passed
Q7 Sales Order Impact	Impacted sales orders and affected customers are identified	13 records and 8 customers identified	Passed

Table 5.5.1.3.1 shows the black box testing results for the stockout and sales impact component. The test was conducted to verify whether the final query outputs answered Q1, Q6, and Q7 correctly.

5.5.1.4 Gold Layer Table Output Testing

Black box testing is used to ensure that the output of each view and fact table is accurate according to the business without inspecting the internal SQL logic. The testing approach as shown in Figure 5.5.1.4.1 by using SQL scripts treats Gold Layer objects and gives the results as in Figure 5.5.1.4.2.

```

SELECT 'dim_material' AS object_name, COUNT(*) AS record_count FROM dim_material
UNION ALL
SELECT 'dim_supplier', COUNT(*) FROM dim_supplier
UNION ALL
SELECT 'dim_customer', COUNT(*) FROM dim_customer
UNION ALL
SELECT 'dim_warehouse', COUNT(*) FROM dim_warehouse
UNION ALL
SELECT 'dim_date', COUNT(*) FROM dim_date
UNION ALL
SELECT 'fact_inventory_status', COUNT(*) FROM fact_inventory_status
UNION ALL
SELECT 'fact_sales_impact', COUNT(*) FROM fact_sales_impact;
GO

```

Figure 5.5.1.4.1: SQL Scripts of Galaxy Schema Validation

object_name	record_count
dim_material	20
dim_supplier	10
dim_customer	15
dim_warehouse	5
dim_date	19
fact_inventory_status	20
fact_sales_impact	13

Figure 5.5.1.4.2: Result of Galaxy Schema Validation

All the views created were passed showing that the black box test was a success. The Gold Layer outputs are verified to be consistent with the expected business questions answers. Table 5.5.1.4.3 shows the dimension and fact record count tests.

Table 5.5.1.4.3: Dimensions and Fact Record Count Tests

Object	Expected Rows	Actual Rows	Status
dim_material	20	20	Pass
dim_supplier	10	10	Pass
dim_customer	15	15	Pass
dim_warehouse	5	5	Pass
dim_date	19	19	Pass

fact_inventory_status	20	20	Pass
fact_sales_impact	13	13	Pass

5.5.1.5 Power BI Dashboard Output Testing

Black box testing for the data loaded from Gold Layer was conducted to ensure that the data can be presented in Power BI. The tests did not delve into the DAX logic or data model configuration in each of the dashboard pages, but rather focused on the final visualization output on each page. Instead of a clustered bar chart, the expected output was a clustered column chart that would only show materials that were not as numerous as the reorder level. A test result was shown as 5 on Q1 chart, which is also the same as the Silver Layer output. In Q2, the goal was to create a list of materials that are considered Recoverable Assets with a table that only displays materials with excess quantities. According to test result, 10 materials had the correct values for both Quantity On Hand Fixed, Consumption 12M Fixed and Excess Quantity Fixed.

For Q3, it was expected that there would be two KPI cards, Dead Stock Count and Provision Amount, and a detail table. The test result showed that there was 1 material (Vinyl Resin) available and the KPI cards could be displayed.

For Q4, the expected result was a donut graph of the amount/value of provision against the expired material as well as KPI cards. Test result: 2 materials expired, with the majority being due to Acetone, and the donut chart displayed properly.

Q5 aim was to produce a KPI card for the total MC provision (combined). The test was \$30.14K.

For Q6. only materials that have a risk level of red conditional formatting should be in the table with a Risk Status label, and should be considered for stockout risk. Two materials were identified in the test result, both Epoxy Resin (red) and Phthalocyanine Blue (red).

For Q7, expected output was a matrix of orders affected, customer versus material. Test result shows that 13 orders were affected out of 8 customers who had applied the correct colouring on the gradient.

Table 5.5.1.5: Black Box Testing Results for Power BI Dashboard Output

Test Case	Expected Output	Actual Output	Status
-----------	-----------------	---------------	--------

Q1 Below Reorder Level	Materials below reorder level shown in bar chart	5 materials displayed	Passed
Q2 Recoverable Assets	Materials where stock exceeds 12-month consumption shown in table	10 materials displayed with correct per-material values	Passed
Q3 MC Dead Stock	KPI cards show dead stock count and provision; detail table shows Vinyl Resin	1 material, \$16,872 provision displayed	Passed
Q4 MC Expired	Donut chart shows provision proportion per expired material	5 materials in donut chart; Acetone largest at 82.28%	Passed
Q5 Total MC Provision	KPI card displays combined total provision amount	\$30.14K displayed	Passed
Q6 Stockout Risk	Table shows at-risk materials with Risk Status and red formatting	2 materials displayed, both Critical with red cells	Passed
Q7 Customer Impact	Heatmap matrix shows impacted orders per customer per material	13 orders across 8 customers with gradient colouring	Passed

5.5.2 White Box Testing

Unlike the previous functional checks, this section examines the internal logic, script execution, and structural integrity of the pipeline's codebase. White box testing focuses on verifying the backend processes, including the conditional loops within pipeline, the data transformation integrity, and the calculation. This internal review ensures that the data schemas are highly optimized, scripts run without structural errors, and data transitions smoothly between the Medallion layers.

5.5.2.1 Bronze Ingestion and Quality Check Logic Testing

White box testing was conducted by inspecting the internal configuration of each transformation stage within `pl_bronze_ingestion` and `df_bronze_quality`. This includes verifying the null detection expressions within the `QualityFlags` Derived Column, the routing conditions of the Conditional Split, the grouping keys applied in the Aggregate transformation for duplicate detection, the success dependency between the Copy Data activities and the Data Flow activity, and the authentication configuration of both linked services. Table 5.5.2.1.1 presents the white box test cases designed and executed for these components.

Table 5.5.2.1.1: White Box Test Cases for Bronze Ingestion and Quality Check

Test Case	Component Tested	Expected Outcome	Actual Outcome	Status
Copy Data source dataset correctly references the raw CSV path	<code>ds_src_materials</code> dataset configuration	File path set to <code>bronze/raw/materials/materials.csv</code> with First row as header enabled	File path and header setting confirmed correct in dataset properties	Pass
Copy Data sink dataset correctly references the ingested path	<code>ds_sink_materials</code> dataset configuration	File path set to <code>bronze/ingested/materials/</code> with linked service <code>ls_adls_ppg</code>	Sink path and linked service confirmed correct	Pass
Null detection expression correctly evaluates null fields	<code>QualityFlags</code> Derived Column in <code>df_bronze_quality</code>	<code>iif(isNull(material_id), 1, 0)</code> returns 1 for null values and 0 for non-null values	Expression validated in Data Flow expression builder and correct return values confirmed	Pass
Conditional Split correctly routes BadRows	Conditional Split transformation in <code>df_bronze_quality</code>	Stream <code>BadRows</code> condition: <code>is_null_material_id == true()</code> <code>is_null_unit_cost == true()</code> routes null rows away from <code>GoodRows</code>	Routing logic verified in Conditional Split configuration	Pass

on null condition			n, correct stream assignment confirmed	
Duplicate detection Aggregate groups by correct primary key	Aggregate transformation in duplicate detection chain	materials entity groups by material_id, inventory_snapshot groups by snapshot_date + material_id	Group-by keys verified in Aggregate transformation settings for all six entities	Pass
Duplicate detection Filter retains only rows with row_count greater than 1	Filter transformation in duplicate detection chain	Expression row_count > 1 retains only genuine duplicate rows	Filter expression verified in transformation configuration	Pass
Duplicate detection chain branches from an independent Source transformation	Data Flow canvas structure	Aggregate transformation connects to a separate Source, not to the QualityFlags Derived Column	Verified on canvas that each duplicate chain has its own Source transformation independent of the quality chain	Pass
Data Flow activity is connected to Copy Data activities via a success dependency	pl_bronze_ingestion pipeline canvas	df_bronze_quality activity executes only on success of all six Copy Data activities	Success dependency arrow verified between Copy Data activities and Data Flow activity in pipeline canvas	Pass

Linked service uses System-Assigned Managed Identity for ADLS Gen2 authentication	ls_adls_ppg linked service configuration	Authentication method set to System Managed Identity targeting storage account ppgdatalake	Linked service configuration verified, Test Connection returned Succeeded	Pass
Linked service uses Azure SQL Database connector for Synapse Serverless endpoint	ls_synapse_ppg linked service configuration	Connector type is Azure SQL Database targeting synapse-ppg-ondemand.sql.azuresynapse.net	Connector type and endpoint verified in linked service properties	Pass

The white box test results confirm that all internal logic components of the Bronze Layer are correctly configured. Attention was given to two architectural decisions that required verification at the configuration level. First, the duplicate detection chains were confirmed to branch from independent Source transformations rather than from the QualityFlags Derived Column, which is necessary to prevent schema contamination between the two transformation chains. Second, the null detection expressions within the Derived Column transformation were verified to use the correct `iif(isNull(column), 1, 0)` syntax, as the `isNull()` function alone would produce a type mismatch error within the ADF Data Flow expression engine. An example of evidence for the null detection expression configuration for materials source dataset is shown in Figure 5.5.2.1.1.

Expression

<code>iif(isNull(material_id), 1, 0)</code>	123
<code>iif(isNull(unit_cost), 1, 0)</code>	123
<code>iif(isNull(reorder_level), 1, 0)</code>	123
<code>iif(isNull(category), 1, 0)</code>	123
<code>iif(isNull(lead_time_days), 1, 0)</code>	123

5.5.2.2 RA and MC Business Logic Testing

White box testing was performed to verify the internal SQL logic used in the RA and MC provision process. The testing covered five internal logic areas that includes latest inventory selection, never consumed material handling, MC Dead Stock condition logic, MC Expired date comparison logic, and financial provision calculation accuracy. A single verification query was constructed to test all five logic areas simultaneously and produce a consolidated result. Each column in the output represented one internal logic test.

For the first test, the total count of unique materials returned by the view was checked to verify that the ROW_NUMBER() window function correctly partitioned records by material_id and selected only the most recent snapshot per material. The expected result was 20, representing one row per material.

For the second test, the maximum months_since_consumed value was checked to verify that the internal CASE WHEN last_consumed_date IS NULL THEN 999.0 logic correctly assigned 999.0 to materials that had never appeared in any production order. The expected result was 999.

For the third test, the count of materials where both months_since_consumed >= 9 and lot_expiry_date IS NULL were true and is_mc_dead_stock = 1 was checked to verify that the AND condition logic inside the Q3 flag was applied correctly. The expected result was 1 which represents the dead stock count.

For the fourth test, the count of materials where lot_expiry_date was earlier than reference date and is_mc_expired = 1 was checked to verify that the internal date comparison logic of the Q4 flag was functioning correctly. The expected result was 2 which represent the expired material count.

For the fifth test, the count of MC materials where the calculated quantity_on_hand × unit_cost matched the stored mc_provision_amount exactly was checked to verify the internal provision formula was producing accurate results. The expected result was 3 which represent the total of MC materials count.

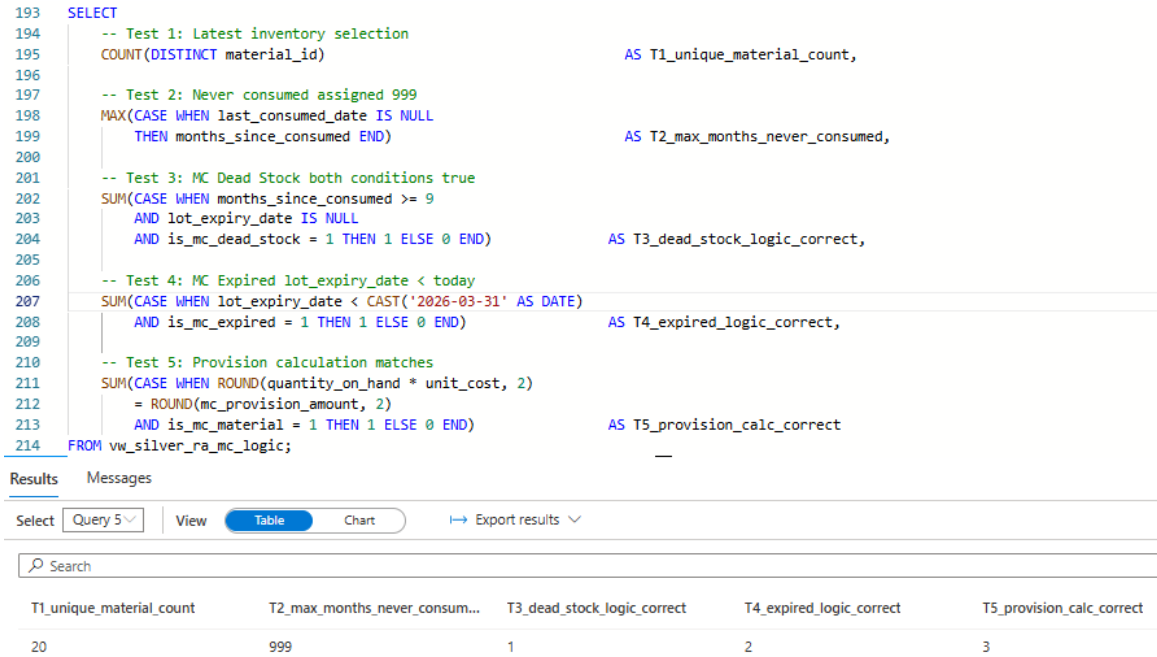


Figure 5.5.2.2.1: White Box Testing Verification Output

Figure 5.5.2.2.1 shows the white box testing output with all five internal logic areas returned the expected values. This confirmed that each condition, calculation, and logic path within the view was functioning correctly.

Table 5.5.2.2.1: White Box Testing Results for RA and MC Provision Logic

Test Area	Internal Logic Tested	Expected	Actual	Status
Latest Inventory Selection	ROW_NUMBER() partitioned by material_id returns one row per material	20	20	Passed
Never Consumed Handling	CASE WHEN last_consumed_date IS NULL assigns 999.0	999	999	Passed
MC Dead Stock Condition	AND logic requires months >= 9 AND lot_expiry_date IS NULL simultaneously	1	1	Passed
MC Expired Date Comparison	lot_expiry_date < CAST('2026-03-31' AS DATE)	2	2	Passed

	DATE) correctly identifies expired materials			
Provision Calculation	quantity_on_hand × unit_cost matches mc_provision_amount for all MC materials	3	3	Passed

Table 5.5.2.2.1 shows the white box testing results for the internal SQL logic used in the RA and MC provision component. All five internal logic areas passed and confirming that the view logic was implemented without error and producing accurate results across all business rules.

5.5.2.3 Stockout and Sales Impact Logic Testing

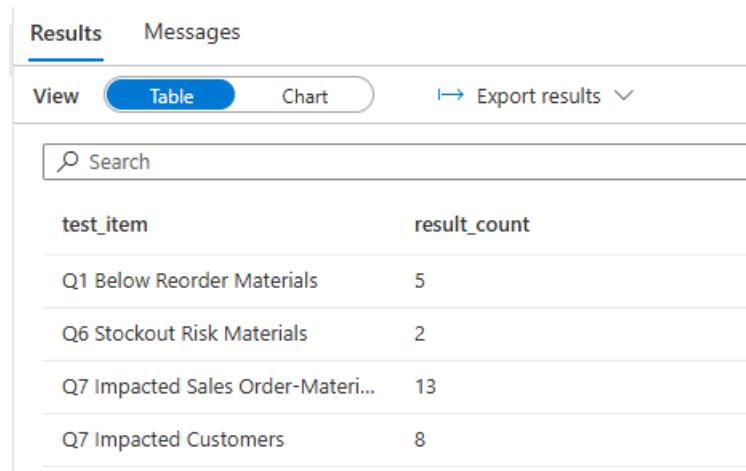
The internal SQL logic was checked during white box testing for the stockout and sales impact process. The first test was on date conversion. In previous tests, the average daily consumption was first set to fallback values of 0.001. This indicated that the production order dates were not being correctly converted. After the production dataset was checked, it was found that the production_date column used the yyyy-mm-dd format.

This SQL view was then modified to handle several different date formats with TRY_CONVERT.

The second test was for the fixed reference date logic. Once the new mock information has been added, the SQL script was tweaked to use 31 March 2026 as a reference date. This meant that the most recent inventory snapshot was chosen appropriately and the 12-months production consumption window was aligned with the new data set. The query result showed that all 20 materials were based on the inventory snapshot of 31 March 2026.

The third test was on the consumption calculation. The total consumption and average daily consumption values were correctly computed from the production order records after the date conversion and fixed reference date logic were fixed. This validated that the Stock Cover days could be determined with valid consumption values.

The fourth test was on the join logic. The materials that were identified as having a risk for going out of stock were checked to see if they could be traced back to finished materials based on production orders. The final products were then attached to open sales orders. The result was that sales orders that were affected and customers that were affected could be traced back properly.



test_item	result_count
Q1 Below Reorder Materials	5
Q6 Stockout Risk Materials	2
Q7 Impacted Sales Order-Materi...	13
Q7 Impacted Customers	8

Figure 5.5.2.3.1: Final Quick Check Results for Stockout and Sales Impact Views

Figure 5.5.2.3.1 shows the final quick check results for the stockout and sales impact views. The results confirm that the SQL views were functioning correctly after the updated mock dataset was reloaded and the fixed reference date was applied.

Table 5.5.2.3.1: White Box Testing Results for Stockout Risk and Sales Impact Logic

Test Area	Internal Logic Tested	Result	Status
Fixed Reference Date	Reference date was set to 31 March 2026 to align with the updated mock dataset	Latest inventory snapshot and 12-month consumption window were applied correctly	Passed

Table 5.5.2.3.1 shows the white box testing results for the internal SQL logic used in the stockout and sales impact component. The test focused on date conversion, fixed reference date logic, consumption calculation, stockout risk logic, and impacted sales order tracing.

5.5.2.4 Gold Layer Galaxy Schema and Loading Logic Testing

The White box testing in Gold Layer was examining the internal SQL logic line to validate the rules, JOIN conditions, data type handling, and NULL management. This testing existed to ensure the code is produced correctly and maintain robust, scalability, and consistent.

Table 5.5.2.4.1: White Test Cases in Gold Layer Star Schema

Component	Logic Tested	Method	Result
fact_inventory_status	FULL OUTER JOIN captures all materials from both silver view	Verify row count = 20 (all materials represented)	PASS
fact_inventory_status	COALESCE on material_id returns non-NULL for all rows	SELECT COUNT(*) WHERE material_id IS NULL should return 0	PASS
fact_inventory_status	COLLATE fix prevents join collation mismatch error	View runs without error after COLLATE applied on join keys	PASS
fact_inventory_status	ISNULL defaults prevent NULL propagation into flags	SELECT COUNT (*) WHERE is_recoverable_assets IS NULL should return 0	PASS
fact_inventory_status	mc_provision_amount = quantity_on_hand x unit_cost for MC materials	Cross check each MC material: provision / qty = unit_cost from dim_material	PASS
fact_inventory_status	months_since_consumed = 999 for never consumed materials	Verify material with NULL last_consumed_date have months_since_consumed = 999	PASS
fact_sales_impact	Delivery_risk_days formula is correct	Manually verify: DATEDIFF(required_delivery_date, 2026-03-31) - stock_cover_days	PASS

fcct_sales_impact	Only OPEN/ PENDING/ BACKORDER orders included	Verify no CLOSED or COMPLETED status rows appear in fact_sales_impact	PASS
dim_customer	All 7 CSV columns declared in WITH clause	View runs without column mapping error	PASS
dim_date	4 UNION sources collect all unique dates without duplicate	SELECT COUNT (*) vs COUNT (DISTINCT date_id_ must be equal	PASS
dim_warehouse	Header row excluded by WHERE clause	No row with warehouse_location = “warehouse_location” appears	PASS
fact_inventory_status	avg_daily_consumption = 0.001 fallback for zero consumption materials	Verify MAT005, MAT009, MAT010, MAT014, MAT019 show 0.001	PASS
fact_inventory_status	High stock_cover_days for never consumed materials is expected	High values only occur for materials with avg_daily = 0.001	PASS

During white box testing, the five materials were observed which were MAT005, MAT009, MAT010, MAT014, and MAT019 have no consumption history. The avg_daily_consumption in materials defaulted to 0.001 to identify there was no consumption and to prevent any errors that happened.

5.5.2.5 Power BI Data Model and DAX Logic Testing

White box testing was carried out to test Data Model of Power BI internally for DAX logic, relationships in Data Model and measures in the Data Model of Power BI. The following

five top internal logic areas were tested: TREATAS filter context fix, FILTER based KPI Measures, Customers Affected measure, Risk Status calculated column, and the heatmap conditional formatting logic.

The TREATAS fix has been tested and confirmed using the table visual and material_name from dim_material, by comparing raw SUM measures with TREATAS SUM measures to ensure that they are the same. All returned SUMs had the same value, which meant that the relationship path was not being followed. When replacing with TREATAS measures, each row was correctly displayed, and thus the change was the solution to the filter context problem.

The Dead Stock Count and Dead Stock Provision FILTER measures were checked for the second test, by comparing the values displayed on the KPI card with the values shown in the Gold Layer view data. As you can see in the vw_silver_ra_mc_logic single Vinyl Resin row with is_mc_dead_stock = 1, the KPI cards display the values 1 material and \$16,872 respectively.

To verify the Expired Count and Expired Provision measures for third test, filter vw_silver_ra_mc_logic for is_mc_expired equals 1 in Power Query, and verify the row count and provision sum are equal to the KPI card values for 2 materials and \$13,263.

The Customers Affected measure was verified in the 4th test by counting the number of unique values of the Customer_name field in the vw_silver_impacted_sales table where is_stockout_risk equals 1. There are 8 unique customer rows in the Q7 matrix that resulted in 8 customers returned by the measure.

The Risk Status calculated column was verified in Data view, to ensure that all rows with stock_cover_days = 0 had the Risk Status of Critical, and all rows with stock_cover_days > 0 had the Risk Status of At Risk for the fifth test. It was confirmed that the type of the Column was Text and it was shown as Text in the Q6 table visual.

Table 5.5.2.5.1: White Box Testing Results for Power BI Data Model and DAX Logic

Test Area	Internal Logic Tested	Expected	Actual	Status
TREATAS Filter Context	TREATAS maps dim_material filter into analytical views	Per-material values in table visual	Correct per- material values after fix	Passed

Dead Stock Measures	FILTER iterates vw_silver_ra_mc_logic for is_mc_dead_stock = 1	1 material, \$16,872	1 and \$16,872 on KPI cards	Passed
Expired Measures	FILTER iterates vw_silver_ra_mc_logic for is_mc_expired = 1	2 materials, \$13,263	2 and \$13,263 on KPI cards	Passed
Customers Affected	DISTINCT + SELECTCOLUMNS counts unique customers where is_stockout_risk = 1	8 distinct customers	8 displayed on KPI card	Passed
Risk Status Column	IF condition classifies stock_cover_days = 0 as Critical, else At Risk	Critical for 0 cover, At Risk otherwise	Labels display correctly in Q6 table	Passed

5.5.3 User Testing

User testing means how well the system performs in real-world scenarios from the perspective of the team and business stakeholders. This phase focuses on triggering the automated pipelines to validate the scripts accurately apply business and financial rules and ensuring the Power BI dashboards are easy to interpret. Testing the entire workflow from ingestion to visualization is crucial to confirm that the pipeline is reliable, accurate, and ready for day-to-day business operations.

5.5.3.1 Bronze Pipeline Usability Review

User testing for the Bronze Layer was conducted by Member 1 as the pipeline owner, with feedback gathered from Member 2 and Member 3 who depend on the Bronze Layer outputs as inputs to their respective Silver Layer components. Testing was structured around three key interaction points which are triggering the pipeline, monitoring its execution, and locating the output directories in the storage account.

The pipeline trigger was initiated without difficulty through the Trigger Now function in ADF Studio. The ADF Monitor tab provided clear activity-level visibility into execution progress, displaying the status, duration, and result for all six Copy Data activities and the subsequent Data Flow activity. Member 2 and Member 3 were both able to independently locate the bronze/validated/ output directories in the Azure Portal and confirm the correct ABFSS path for use in their respective Synapse Notebooks without requiring assistance from Member 1.

One procedural note was established during testing, when re-triggering the pipeline after a source file update such as the addition of the `lot_expiry_date` column to `inventory_snapshot.csv`, existing files in bronze/validated/ should be deleted prior to re-execution to prevent stale data from persisting alongside the new output. The pipeline re-ran successfully following this procedure without any modification to the pipeline configuration itself. Overall, the Bronze Layer pipeline was found to be operationally clear and accessible to all relevant team members.

5.5.3.2 RA and MC Output Review

User testing was carried out to evaluate the clarity and readiness of the stockout risk and sales impact outputs by rerunning the Silver Layer views with updated mock data using a fixed reference date of 31 March 2026. The output from `vw_silver_stockout_risk` was validated to ensure the key reporting fields such as material names, quantities, recoverable asset, last consumed dates, dead stock, expired stock and provision amount were present on the dashboard. This test run successfully identified 10 recoverable assets, 1 dead stock item, and 2 expired materials and a total provision amount of USD 30144. The query output proved to be clear, complete, and fully ready for integration into the Gold layer and Power BI visualization dashboard.

5.5.3.3 Stockout and Sales Impact Output Review

The user testing was carried out by checking the understanding and appropriateness of the generated stockout and sales impact outputs for reporting purposes. The Silver Layer views were then rerun with the updated mock data, and using the fixed reference date of 31 March 2026. The output of `vw_silver_stockout_risk` was validated to make sure that the important

fields like material name, category, quantity on hand, reorder level, stock cover days, and lead time days were available for dashboard development.

In addition, the output from `vw_silver_impacted_sales_orders` was examined to ensure that impacted customer orders are easily identified. The result contained fields like sales order ID, customer name, finished product, material name, required delivery date and quantity ordered. These fields were deemed appropriate for Power BI dashboard visualisation to be able to analyse stock out risk and customer order impact.

Based on the updated results, the system identified 5 materials below reorder level, 2 materials at stockout risk, 13 impacted sales order-material records, and 8 affected customers. Overall, the query outputs were found to be clear, relevant, and suitable for use in the Gold Layer and dashboard development stage.

5.5.3.4 Gold Layer Data Readiness Review

User testing for the Gold Layer was conducted as a data readiness review to confirm that the Gold Layer objects are accessible, understandable, and ready for consumption by Member 5 (Power BI dashboard) and by the project team. The review involved cross-checking the Gold Layer outputs against the results reported by upstream members and verifying that all required data is available for dashboard development.

The Gold Layer data readiness review confirmed that all required data objects are in place, all business question answers are verified, and the layer is fully ready for Power BI dashboard development.

The Gold Layer was successfully implemented, tested, and validated. All seven business questions were answered correctly and ready to serve the Power BI reporting and dashboard deliverable.

5.5.3.5 Power BI Dashboard Usability Review

The Power BI Dashboard was user tested and feedback was obtained from the project leader and another member of the team who are not technically experienced in the underlying pipeline but viewed the dashboard from the perspective of a business stakeholder.

Testing was broken down into three specific interaction points, entering between the two pages of the dashboard, understanding the data context presented in each KPI card and the

visuals from the graph, and using the visual-level filters to understand the data context presented on each visual.

The flow of pages between Inventory Health and Operational Risk was intuitive and the title of each page gave a clear indication of the business questions being answered. The KPI cards were on each page, and were recognized right away as the headline numbers. Reviewers were able to mention the sum of the MC provision offered, number of stockout risk materials, number of customers without assistance.

There was one usability improvement observed in the review. The reviewers commented that the Q4 donut chart would be a great way to communicate the breakdown of financial exposure of expired material without having to read the table. The Q7 heatmap matrix was also well received and the gradient red colouring enabled reviewers to see the most impacted customers on one glance without having to count individual cells.

A minor issue was found on the Operational Risk page: the initial row and column references for Sum of lead_time_days and Sum of stock_cover_days were also raw data, which resulted in inflated numbers as with the quantity on hand. These were updated with the Lead Time Fixed and Stock Cover Fixed TREATAS measures. The reviewer verified all the values in the table in Q6 after correction.

The overall impression of the dashboard was that it was easy to understand, engaging, and could be shown to business stakeholders as a stand-alone inventory risk reporting tool. The two-page format is noted as an improvement for a 3-page format, where there are a step fewer to navigate between the inventory health and operational risk content.

5.6 Summary

For the chapter 5 has implemented and validated the full PPG Recoverable Assets (RA) and Inventory Risk Management (IRM) data engineering pipeline, starting from Bronze layer ingestion and data quality checks, through Silver layer business logic for RA, Magna

Carta, and stockout risk, up to the Gold layer star schema and Power BI dashboard for inventory analytics. The results of Black Box, White Box, and User Testing confirm that the pipeline correctly applies the defined business rules, produces consistent analytics-ready data, and delivers dashboards that stakeholders can interpret to support financial provisioning and operational decisions. Overall, the work in this chapter demonstrates that the proposed Medallion-based architecture on Microsoft Azure is technically feasible, reliable, and suitable as a foundation for inventory risk analysis at PPG.

REFERENCES

- Bonthu, C. (2025). *Real-Time Data Processing in ERP Systems: Benefits and Challenges*. Journal of Information Systems Engineering and Management, 10(45).
<https://www.jisem-journal.com/index.php/journal/article/download/8889/4102/14822>
- Codiin. (2024). *Medallion Architecture: Bronze, Silver, Gold*.
<https://www.codiin.com/data-engineering/articles/medallion-architecture.html>
- CloudWebSchool. (t.t.). *Data Lake Architectures in Azure*.
<https://cloudwebschool.com/docs/azure/data-and-analytics/data-lake-architectures/>
- Coates, M. (2026). *Essentials of Azure Data Lake Storage Gen2*.
<https://www.scribd.com/document/787953513/EssentialsOfAzureDataLakeStorageGen2-MelissaCoates>
- Das, S., Grbic, M., Ilic, I., Jovandic, I., Jovanovic, A., Narasayya, V. R., Radulovic, M., Stikic, M., Xu, G., & Chaudhuri, S. (2019). Automatically indexing millions of databases in Microsoft Azure SQL Database. *Storage & Indexing*, 666–679.
<https://doi.org/10.1145/3299869.3314035>
- Data Studios. (2026). *How inventory write-downs affect profitability and financial ratios*.
<https://www.datastudios.org/post/how-inventory-write-downs-affect-profitability-and-financial-ratios>
- Global, C. (2025, March 11). *Microsoft Azure & AI: Optimizing Retail Supply Chains*. Charter Global. <https://charterglobal.com/microsoft-azure-for-retail-supply-chain/>
- Gentyala, R. (2024). From Bronze to Broken: A Grounded Theory Study of Anti-Patterns and Accruing Data Debt in Medallion Lakehouse Deployments. *European Journal of Advances in Engineering and Technology*, 11(1), 90-100.

Kancharla, S. S. R. (2025). ETL vs ELT: Evolving Approaches to Data Integration. *Journal Of Engineering And Computer Sciences*, 4(7), 1065-1074.

Krystofik, M., Valant, C. J., Archbold, J., Bruessow, P., & Nenadic, N. G. (2020). Risk Assessment Framework for Outbound Supply-Chain Management. *Information*, 11(9), 417. <https://doi.org/10.3390/info11090417>

Microsoft. (2023). *Inventory Visibility Power BI dashboard – Dynamics 365 Supply Chain Management documentation*. <https://learn.microsoft.com/en-us/dynamics365/supply-chain/inventory/inventory-visibility-dashboard>

Normey13, S., Etcheverry, L., Marotta, A., & Consens, M. P. Findings from Two Decades of Research on Schema Discovery using a Systematic Literature.

Krantz, T., & Jonker, A. (2024). *What is a data architecture?* IBM Think. <https://www.ibm.com/think/topics/data-architecture>